Causality-Aware Query Answering on Incomplete Data

Lubna Alzamil
Oregon State University
Corvallis, Oregon, USA
alzamill@oregonstate.edu

Arash Termehchy
Oregon State University
Corvallis, Oregon, USA
termehca@oregonstate.edu

Karthika Mohan
Oregon State University
Corvallis, Oregon, USA
mohank@oregonstate.edu

Abstract

Missing data are ubiquitous in databases. Data missingness often depends on observed or unobserved data in the database, i.e., *causes of missingness*. Current approaches to querying incomplete databases ignore these causes and, as a result, return biased or inaccurate answers. We propose a novel semantics of query answers over incomplete data that considers causes of missingness. Using this semantics, we propose efficient query answering algorithms to return accurate and unbiased results on incomplete data. When the causal structure of data missingness is too complex, it is challenging to answer queries efficiently. We propose to minimally repair the data to restore tractability. We show that this problem is NP-hard and propose efficient approximation algorithms. Our extensive empirical study shows that our query answering algorithms are more accurate and efficient than current algorithms and that our repair algorithms efficiently approximate the minimal repair.

1 Introduction

Databases often contain missing values. Answering queries on data with missing values is an important problem in data management [1, 16, 32, 77]. One method of answering queries on incomplete data is to find the correct values of the missing data and answer queries on the *repaired database* (repair). However, this usually requires significant time and effort [13, 28, 59]. It also obscures the uncertainty in the results arising from missing values, which is often crucial for users to know. Thus, researchers have proposed methods to answer queries directly on incomplete data [1, 16–18, 23, 39, 76].

Data missingness often depends on observed or unobserved data in the database, i.e., *causes of missingness* [27, 40, 45, 62].

Example 1.1. In some settings, people with high-paying jobs may not often report their income. For example, consider the relation *jobs* in Table 1b that has some missing values in attribute *income*. The missingness in attribute *income* depends on the values in attribute *job*. Missing data appear more often in tuples with high-paying jobs, e.g., *Executives*, than in tuples with other values of *job*.

Table 1: An incomplete relation and its complete version

(a) A complete *jobs* relation

ID	*job*	*skills*	*income*
1	Assistant	Basic	110K
2	Consultant	Expert	100K
3	Consultant	Basic	70K
4	Executive	Expert	500K
5	Executive	Basic	800K
6	Executive	Expert	900K

(b) An incomplete *jobs* relation

ID	*job*	*skills*	*income*
1	Assistant	Basic	110K
2	Consultant	Expert	$\perp_1$
3	Consultant	$\perp_3$	70K
4	Executive	Expert	500K
5	Executive	$\perp_4$	$\perp_2$
6	Executive	$\perp_5$	$\perp_6$

Current methods of answering queries on incomplete data ignore the causes of missingness. Thus, they often return biased and inaccurate results. For example, given a query that returns the salaries in Table 1b, the SQL semantics, i.e., *three-valued logic*, ignores the missing salaries and returns the observed ones, which are biased toward relatively low-paying jobs. The semantics of *certain answers* returns only the results that hold over all possible repairs of the database [1, 15, 16, 38, 39]. As it does not consider the causes of missingness, it returns the same biased answers in our example.

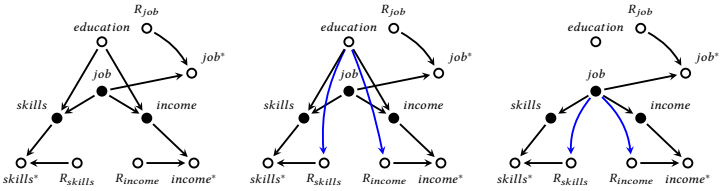
One approach to this problem is to extend current query answering methods to consider the causes of missingness. However, these methods provide unclear semantics, may not return informative answers, or are difficult to implement on current database systems. SQL semantics often returns confusing and contradictory results [16, 20, 38]. Due to the restrictive nature of certain answers, many queries, e.g., queries with aggregation functions, have empty certain answers. One must often substantially modify existing database systems to compute certain answers [10, 24]. One can return the answers of the query on all possible repairs, i.e., *possible answers* [17, 18, 76]. As an incomplete database usually has numerous possible repairs, possible answers are often uninformative [76]. Researchers model uncertainties of missing values using input probability distributions and evaluate queries in the resulting probabilistic database [18]. However, the probabilities of missing values are generally not available [40, 55]. Moreover, these methods do not consider the causes of missingness. There is also no implementation of these methods available.

In this paper, we propose a formal semantics for answering queries on incomplete data that considers the causes of missingness. Users declare the causes of data missingness in a high-level representation, which can also be learned from the data [45, 67]. We define the answer of a query on an incomplete database as a probability space over the results of the query on the space of database repairs that comply with the given causal structure. Since the probabilities of database repairs are not usually available, query answering algorithms should estimate them, e.g., from the observed data. Thus, we define consistency, i.e., that its results converge to the true probabilities as the observed data grows, and a confidence interval, i.e., the returned interval that contains the true answers with high likelihood, for a query answering algorithm. We propose efficient consistent query evaluation algorithms with informative confidence intervals. Our contributions:

- We propose a formal semantics for query answers on incomplete data as a probability space over the data repairs that comply with the given causes of missingness. We define consistency and confidence of query answering for this semantics (Section 3).
- We propose efficient algorithms that estimate the results of queries on a database using observed data by rewriting the input query. We show that our algorithms are consistent and provide

ID	*education*	*job*	R_{job}	*job**	*skills*	R_{skills}	*skills**	*income*	R_{income}	*income**
1	Grad	$\perp_3$	1	M	Basic	0	Basic	30K	0	30K
2	Undergrad	Assistant	0	Assistant	Basic	0	Basic	$\perp_2$	1	M
3	Grad	Assistant	0	Assistant	$\perp_1$	1	M	110K	0	110K
4	Undergrad	Assistant	0	Assistant	Expert	0	Expert	110K	0	110K
5	Grad	Executive	0	Executive	Expert	0	Expert	100K	0	100K
6	Grad	Executive	0	Executive	Expert	0	Expert	$\perp_2$	1	M
7	Grad	Executive	0	Executive	Expert	0	Expert	400K	0	400K
8	Undergrad	$\perp_3$	1	M	$\perp_1$	1	M	900K	0	900K
9	Grad	$\perp_3$	1	M	Basic	0	Basic	100K	0	100K
10	Undergrad	Consultant	0	Consultant	Expert	0	Expert	120K	0	120K
11	Grad	Consultant	0	Consultant	$\perp_1$	1	M	$\perp_2$	1	M
12	Grad	Consultant	0	Consultant	Basic	0	Basic	130K	0	130K
13	Grad	Consultant	0	Consultant	Basic	0	Basic	$\perp_2$	1	M
14	Undergrad	Consultant	0	Consultant	$\perp_1$	1	M	90K	0	90K

(a) Augmented *jobs* relation



(b) MCAR **(c) MAR** **(d) MNAR**

Figure 1: An augmented relation and some possible m-graphs. Solid nodes represent incomplete attributes. Starred nodes represent their proxy attributes. The other hollow nodes represent complete attributes and missingness mechanisms. Blue edges into missingness mechanisms R indicate the cause of missingness.

informative and high-likelihood confidence intervals. They run on database systems without modifying their internals and support marked and non-repeating nulls [1] (Sections 7 and 8).

- When the causal structure of data missingness is too complex, query answering algorithms cannot efficiently return answers. In this case, we propose to repair a minimal subset of data to make efficient query answering possible. We show that this problem is generally NP-hard and propose an efficient approximation algorithm with a guaranteed performance ratio (Section 9).
- We conduct an extensive empirical study on several real-world datasets. Our results indicate that our algorithms are more efficient and return more accurate and informative answers than current query answering methods. They also show that our proposed repair algorithm accurately approximates the minimal subset of the data to repair efficiently (Section 10).

The proofs and details of the empirical study are in [4].

2 Background

Database and Query. We fix a finite set of relation schemas. Each relation schema $\mathcal{T}$ contains a finite set of attribute symbols. We associate every attribute $A \in \mathcal{T}$ with a domain $\text{Dom}(A)$. A relation instance (relation) T over $\mathcal{T}$ is any finite subset of the Cartesian product of the domains of attributes in $\mathcal{T}$. The set of values of an attribute in relation T is the attribute instance (attribute for short) in T. The set of distinct observed (non-missing) values of an attribute A in relation T is the *active domain* of A in T, denoted $\text{adom}(A)$. The active domain of a set of attributes $\mathbf{A}$ is $\text{adom}(\mathbf{A}) = \{\mathbf{a} \mid \exists t \in T, t[\mathbf{A}] = \mathbf{a}\}$, where every value in $\mathbf{a}$ is observed (non-missing). For example, when $\mathbf{A} = \{A_1, A_2\}$, then $\text{adom}(\mathbf{A}) = \{(a_1, a_2) \mid \exists t \in T, t[A_1] = a_1, t[A_2] = a_2\}$. We denote the set of all attributes in relation T as $\mathbf{A}_T$ and $\mathbf{A}$ when T is clear from the context. A database schema $\mathcal{S}$ is a finite collection of relation schemas. A database instance (database) of $\mathcal{S}$ is a mapping that assigns to each $\mathcal{T} \in \mathcal{S}$ a relation T that is an instance of $\mathcal{T}$. The query q on the database schema $\mathcal{S}$ is a function from instances of $\mathcal{S}$ to relations of a single fixed schema. We consider queries that contain relational algebra operators (σ, π, $\bowtie$, $\cup$, $\cap$, $\setminus$), `GROUP BY` (γ), and aggregation functions.

Incomplete Data. The domain of each attribute contains a set of distinct members $\perp_i$, $1 \leq i \leq \ell$, which denote missing values (marked nulls) [1, 16]. This approach generalizes SQL null, i.e., *non-repeating nulls*, in which each missing value represents a distinct value in the domain [1]. Our algorithms support both marked and non-repeating nulls. For brevity, we denote the set of observed (complete) values in Y as $Y \neq \perp$ and the fact that value v is observed as $v \neq \perp$. A tuple or attribute is *incomplete* if it has at least one missing value. A relation is incomplete if it has an incomplete tuple, and a database is incomplete if it has an incomplete relation. They are *complete* otherwise. We denote the set of complete attributes in a relation T as $\mathbf{A}_o$ and the set of incomplete attributes as $\mathbf{A}_m$. Unless stated otherwise, we assume that every relation in an incomplete database is incomplete; our results extend to other cases. We refer to an incomplete relation/database as a relation/database. We denote the distinct null symbols in relation T by $\text{Null}(T) = \{\perp_1, \ldots, \perp_\ell\}$.

Bayesian Network. A *Bayesian network* over a relation T is a DAG (directed acyclic graph) $G_T = \langle \mathcal{V}, \mathcal{E} \rangle$ such that each node in $\mathcal{V}$ corresponds to an attribute of T, and the edges in $\mathcal{E}$ indicate probabilistic dependencies. For example, consider the relation T with attributes X and Y. The Bayesian network over T with an edge $X \longrightarrow Y$ indicates that Y is conditionally dependent on X.

Missingness Graph. To formalize the causes of missingness in a relation T, we use *missingness graphs* that augment the Bayesian network of T with missingness mechanisms and proxy attributes [45]. For each incomplete attribute Y in T, we augment the relation T with a missingness mechanism attribute $R_Y \in \{0, 1\}$ where $R_Y = 0$ means Y is observed and $R_Y = 1$ means Y is missing. Let M be a special value that denotes missingness. For each incomplete attribute Y, the m-graph also contains a proxy attribute Y^* that equals Y when $R_Y = 0$ and equals M when $R_Y = 1$ [45]. The m-graph contains the edges $Y \longrightarrow Y^*$ and $R_Y \longrightarrow Y^*$. For the remainder of the paper, we do not show the proxy attributes in m-graphs to keep the figures simple [45]. We denote the set of missingness mechanisms as $R_{\mathbf{A}_m}$. The attributes in $\mathbf{A}_o \cup R_{\mathbf{A}_m}$ and the proxy attributes are complete. The attributes in $\mathbf{A}_m$ remain incomplete in the augmented relation. The resulting Bayesian network is the *missingness graph* (m-graph) of T [45, 46].

Example 2.1. Figure 1a is an example of an augmented relation. The mechanism R_{income} indicates whether *income* is observed ($R_{income} = 0$) or missing ($R_{income} = 1$). The same applies to the *skills* and *job* attributes. The m-graphs also include the complete attribute *education*. The proxy attributes are *job**, *skills**, and

$income^*$. Hence, the incomplete attributes are $\{job, income, skills\}$. The proxy attributes, the missingness mechanisms, and *education* are complete.

We do *not* materialize the augmented relation and use it only to define the m-graph for a relation. The missingness graph of the relation T constrains the joint distribution over the augmented attribute set of T, denoted by $\mathbb{P}_T$. The tuples of T are drawn independently from $\mathbb{P}_T$. This independence concerns the generation of T and does not make tuples sharing a marked null independent in the repair probability space. The m-graphs are either provided by the user or learned from the data [67]. For the rest of the paper, we assume that each relation has a distinct m-graph and that tuples from distinct relations are generated independently according to the distributions constrained by their m-graphs.

Types of Missingness. The m-graph of a relation models the causes of missingness in the relation. For example, if the missingness is random (e.g., due to a power outage), the missingness mechanism would be a root node in the m-graph. More precisely, the values of an attribute in a relation T are **Missing Completely at Random** (MCAR) if the reason for their missingness is random and independent of other attributes in T [45, 62]. Formally, given the m-graph G_T, an incomplete attribute Y is an **MCAR attribute** if its missingness mechanism R_Y has no parents in G_T, i.e., all attributes in $\mathbf{A}_T$ are independent of the missingness mechanism R_Y ($\mathbf{A}_T \perp\!\!\!\perp R_Y$). For example, the m-graph in Figure 1b shows that *income* is an MCAR attribute, where all attributes in the m-graph are independent of R_{income}.

Example 2.2. One can estimate the distribution of the *income* attribute in Figure 1b by

$$\begin{aligned}\mathbb{P}_{Jobs}(income) &= \mathbb{P}_{Jobs}(income)\\ &= \mathbb{P}_{Jobs}(income^* | R_{income} = 0)\end{aligned}$$

Missingness in some attributes in a tuple may depend on the values of complete attributes in the tuple. For example, in the *jobs* relation in Table 1b, income data are missing more often for relatively high earners than for relatively low earners, which makes the probability of missing income of a tuple dependent on its job. This type is called *Missing at Random* (MAR). Formally, given the m-graph G_T, an incomplete attribute Y of T is an **MAR attribute** if it is conditionally independent of its missingness mechanism R_Y given some complete attributes in T, i.e., the missingness mechanism R_Y has only complete parents. For example, the m-graph in Figure 1c shows that *income* is an MAR attribute and its missingness depends on *education*.

If an attribute is incomplete and neither MCAR nor MAR, its data are Missing Not At Random (MNAR). That is, given the m-graph G_T, an incomplete attribute Y is an **MNAR attribute** if its missingness mechanism R_Y has incomplete parents $\mathbf{X} \subseteq \mathbf{A}_m$ in G_T, i.e., the cause of missingness contains incomplete attributes.

These three mechanisms differ in how much the missingness mechanism may depend on other attributes: MCAR allows no such dependence, MAR allows dependence only on complete attributes, and MNAR allows dependence on incomplete attributes.

D-separation and Separating Sets. The *d-separation* criterion ("d" denotes directional) determines whether a set of attributes $\mathbf{Y}$ is *independent* of another set of attributes $\mathbf{Z}$ given a third set of attributes $\mathbf{X}$ in a Bayesian network ($\mathbf{Y} \perp\!\!\!\perp \mathbf{Z} \mid \mathbf{X}$) [54]. Formally, given a Bayesian network G_T, a path between two attributes A and B in G_T is *d-separated* by a set of attributes $\mathbf{X}$ if: (1) the path has a chain $A \to C \to B$ or a fork $A \leftarrow C \to B$ where attribute $C \in \mathbf{X}$, or (2) the path has a collider $A \to C \leftarrow B$ where neither C nor its descendants are in $\mathbf{X}$. Two sets of attributes $\mathbf{Y}$ and $\mathbf{Z}$ are d-separated by $\mathbf{X}$ if every path between any attribute in $\mathbf{Y}$ and any attribute in $\mathbf{Z}$ is d-separated by $\mathbf{X}$. In this case, $\mathbf{X}$ is defined as the **separating set** such that $\mathbf{Y} \perp\!\!\!\perp \mathbf{Z} \mid \mathbf{X}$. A **minimal separating set** is a separating set such that none of its proper subsets is a separating set. We refer to a minimal separating set as a separating set, unless otherwise noted. We can compute the separating set of input attribute sets by d-separation tests on G_T in $O(|\mathcal{V}|^2)$ time [71].

Example 2.3. The m-graph in Figure 1c illustrates MAR missingness where the attribute *education* is a fork. Thus, conditioning on the separating set containing the single attribute $\{education\}$ separates the incomplete attribute *income* from its mechanism R_{income}, making them independent: ($income \perp\!\!\!\perp R_{income} \mid education$).

Factorization. When there is no additional information available, it is natural to use the observed data in a database to estimate the uncertainty of its missing data [13, 40, 55]. As illustrated in Section 1, using observed data for an MAR or MNAR attribute directly may yield biased estimates. To eliminate this bias and obtain a consistent estimator, we find an attribute X such that $Y \perp\!\!\!\perp R_Y \mid X$ in G_T. This independence gives $\mathbb{P}_T(Y \mid X) = \mathbb{P}_T(Y^* \mid R_Y = 0, X)$. In the remainder of the paper, we use $Y \neq \perp$ in place of $R_Y = 0$ when selecting the observed values of Y. Thus, for each value of X, the tuples satisfying $Y \neq \perp$ estimate $\mathbb{P}_T(Y \mid X)$. Therefore, we can estimate $\mathbb{P}_T(Y) = \sum_X \mathbb{P}_T(Y \mid X)\mathbb{P}_T(X)$ from the observed data if X is complete. If X is incomplete, for the same reason, an accurate estimation requires $Y \perp\!\!\!\perp R_Y, R_X \mid X$. Since X is incomplete, we also find an attribute Z such that $X \perp\!\!\!\perp R_X \mid Z$ in G_T. Assume first that Z is complete and $Y \perp\!\!\!\perp Z \mid X$. Using this information, we can reliably estimate $\mathbb{P}_T(Y) = \sum_{X,Z} \mathbb{P}_T(Y \mid X)\, \mathbb{P}_T(X \mid Z)\, \mathbb{P}_T(Z)$ from the observed data. If Z is incomplete, we apply the same process to estimate $\mathbb{P}_T(Z)$. Following [45], estimating the distribution of an incomplete attribute uses an order in which each incomplete attribute is either MCAR or has a separating set among the attributes that follow it in the order. This order specifies how the joint distribution decomposes into conditionals that are estimated separately.

Formally, given the set of attributes $\mathbf{X}$ in the m-graph G_T, we define the set of missingness mechanisms of $\mathbf{X}$, $R_\mathbf{X}$, as $\{R_X \mid X \in \mathbf{X} \cap \mathbf{A}_m\}$. Given the set of attributes $\mathbf{Y} \subseteq \mathbf{A}_m$, we define the **factorization attributes** of $\mathbf{Y}$, $\mathbf{F_Y}$, recursively as the smallest set that contains $\mathbf{Y}$ and the separating sets of every incomplete attribute in $\mathbf{F_Y}$. A **factorization** for $\mathbf{Y}$, denoted as $\mathcal{F}(\mathbf{Y})$, is a sequence of pairs $[(\mathbf{Z}_1, \mathbf{X}_1), \ldots, (\mathbf{Z}_n, \mathbf{X}_n)]$ such that $\mathbf{Z}_i$, $1 \leq i \leq n$, are disjoint subsets of $\mathbf{A}_T$, $\mathbf{Y} \subseteq \bigcup_{1 \leq i \leq n} \mathbf{Z}_i$, and each $\mathbf{X}_i \subseteq \bigcup_{i+1 \leq j \leq n} \mathbf{Z}_j$ is a minimal set satisfying the two conditions $\mathbf{Z}_i \perp\!\!\!\perp (\bigcup_{i+1 \leq j \leq n} \mathbf{Z}_j \setminus \mathbf{X}_i) \mid \mathbf{X}_i$ and $\mathbf{Z}_i \perp\!\!\!\perp (R_{\mathbf{Z}_i}, R_{\mathbf{X}_i}) \mid \mathbf{X}_i$ [45]. The first independence reduces the chain-rule factor $\mathbb{P}_T(\mathbf{Z}_i \mid \bigcup_{i+1 \leq j \leq n} \mathbf{Z}_j)$ to $\mathbb{P}_T(\mathbf{Z}_i \mid \mathbf{X}_i)$, and the second condition makes each factor estimable from the observed data. Given a factorization $[(\mathbf{Z}_1, \mathbf{X}_1), \ldots, (\mathbf{Z}_n, \mathbf{X}_n)]$, we have $\mathbb{P}_T(\mathbf{Y}) = \sum_\mathbf{V} \prod_{i=1}^n \mathbb{P}_T(\mathbf{Z}_i \mid \mathbf{X}_i)$, where $\mathbf{V} = \bigcup_{1 \leq i \leq n} \mathbf{Z}_i \setminus \mathbf{Y}$. Factorization marginalizes the product over the eliminated attributes $\mathbf{V}$. We call $\mathbf{Y}$ **factorizable** and each $\mathbb{P}_T(\mathbf{Z}_i \mid \mathbf{X}_i)$ a **factor**.

Example 2.4. Consider the relation *jobs* in Figure 1a with m-graph in Figure 1d, where *income* is MNAR with the separating set $\{job\}$ and *job* is MCAR. The sequence $[(\{income\}, \{job\}), (\{job\}, \emptyset)]$ is a factorization for $\mathbf{Y} = \{income\}$. The set $\mathbf{X}_1 = \{job\}$ satisfies $income \perp\!\!\!\perp R_{income}, R_{job} | job$, and $\mathbf{X}_2 = \emptyset$ satisfies $job \perp\!\!\!\perp R_{job}$. With $\mathbf{V} = \{job\}$, $\mathbb{P}_T(income) = \sum_{job} \mathbb{P}_T(income \mid job)\, \mathbb{P}_T(job)$. Both factors are estimated from the observed data.

The MCAR and MAR attributes are always factorizable, but some MNAR attributes are not factorizable [45].

3 Semantics of Query Answering

Repairs. A repair of an incomplete relation T replaces every distinct null symbol $\perp_i \in \text{Null}(T)$ with a single value in the domain of its attribute, producing a complete relation. All occurrences of $\perp_i$ in T take this value. We denote the set of repairs for the relation T by $\Delta(T)$. The definition of repairs naturally extends to databases. Following current approaches in data management, ML, and statistics, we assume that an incomplete database represents a probability distribution over the set of its repairs [18, 40, 55]. Given the database $\mathbf{T}$, we define a *probability space* $\mathcal{D}(\mathbf{T}) = (\Delta(\mathbf{T}), \Sigma_{\mathbf{T}}, \mathsf{P}_{\mathbf{T}})$, where $\Sigma_{\mathbf{T}}$ is a family of subsets of $\Delta(\mathbf{T})$ that contains $\Delta(\mathbf{T})$ and is closed under complement and countable unions (σ-algebra on $\Delta(\mathbf{T})$). The probability measure $\mathsf{P}_{\mathbf{T}} : \Sigma_{\mathbf{T}} \to [0, 1]$ satisfies $\mathsf{P}_{\mathbf{T}}(\emptyset) = 0$, $\mathsf{P}_{\mathbf{T}}(\Delta(\mathbf{T})) = 1$, and countable additivity: for any countable sequence of pairwise disjoint $\mathcal{A}_i \in \Sigma_{\mathbf{T}}$, $\mathsf{P}_{\mathbf{T}}(\bigcup_{i=1}^{n} \mathcal{A}_i) = \sum_{i=1}^{n} \mathsf{P}_{\mathbf{T}}(\mathcal{A}_i)$.

Query Answers. The set of possible results for query q and database $\mathbf{T}$ is $\Omega_{q(\mathbf{T})} = \{q(\mathbf{T}') \mid \mathbf{T}' \in \Delta(\mathbf{T})\}$. We define a probability space over $\Omega_{q(\mathbf{T})}$ as follows. Let $\Sigma_{(q,\mathbf{T})}$ be the collection of subsets of $\Omega_{q(\mathbf{T})}$ consisting of all $\mathcal{A} \subseteq \Omega_{q(\mathbf{T})}$ such that $\{\mathbf{T}' \in \Delta(\mathbf{T}) \mid q(\mathbf{T}') \in \mathcal{A}\} \in \Sigma_{\mathbf{T}}$. $\Sigma_{(q,\mathbf{T})}$ is a σ-algebra [18]. The probability measure $\mathsf{P}_{q(\mathbf{T})}$ on $\Omega_{q(\mathbf{T})}$ is induced by the probability measure in $\mathcal{D}(\mathbf{T})$: for any measurable set of answers $\mathcal{A} \in \Sigma_{(q,\mathbf{T})}$, $\mathsf{P}_{q(\mathbf{T})}(\mathcal{A}) = \mathsf{P}_{\mathbf{T}}\big(\{\mathbf{T}' \in \Delta(\mathbf{T}) \mid q(\mathbf{T}') \in \mathcal{A}\}\big)$. The *answer* of a query q on the database $\mathbf{T}$, $q(\mathbf{T})$, is the probability space $\mathcal{D}_q(\mathbf{T}) = \big(\Omega_{q(\mathbf{T})}, \Sigma_{(q,\mathbf{T})}, \mathsf{P}_{q(\mathbf{T})}\big)$.

Query Estimation. To answer queries on a database precisely, we should know the probability space the database represents, which in turn requires knowing the uncertainties of missing values, e.g., probability distribution for each missing value. However, this information is usually not available [13, 21, 27, 40, 41, 55, 62]. Even if they were available, exact answers would still be challenging to obtain because databases generally have numerous possible repairs [18]. Thus, we design algorithms that deliver accurate estimates of the query results given the observed data in the incomplete database and its m-graph. Given q, a **query estimator** $\hat{q}$ returns for database $\mathbf{T}$ a probability space $\mathcal{D}_{\hat{q}}(\mathbf{T}) = \big(\Omega_{q(\mathbf{T})}, \Sigma_{(q,\mathbf{T})}, \mathsf{P}_{\hat{q}(\mathbf{T})}\big)$. Since the probabilities of repairs are usually not available, in the absence of additional information, researchers and practitioners in data management, ML, and statistics often use the observed data in an incomplete dataset to estimate its uncertainties [13, 21, 27, 40, 41, 55, 62]. Hence, to answer queries, a reasonable approach is to estimate query results using the observed data in the database. Ideally, as we obtain more observed data, a query estimator should return more accurate estimates and quantify their uncertainty more precisely. Next, we formalize these properties.

Consistency. Let $\nu(T)$ denote the set of tuples with at least one missing value in T, and let $\kappa(T) = T \setminus \nu(T)$ denote the set of complete tuples. A relation T' is an *augmentation* of relation T if: (i) T and T' share the same schema and m-graph, (ii) T and T' have the same active domains, (iii) $\nu(T) \subseteq \nu(T')$ and $\kappa(T) \subseteq \kappa(T')$, and (iv) T' has more tuples than T ($|T'| > |T|$). Given databases $\mathbf{T}'$ and $\mathbf{T}$ with the same schema, $\mathbf{T}'$ is an augmentation of $\mathbf{T}$ if each relation in $\mathbf{T}'$ is an augmentation of its corresponding relation with the same schema in $\mathbf{T}$. A *sequence of databases* $\{\mathbf{T}_n\}_{n=1}^{\infty}$ is a set of databases where $\mathbf{T}_{n+1}$ is an augmentation of $\mathbf{T}_n$.

Definition 3.1. The query estimator $\hat{q}$ is consistent if for every sequence of databases $\{\mathbf{T}_n\}_{n=1}^{\infty}$ and every $\mathcal{A} \in \Sigma_{(q,\mathbf{T}_n)}$:
$\lim_{n\to\infty} P\Big(|\mathsf{P}_{\hat{q}(\mathbf{T}_n)}(\mathcal{A}) - \mathsf{P}_{q(\mathbf{T}_n)}(\mathcal{A})| > \epsilon\Big) = 0, \quad \forall \epsilon > 0.$

Intuitively, Definition 3.1 states that for sufficiently large observed data, the estimator's measure $\mathsf{P}_{\hat{q}(\mathbf{T}_n)}(\mathcal{A})$ converges to the true measure $\mathsf{P}_{q(\mathbf{T}_n)}(\mathcal{A})$ for each measurable set of answers $\mathcal{A}$. The estimator's measure is a random variable whose uncertainty comes from the random selection of $\mathbf{T}_n$.

Bounding Uncertainty. The consistency of a query estimator does not account for the uncertainty introduced by the limited observed data. We associate estimators with confidence intervals to quantify their uncertainty.

Definition 3.2. Given a query q and real value $0 < \alpha < 1$, let $u(\mathbf{T}_n, q, \mathcal{A}, \alpha)$ be a function whose range is $[0, 1]$. The **confidence interval** of the query estimator $\hat{q}$ is $u(\mathbf{T}_n, q, \mathcal{A}, \alpha)$ if for every sequence of databases $\{\mathbf{T}_n\}_{n=1}^{\infty}$ and every $\mathcal{A} \in \Sigma_{(q,\mathbf{T}_n)}$, we have:

$$\lim_{n\to\infty} P\Big(\mathsf{P}_{q(\mathbf{T}_n)}(\mathcal{A}) \in [\mathsf{P}_{\hat{q}(\mathbf{T}_n)}(\mathcal{A}) \pm u(\mathbf{T}_n, q, \mathcal{A}, \alpha)]\Big) = 1 - \alpha.$$

Convergent. If a query has many possible answers and each answer has a relatively high probability, it may be difficult for the user to interpret the answer. Hence, a desirable case is that as the amount of observed data grows, almost all of the probability mass becomes concentrated on a single possible answer (a member of $\Omega_{q(\mathbf{T})}$). In this case, for sufficiently large data, we can provide easy to understand results for users.

Definition 3.3. Given a query q, a consistent query estimator $\hat{q}$ is **convergent** if for every sequence of databases $\{\mathbf{T}_n\}_{n=1}^{\infty}$ and some event $\mathcal{A} = \{a\} \in \Sigma_{(q,\mathbf{T}_n)}$ for a possible answer $a \in \Omega_{q(\mathbf{T}_n)}$: $\lim_{n\to\infty} P\Big(|1 - \mathsf{P}_{\hat{q}(\mathbf{T}_n)}(\mathcal{A})| > \epsilon\Big) = 0, \quad \forall \epsilon > 0$. We call a the *convergent answer* of $\hat{q}$. The convergent answer a is independent of n. It is the answer on which $\mathsf{P}_{\hat{q}(\mathbf{T}_n)}$ concentrates as $n \to \infty$.

It may not be possible to find a convergent estimator for some queries, as the answer space may contain too many possible answers with relatively high probability over every database. Next, we represent the results of a convergent query estimator in a way that illustrates its uncertainty over limited data.

Definition 3.4. Given a query q with convergent estimator $\hat{q}$ and value $0 < \alpha < 1$, let $u(\mathbf{T}_n, q, \alpha)$ be a function whose range is $\Sigma_{(q,\mathbf{T}_n)}$. The **convergent confidence interval** of $\hat{q}$ is $u(\mathbf{T}_n, q, \alpha)$ if for every sequence of databases $\{\mathbf{T}_n\}_{n=1}^{\infty}$ and the convergent answer a of $\hat{q}$ with event $\mathcal{A} = \{a\} \in \Sigma_{(q,\mathbf{T}_n)}$: $\lim_{n\to\infty} P(\mathcal{A} \subseteq u(\mathbf{T}_n, q, \alpha)) = 1 - \alpha$.

For small α, the interval contains the convergent answer with high probability. As an example, if the possible results of q are real values (e.g., q is an aggregation query), then $u(\mathbf{T}_n, q, \alpha)$ is an interval of real numbers centered at the estimate.

4 Factorization Query

Precomputing Factorizations. The factorization of a set of attributes provides a plan for computing the joint probabilities of (subsets of) those attributes. We precompute the factorization of the set of attributes in $\mathbf{A}_m$ for each relation T. At query time, we marginalize the attributes that are not needed to evaluate q. We find the separating set of each attribute in $\mathbf{A}_m$ and its missingness mechanism using the m-graph of T. We continue this step iteratively by finding the separating sets of the separating sets in the preceding step until no new separating set is found. Then, we generate and check possible factorizations $[(\mathbf{Z}_1, \mathbf{X}_1), \ldots, (\mathbf{Z}_n, \mathbf{X}_n)]$ where $\mathbf{A}_m \subseteq \bigcup_{1 \le i \le n} \mathbf{Z}_i$ and each $\mathbf{X}_i$, $1 \le i \le n$, contains separating sets of attributes in $\mathbf{Z}_i$, such that $\mathbf{Z}_i \perp\!\!\!\perp (R_{\mathbf{Z}_i}, R_{\mathbf{X}_i}) \mid \mathbf{X}_i$. However, the current $\mathbf{X}_i$ may not make $\mathbf{Z}_i$ independent of the remaining attributes in the factorization. In this case, we add attributes from the m-graph to later sets $\mathbf{Z}_j$ and include them in $\mathbf{X}_i$ until $\mathbf{Z}_i$ becomes independent of the remaining attributes in the factorization given $\mathbf{X}_i$. We check both conditions of a factorization using the d-separation test. We backtrack as soon as the test fails and return a factorization if it is found. The resulting $\mathbf{X}_i$ satisfies both conditions but is not necessarily a minimal set that satisfies them. A separating set for one attribute may also be the separating set for another attribute in $\mathbf{Z}_i$. We remove an attribute from $\mathbf{X}_i$ when both conditions hold without it. We repeat this step until no attribute can be removed. Each check is one d-separation test. The worst-case time complexity of this algorithm is exponential in the number of incomplete attributes. Nevertheless, it is practical for a pre-processing task, as the number of incomplete attributes in a relation is often relatively small. There are also heuristics to speed up computation, e.g., discarding candidates that cannot yield a factorization early on [45].

Optimizing Factorization. If each subset of factors in a factorization refers to a disjoint subset of $\mathbf{V}$, we can reorganize the factorization into a set of *independent sums* to reduce computation, e.g., $\sum_{X_1,X_2} \mathbb{P}_T(Z_1 \mid X_1)\,\mathbb{P}_T(Z_2 \mid X_2) = \big(\sum_{X_1} \mathbb{P}_T(Z_1 \mid X_1)\big)\big(\sum_{X_2} \mathbb{P}_T(Z_2 \mid X_2)\big)$ gives two independent sums, which we can evaluate separately. Given a factorization $\sum_{\mathbf{V}} \Pi_i \mathbb{P}_T(\mathbf{Z}_i \mid \mathbf{X}_i)$, let $\mathbf{V}_1, \ldots, \mathbf{V}_m$ and $\mathbf{F}_1, \ldots, \mathbf{F}_m$ be a collection of disjoint and collectively exhaustive subsets of $\mathbf{V}$ and factors in the factorization, respectively, such that the attributes in $\mathbf{V}_i$, $1 \le i \le m$, appear only in the factors of $\mathbf{F}_i$. We can write this factorization as a set of independent sums $\Pi_{1 \le i \le m} \sum_{\mathbf{V}_i} \Pi_{1 \le j \le \kappa(i)} \mathbf{f}_j^i$ where $\mathbf{f}_j^i \in \mathbf{F}_i$ and $\kappa(i)$ is the number of factors in $\mathbf{F}_i$. We represent each independent sum s_i as the tuple $(\mathbf{V}_i, [(\mathbf{Z}_1^i, \mathbf{X}_1^i) \ldots, (\mathbf{Z}_{\kappa(i)}^i, \mathbf{X}_{\kappa(i)}^i)])$.

Factorization Query Generation. Given a set of incomplete attributes $\mathbf{Y} \subseteq \mathbf{A}_m$ in relation T with factorization $\mathcal{F}(\mathbf{Y})$, Algorithm 1 generates the query $\ddot{q}$ that estimates $\mathbb{P}_T(\mathbf{Y})$ using $\mathcal{F}(\mathbf{Y})$. Let $\mathcal{F}(\mathbf{Y})$ be a set of independent sums $s_1, \ldots, s_m$. To estimate the probability of each factor $\mathbb{P}_T(\mathbf{Z}_j^i \mid \mathbf{X}_j^i)$ in s_i, the algorithm generates the *factor query* $C_j^i := \gamma_{\mathbf{X}_j^i, \mathbf{Z}_j^i;\, Pr \leftarrow \mathrm{count}(*)\,/\,n_j^i}\big(\sigma_{\mathbf{Z}_j^i \neq \perp \wedge \mathbf{X}_j^i \neq \perp}(T)\big)$, where n_j^i is the size of the group $\mathbf{X}_j^i = x$, obtained by partitioning the selected tuples by $\mathbf{X}_j^i$ using a window function [37]. When $\mathbf{X}_j^i = \emptyset$, the selection and grouping on $\mathbf{X}_j^i$ are omitted.

The algorithm estimates each independent sum s_i by building J_i from its factor queries. It initializes J_i with C_1^i and, for $2 \le j \le \kappa(i) - 1$, joins J_i with C_j^i on $\mathbf{H}_j^i = (\mathbf{Z}_j^i \cup \mathbf{X}_j^i) \cap \bigcup_{h<j} (\mathbf{Z}_h^i \cup \mathbf{X}_h^i)$. The intermediate query J_i keeps the attributes of the factors of s_i. The query C_{s_i} sums its Pr column over $\mathbf{V}_i$. The algorithm joins the queries for the independent sums, multiplies their Pr columns, and returns $\ddot{q}$ and the intermediate queries J_i, $1 \le i \le m$.

When $\kappa(i) > 1$, the last factor of an independent sum produced by the search is the marginal of the last separating set ($\mathbf{Z}_{\kappa(i)}^i = \mathbf{X}_{\kappa(i)-1}^i$ and $\mathbf{X}_{\kappa(i)}^i = \emptyset$). The preceding factor query also returns this marginal, $marg(x) = |\sigma_{\mathbf{X}_{\kappa(i)-1}^i = x}(T)| \,/\, |\sigma_{\mathbf{X}_{\kappa(i)-1}^i \neq \perp}(T)|$, computed over all tuples in which $\mathbf{X}_{\kappa(i)-1}^i$ is observed. When some attributes of $\mathbf{F_Y}$ are missing in a tuple, we estimate their distribution via factor queries. A tuple in which all attributes of $\mathbf{F_Y}$ are missing does not contribute to any factor query, and we skip such a tuple. We assume that each factor query has tuples in which its attributes and separating set are observed.

Example 4.1. Let $\mathbf{Y} = \{income, skills\}$ with factorization $\mathcal{F}(\mathbf{Y}) = [(\{income\}, \{job\}), (\{skills\}, \{job\}), (\{job\}, \emptyset)]$. This factorization gives: $\mathbb{P}_{Jobs}(income, skills) = \sum_{job} \mathbb{P}_{Jobs}(income \mid job)\mathbb{P}_{Jobs}(skills \mid job)\mathbb{P}_{Jobs}(job)$. The two conditional factors are computed by factor queries. The factor query for $\mathbb{P}_{Jobs}(skills \mid job)$ also returns the marginal $\mathbb{P}_{Jobs}(job)$. The last factor reuses this marginal, so we do not generate a separate query for $(\{job\}, \emptyset)$.

Implementing Factorization Queries. Algorithm 1 expresses factorization queries as joins of subqueries. The factor queries can be written as nested subqueries, common table expressions (CTEs), or window functions [64]. Window functions do not require joins, but database systems often sort the input on the partition attributes [57]. Our implementation uses inline CTEs. CTEs provide more flexibility. If the same factor query appears in multiple factorization queries, we can materialize the CTEs and reuse them. Our experiments show that using inline CTEs is slightly more efficient than window functions. For simplicity of exposition, we assume for the remainder of the paper that the factorization has a single independent sum unless otherwise noted. The same construction applies when the factorization has multiple independent sums.

Complexity & Consistency. Under MCAR, $\mathcal{F}(\mathbf{Y})$ has the single factor $(\mathbf{Y}, \emptyset)$ with no separating set. Hence, $\ddot{q}$ contains no join because the factor query returns only $\hat{\mathbb{P}}_T(\mathbf{Y})$. Under MAR, $\mathcal{F}(\mathbf{Y})$ has the two factors $(\mathbf{Y}, \mathbf{X})$ and $(\mathbf{X}, \emptyset)$ with $\mathbf{X}$ complete. A single factor query returns both the conditional distribution $\hat{\mathbb{P}}_T(\mathbf{Y} \mid \mathbf{X})$ and the marginal distribution $\hat{\mathbb{P}}_T(\mathbf{X})$. Therefore, as in MCAR, $\ddot{q}$ contains no join. We have the following for the general case.

Proposition 4.2. *Consider a factorizable set of attributes $\mathbf{Y} \subseteq \mathbf{A}_m$ in relation T with factorization $\mathcal{F}(\mathbf{Y})$. Then $\ddot{q}$ produced by Algorithm 1 is a consistent estimator of $\mathbb{P}_T(\mathbf{Y})$ and has $\sum_{s_i \in \mathcal{F}(\mathbf{Y})} \max\{\kappa(i) - 2, 0\} + \big(|\mathcal{F}(\mathbf{Y})| - 1\big)$ join operations.*

Continuous Domains. For continuous domains, factor queries do not enumerate all possible values. If the input query only needs to determine whether a continuous value satisfies a query condition, the factor query estimates this probability for tuples with the same values of the separating set. Thus, the generated query computes only the probabilities needed by the input query, not the full distribution over the continuous domain. When a continuous attribute

appears in a separating set, we discretize it before applying the factor query to avoid estimates based on a single tuple.

Offline/Online Computation. An offline method would run Algorithm 1 for all combinations of incomplete attributes in each relation and store the resulting probability distributions before query time. This method may take a very long time even for a database with a modest number of incomplete attributes. It must also be repeated frequently as the database evolves. To avoid these overheads, since an input query q often refers to a small subset of incomplete attributes, one can compute these probabilities for the incomplete attributes of q at query time. In this paper, we follow the query-time approach. One can also precompute probabilities of the frequently accessed attributes to speed up query answering.

We denote the set of attributes in query q by $\mathbf{A}^q$. Let $\mathbf{A}^q_m = \mathbf{A}^q \cap \mathbf{A}_m$. Let $A_1, \ldots, A_r$ be the incomplete attributes in $\mathbf{A}^q \cup \mathbf{F}_{\mathbf{A}^q_m}$. Let $\Gamma_1, \ldots, \Gamma_{2^r}$ be all conjunctions of the form $\Gamma_i = \bigwedge_{j=1}^{r} \gamma^i_j$, where γ^i_j is either $A_j = \perp$ or $A_j \neq \perp$. Each predicate Γ_i selects the tuples that share the same set of attributes with missing values. Let $\mathbf{A}^{\Gamma_i}_m$ denote the attributes equal to $\perp$ in Γ_i.

For each Γ_i with $\mathbf{A}^{\Gamma_i}_m \cap \mathbf{A}^q \neq \emptyset$, apply Algorithm 1 to $\mathcal{F}(\mathbf{A}^q \cap \mathbf{A}^{\Gamma_i}_m)$ and construct C_{Γ_i} from its intermediate queries $J_1, \ldots, J_m$. The construction joins the intermediate queries on their common attributes and multiplies their *Pr* columns. It retains the factorization attributes observed in tuples satisfying Γ_i as grouping attributes. It sums over the eliminated factorization attributes that are missing in those tuples. It normalizes over $\mathbf{A}^q \cap \mathbf{A}^{\Gamma_i}_m$ for each value of the retained attributes. Thus, C_{Γ_i} returns the conditional distribution of the missing query attributes given the factorization values observed in the tuple. The generated query joins $\sigma_{\Gamma_i}(T)$ with C_{Γ_i} on the retained observed factorization attributes.

When $\mathbf{A}^{\Gamma_i}_m \cap \mathbf{A}^q = \emptyset$, the generated query retains $\sigma_{\Gamma_i}(T)$ with probability 1. The union of the results over all Γ_i provides each tuple with the distribution of its missing query attributes. The query estimators receive their input relations in this form and do not recompute the distributions.

Algorithm 1 Factorization Query Generator

input: Relation T, attributes $\mathbf{Y}$ in T, and factorization $\mathcal{F}(\mathbf{Y})$
output: factorization query $\ddot{q}$ and intermediate queries J_i, $1 \leq i \leq m$

1: **for each** $s_i = (\mathbf{V}_i, [(\mathbf{Z}^i_1, \mathbf{X}^i_1), \ldots, (\mathbf{Z}^i_{\kappa(i)}, \mathbf{X}^i_{\kappa(i)})])$ in $\mathcal{F}(\mathbf{Y})$ **do**
2: $\quad \mathbf{A}_i \leftarrow \bigcup_{1 \leq k \leq \kappa(i)} (\mathbf{Z}^i_k \cup \mathbf{X}^i_k)$
3: $\quad$ **if** $\kappa(i) = 1$ **then** $J_i \leftarrow$ factor query of $(\mathbf{Z}^i_1, \emptyset)$
4: $\quad$ **else**
5: $\quad\quad$ **for** $j = 1, \ldots, \kappa(i) - 1$ **do** $C^i_j \leftarrow$ factor query of $(\mathbf{Z}^i_j, \mathbf{X}^i_j)$
6: $\quad\quad$ $C^i_{\kappa(i)-1}$ also returns *marg*, the marginal of $\mathbf{X}^i_{\kappa(i)-1}$
7: $\quad\quad$ $J_i \leftarrow C^i_1$
8: $\quad\quad$ **for** $j = 2, \ldots, \kappa(i) - 1$ **do**
9: $\quad\quad\quad$ $\mathbf{H}^i_j \leftarrow (\mathbf{Z}^i_j \cup \mathbf{X}^i_j) \cap \bigcup_{h<j} (\mathbf{Z}^i_h \cup \mathbf{X}^i_h)$
10: $\quad\quad\quad$ $J_i \leftarrow J_i \bowtie_{\mathbf{H}^i_j} C^i_j$
11: $\quad\quad$ $J_i \leftarrow \pi_{\mathbf{A}_i; Pr \leftarrow \prod_{j=1}^{\kappa(i)-1} C^i_j.Pr \times C^i_{\kappa(i)-1}.marg}(J_i)$
12: $\quad C_{s_i} \leftarrow \gamma_{\mathbf{A}_i \setminus \mathbf{V}_i; Pr \leftarrow \text{sum}(J_i.Pr)}(J_i)$
13: **for each** s_i **do** $\mathbf{J}_i \leftarrow (\mathbf{A}_i \setminus \mathbf{V}_i) \cap \bigcup_{i'<i} (\mathbf{A}_{i'} \setminus \mathbf{V}_{i'})$
14: **return** $\ddot{q} \leftarrow \pi_{\mathbf{Y}; Pr \leftarrow \prod_{s_i \in \mathcal{F}(\mathbf{Y})} C_{s_i}.Pr} \left(\bowtie_{\mathbf{J}_i \atop s_i \in \mathcal{F}(\mathbf{Y})} C_{s_i} \right)$ and J_i, $1 \leq i \leq m$

The factor queries induce a distribution for each missing cell in T. Estimating and materializing these distributions for all incomplete attributes is impractical because an input query may reference only a subset of the incomplete attributes and only part of their active domains. Therefore, for an input query q, we apply Algorithm 1 only to the incomplete attributes that appear in q and the attributes required by their factorizations.

Throughout the paper, we present the algorithms for MNAR attributes, whose separating sets may contain incomplete attributes. MCAR and MAR are special cases for which the queries simplify.

5 Extensional Query Evaluation

In some practical settings, e.g., SQL null, the repairs and uncertainty of missing values in a tuple are assumed to be independent of those of other tuples in the database, i.e., no null symbol is shared by two tuples. In this case, we can efficiently compute the results of many queries by tuple-wise modifying the uncertainties of tuples in the database, known as *extensional evaluation* [66]. Similar ideas have been used to answer queries on tuple/block-independent probabilistic databases (PDBs) [25, 66]. However, we must compute the uncertainties of each tuple at query time, whereas these probabilities are assumed to be available in PDBs. This makes efficient query computation challenging.

Probability Space of the Database. Given relation T and tuple $t \in T$, let $\Delta(t)$ denote the repair set for t and let P_t denote its probability measure. The probability space on $\Delta(T) = \times_{t \in T} \Delta(t)$ is the product of the probability spaces of its tuples with the product σ-algebra [42]. For measurable subsets $r_t \subseteq \Delta(t)$, the probability of $\times_{t \in T} r_t$ is $\prod_{t \in T} \mathsf{P}_t(r_t)$. We define the database probability space using the product of the spaces for its relations.

Computing Uncertainty. We estimate $\mathsf{P}_t(.)$ from the observed data using the factor queries for the attributes with missing values in t. We compute each required joint or conditional probability from the factorization obtained from the m-graph of relation T. We compute these probabilities during query processing and do not

materialize them. Next, we present the algorithms that compute, for each output tuple t of query q on the database $\mathbf{T}$, the probability $\mathsf{P}_{q(\mathbf{T})}(t) = \mathsf{P}_{\mathbf{T}}\big(\{\mathbf{T}' \in \Delta(\mathbf{T}) \mid t \in q(\mathbf{T}')\}\big)$ that t appears in the answer of q, using the database probability space. We call this probability the *tuple marginal* of t. The results in this section apply to queries over non-repeating nulls under tuple independence.

5.1 Selection

5.1.1 Computation Rule. Given query $q = \sigma_{\theta(\mathbf{A}^q)}(T)$ and tuple $t \in T$, let $\mathsf{P}(t)$ be the tuple marginal attached to t. Let $\mathbf{A}_m^{q,t}$ denote the attributes in $\mathbf{A}^q$ with missing values in t. We have $\mathsf{P}_{q(T)}(t) = \mathsf{P}(t)\mathsf{P}_t(\theta(\mathbf{A}^q))$. If $\mathbf{A}_m^{q,t} = \emptyset$, then $\mathsf{P}_t(\theta(\mathbf{A}^q))$ is 1 if t satisfies θ and 0 otherwise. Otherwise, let $\theta(\mathbf{A}_m^{q,t})$ be obtained from θ by replacing each observed query attribute with its value in t. Then $\mathsf{P}_t(\theta(\mathbf{A}^q)) = \mathsf{P}_t(\theta(\mathbf{A}_m^{q,t}))$.

5.1.2 Query Rewriting. The input relation provides each tuple t with its tuple marginal $\mathsf{P}(t)$ and the distribution of its missing query attributes. If $\mathbf{A}_m^{\Gamma_i} \cap \mathbf{A}^q = \emptyset$, the query estimator checks θ using $\hat{q}_i := \pi_{\mathbf{A}_T,\, \mathsf{P}_{\hat{q}}(t) \leftarrow \mathsf{P}(t)}\big(\sigma_{\Gamma_i \wedge \theta}(T)\big)$. Otherwise, the query estimator for Γ_i is $\hat{q}_i := \sigma_{\mathsf{P}_{\hat{q}}(t)>0}\big(\gamma_{\mathbf{A}_T;\, \mathsf{P}_{\hat{q}}(t) \leftarrow \mathsf{P}(t)\, \mathrm{sum}(C_{\Gamma_i}.Pr)}\big(\sigma_\theta\big(\sigma_{\Gamma_i}(T) \bowtie C_{\Gamma_i}\big)\big)\big)$. In $\hat{q}_i$, the join is on the factorization attributes observed in tuples satisfying Γ_i. The selection σ_θ uses the observed query-attribute values from $\sigma_{\Gamma_i}(T)$ and the missing query-attribute values from C_{Γ_i}. The query estimator is $\hat{q} = \bigcup_i \hat{q}_i$. When $\mathbf{A}_m^{q,t} \neq \emptyset$, let $\hat{\mathsf{P}}_t(\theta(\mathbf{A}^q))$ denote the value of $\mathrm{sum}(C_{\Gamma_i}.Pr)$ in $\hat{q}_i$.

Theorem 5.1. *Consider $q = \sigma_\theta(T)$. Then $\hat{q}$ is consistent. The asymptotic confidence interval is $[\mathsf{P}_{\hat{q}}(t) \pm z(\alpha)\hat{\sigma}_t]$ with probability $1-\alpha$, where $z(\alpha)$ is the $1-\alpha/2$ quantile of the standard normal distribution and*

$$\hat{\sigma}_t^2 = \hat{\mathsf{P}}_t(\theta(\mathbf{A}^q))^2\,\mathrm{Var}(\mathsf{P}(t)) + \mathsf{P}(t)^2\,\mathrm{Var}\Big(\hat{\mathsf{P}}_t(\theta(\mathbf{A}^q))\Big) + 2\mathsf{P}(t)\hat{\mathsf{P}}_t(\theta(\mathbf{A}^q))\,\mathrm{Cov}\Big(\mathsf{P}(t), \hat{\mathsf{P}}_t(\theta(\mathbf{A}^q))\Big).$$

When $\mathbf{A}_m^{q,t} = \emptyset$ and t satisfies θ, $\hat{\sigma}_t^2 = \mathrm{Var}(\mathsf{P}(t))$.

Query Complexity. For each Γ_i, the factor queries are evaluated once, and their results are shared by all tuples satisfying Γ_i. However, $\hat{q}_i$ computes $\mathsf{P}_{\hat{q}}(t)$ separately for each such tuple.

5.2 Join

5.2.1 Computation Rule. Given query $q = T \bowtie_{\Phi(\mathbf{A}^q)} S$ and tuples $t \in T$ and $s \in S$, let $\mathbf{A}_m^{q,t}$ and $\mathbf{A}_m^{q,s}$ denote the attributes in $\mathbf{A}^q$ with missing values in t and s, respectively. Each tuple $t \in T$ and $s \in S$ has an attached tuple marginal $\mathsf{P}(t)$ and $\mathsf{P}(s)$, respectively. Let $\mathsf{P}_\Phi(t,s)$ be the probability that t and s satisfy Φ. If $\mathbf{A}_m^{q,t} \cup \mathbf{A}_m^{q,s} = \emptyset$, $\mathsf{P}_\Phi(t,s)$ is 1 when the observed values satisfy Φ and 0 otherwise. Otherwise, let $\Phi(\mathbf{A}_m^{q,t}, \mathbf{A}_m^{q,s})$ be the Boolean function created by replacing the observed attributes in $\mathbf{A}^q$ with their values in t and s. Let $\mathbf{a}^t$ and $\mathbf{a}^s$ range over the possible values of $\mathbf{A}_m^{q,t}$ and $\mathbf{A}_m^{q,s}$, respectively. We use $\mathsf{P}_t(\mathbf{a}^t)$ and $\mathsf{P}_s(\mathbf{a}^s)$ for the probabilities of these values in t and s, respectively. We have

$$\mathsf{P}_\Phi(t,s) = \sum_{\mathbf{a}^t} \sum_{\mathbf{a}^s} \mathsf{P}_t(\mathbf{a}^t)\mathsf{P}_s(\mathbf{a}^s)\Phi(\mathbf{a}^t, \mathbf{a}^s). \tag{1}$$

The joined-tuple marginal is:

$$\mathsf{P}_{q(T,S)}(t \bowtie s) = \mathsf{P}(t)\mathsf{P}(s)\mathsf{P}_\Phi(t,s). \tag{2}$$

5.2.2 Query Rewriting. The input relations provide each tuple, its marginal, and the distributions of its missing join attributes. Let $\hat{P}_\Phi(t,s)$ estimate $\mathsf{P}_\Phi(t,s)$ from the input distributions. Let $T_o := \sigma_{\mathbf{A}^q \cap \mathbf{A}_T \neq \perp}(T)$ and $T_m := \sigma_{\bigvee_{A \in \mathbf{A}^q \cap \mathbf{A}_T} A = \perp}(T)$. We define S_o and S_m similarly for S. For $t \in T_o$ and $s \in S_o$, $\mathbf{A}_m^{q,t} \cup \mathbf{A}_m^{q,s} = \emptyset$, so $\hat{P}_\Phi(t,s)$ is 1 or 0. The query estimator is $\hat{q}_o := \pi_{\mathbf{A}_T, \mathbf{A}_S,\, \mathsf{P}_{\hat{q}}(t \bowtie s) \leftarrow \mathsf{P}(t)\mathsf{P}(s)}\big(T_o \bowtie_\Phi S_o\big)$. For pairs with at least one missing value in $\mathbf{A}^q$, the query estimator is:

$$\hat{q}_m := \sigma_{\mathsf{P}_{\hat{q}}(t \bowtie s)>0}\Big(\pi_{\mathbf{A}_T, \mathbf{A}_S,\, \mathsf{P}_{\hat{q}}(t \bowtie s) \leftarrow \mathsf{P}(t)\mathsf{P}(s)\hat{P}_\Phi(t,s)}\big((T_m \times S) \cup (T_o \times S_m)\big)\Big).$$

The final query estimator is $\hat{q} := \hat{q}_o \cup \hat{q}_m$.

Theorem 5.2. *Consider $q = T \bowtie_{\Phi(\mathbf{A}^q)} S$ over distinct relations. Then $\hat{q}$ is consistent. The asymptotic confidence interval is $[\mathsf{P}_{\hat{q}}(t \bowtie s) \pm z(\alpha)\,\hat{\sigma}_{t \bowtie s}]$ with probability $1-\alpha$, where $z(\alpha)$ is the $1-\alpha/2$ quantile of the standard normal distribution and*

$$\hat{\sigma}^2_{t \bowtie s} = \big(\mathsf{P}(s)\hat{P}_\Phi(t,s)\big)^2\,\mathrm{Var}(\mathsf{P}(t)) + \big(\mathsf{P}(t)\hat{P}_\Phi(t,s)\big)^2\,\mathrm{Var}(\mathsf{P}(s)) + \big(\mathsf{P}(t)\mathsf{P}(s)\big)^2\,\mathrm{Var}(\hat{P}_\Phi(t,s)).$$

When $\mathsf{P}(t)$, $\mathsf{P}(s)$, and $\hat{P}_\Phi(t,s)$ are not independently estimated, $\hat{\sigma}^2_{t \bowtie s}$ also includes the covariance terms shown in Appendix B.

Query Complexity. The query estimator computes $T_o \bowtie_\Phi S_o$ and evaluates $\hat{P}_\Phi(t,s)$ for $|T_m||S| + |T_o|\,|S_m|$ tuple pairs.

5.3 Projection

5.3.1 Computation Rule. Consider query $q = \pi_{\mathbf{Y}}(T)$. Each tuple $t \in T$ has an attached tuple marginal $\mathsf{P}(t)$. For an output tuple y, let $\mathsf{P}_t(y)$ be the probability that $t.\mathbf{Y} = y$. When $\mathbf{Y}$ is observed in t, $\mathsf{P}_t(y)$ is 1 for $y = t.\mathbf{Y}$ and 0 otherwise. Let $T_y = \{t \in T \mid \mathsf{P}_t(y) > 0\}$. If the tuples in T_y are independent, then

$$\mathsf{P}_{q(T)}(y) = 1 - \prod_{t \in T_y} \big(1 - \mathsf{P}(t)\mathsf{P}_t(y)\big). \tag{3}$$

If these tuples are dependent, for example, after a join in which several joined tuples share an input tuple, the projection rule may not compute the correct output tuple marginal [19, 66].

5.3.2 Query Rewriting. Let $\hat{P}_t(y)$ estimate $\mathsf{P}_t(y)$. When $\mathbf{Y}$ is observed in t, $\hat{P}_t(y)$ is 1 for $y = t.\mathbf{Y}$ and 0 otherwise. For incomplete projected attributes, the input relation provides $\hat{P}_t(y)$ for each possible projected value y. We define

$$C_\pi := \big\{(t, y, Pr_\pi(t,y)) \mid t \in T,\ \hat{P}_t(y) > 0,\ Pr_\pi(t,y) = \mathsf{P}(t)\hat{P}_t(y)\big\}.$$

The query estimator is $\hat{q} := \sigma_{\mathsf{P}_{\hat{q}}(y)>0}\left(\gamma_{\mathbf{Y};\, \mathsf{P}_{\hat{q}}(y) \leftarrow 1 - \prod\left(1 - Pr_\pi(t,y)\right)}(C_\pi)\right)$.

Theorem 5.3. *Consider $q = \pi_{\mathbf{Y}}(T)$ for which T_y is fixed. Then $\hat{q}$ is consistent. The asymptotic confidence interval is $[\mathsf{P}_{\hat{q}}(y) \pm z(\alpha)\,\hat{\sigma}_y]$ with probability $1-\alpha$, where $z(\alpha)$ is the $1-\alpha/2$ quantile of the standard normal distribution and*

$$\hat{\sigma}_y^2 = \sum_{t \in T_y} \left(\prod_{t' \in T_y \setminus \{t\}} \big(1 - Pr_\pi(t', y)\big)\right)^2 \mathrm{Var}\big(Pr_\pi(t,y)\big).$$

When the estimates $Pr_\pi(t, y)$ are not independently estimated, $\hat{\sigma}^2_y$ also includes the covariance terms shown in Appendix B.

Query Complexity. Constructing C_π evaluates $\mathsf{P}(t)\hat{P}_t(y)$ for each pair (t, y) with $\hat{P}_t(y) > 0$. The final aggregation evaluates one factor for each tuple of C_π.

5.4 Other Query Operators

Union and set difference compute output tuple marginals from independent input tuples using basic probability rules [19, 66]. Aggregation computes expected SUM and expected COUNT from the input tuple marginals, and AVG is their ratio [19]. For unions of conjunctive queries (UCQs), Möbius inversion combines logically equivalent terms in inclusion–exclusion [66]. Appendix B.1 gives the computation rules and guarantees for these operators.

5.5 Combining Operators

Extensional evaluation applies the computation rule for each operator in the order of the query plan [19]. Each relational operator receives the relation returned by the preceding operator, with a tuple marginal attached to each tuple. It computes the tuple marginals attached to its output relation. These output tuple marginals become the input tuple marginals of the next relational operator. An aggregation operator computes its result from the tuple marginals returned by the preceding relational operators.

5.6 Time Complexity and Limitations

Time Complexity. An extensional plan is a query plan whose operators compute the query result and its tuple marginals. An extensional plan is *safe* if it computes the correct output tuple marginals for every tuple-independent input database. A query is *safe* if it has a safe extensional plan and *unsafe* otherwise [19, 66]. The number of factor queries depends only on q, and each factor query and each operator in a safe plan can be evaluated in polynomial time in the size of the input relations. Under tuple independence, computing the exact output tuple marginals of an unsafe UCQ is #P- hard [19, 66]. For an unsafe UCQ whose conjunctive queries have no self-joins, any extensional plan that uses selection, join, projection, and union over the true input tuple marginals computes an upper bound on each output tuple marginal [22, 66]. We use a Monte Carlo estimator to evaluate unsafe UCQs.

Optimizations. For each Γ_i with $\mathbf{A}^{\Gamma_i}_m \cap \mathbf{A}^q \neq \emptyset$, the factor queries in $\hat{q}_i$ are evaluated once for all tuples in $\sigma_{\Gamma_i}(T)$.

Limitations. Although the factor-query results are shared, extensional evaluation computes a tuple marginal separately for every tuple, which creates overhead. Extensional evaluation applies to safe queries under tuple independence. Marked nulls violate tuple independence. When an extensional rule combines tuples that share a marked null, the rule may treat dependent tuples as independent and return incorrect tuple marginals.

6 Monte Carlo Evaluation

Extensional evaluation is limited by query safety and independence conditions. A marked null shared by several tuples violates this assumption because all occurrences of the same $\perp_k$ have the same value. Methods based on Monte Carlo techniques instead evaluate an input query q on sampled database repairs [18, 33]. We propose *Causality-Aware Monte Carlo (CAMC)*, a query estimator that uses the m-graphs and observed data to generate H repairs. CAMC evaluates q for each repair and estimates the probabilities of the query results based on their frequencies in the H results. We first estimate the distribution of each $\perp_k$ from the observed data. We then explain how to sample these values and evaluate the query.

6.1 Estimating the Distribution of Marked Nulls

Following [18], we assume that distinct marked-null symbols are independent. Each $\perp_k$ is associated with a distribution $\mathbb{P}^{\perp_k}_T(v)$ over its possible value v. We denote its estimate by $\hat{\mathbb{P}}^{\perp_k}_T(v)$. When all occurrences of $\perp_k$ are in attribute Y, we also write these distributions as $\mathbb{P}^{\perp_k}_T(Y = v)$ and $\hat{\mathbb{P}}^{\perp_k}_T(Y = v)$. Unlike [18], which receives $\mathbb{P}^{\perp_k}_T(Y = v)$ as input, we estimate it from the observed data.

Marked nulls occurring in one attribute. Consider a marked null $\perp_k$ whose occurrences are all in attribute Y. Let $\mathbf{X}$ be the separating set of Y. For each $x \in \text{adom}(\mathbf{X})$, let $\hat{\mathbb{P}}^{\perp_k}_T(\mathbf{X} = x)$ denote the estimated probability of $\mathbf{X} = x$ among the tuples where $\perp_k$ appears in Y. If $\mathbf{X}$ is incomplete, we use $\mathcal{F}(\mathbf{X})$ to estimate its distribution. We estimate the distribution of $\perp_k$ as

$$\hat{\mathbb{P}}^{\perp_k}_T(Y = v) = \sum_{x \in \text{adom}(\mathbf{X})} \hat{\mathbb{P}}_T(Y = v \mid \mathbf{X} = x)\, \hat{\mathbb{P}}^{\perp_k}_T(\mathbf{X} = x). \quad (4)$$

Functional dependencies. Marked nulls may originate from integrity constraints. Integrity constraints are logical rather than causal dependencies [63]. Consider a functional dependency $X \rightarrow Y$. Suppose that every occurrence of $\perp_k$ in Y has the determinant value $X = x$. If X is not in $\mathcal{F}(Y)$, we add the condition $X = x$ to the factor queries. Algorithm 1 generates these queries from $\mathcal{F}(Y)$. If X is in $\mathcal{F}(Y)$, we fix $X = x$ in every factor that contains X. Both cases give $\hat{\mathbb{P}}^{\perp_k}_T(Y = v) = \hat{\mathbb{P}}_T(Y = v \mid X = x)$. If two observed tuples with $X = x$ disagree on Y, the database is inconsistent with the exact functional dependency. In this case, one may use functional dependency approximation techniques, which are beyond the scope of this paper.

Marked nulls occurring in several attributes. Suppose that the same $\perp_k$ occurs in attributes $Y_1, \ldots, Y_r$. For each Y_i, we use $\mathcal{F}(Y_i)$ to estimate $\hat{\mathbb{P}}^{\perp_k}_T(Y_i = v)$. Every occurrence of $\perp_k$ must have the same value v. We combine the estimated distributions of $\perp_k$ using a product of experts [29]. We write $\hat{\mathbb{P}}^{\perp_k}_T(v)$ for the combined distribution. For every $v \in \bigcap_{i=1}^r \text{adom}(Y_i)$, we estimate the distribution by

$$\hat{\mathbb{P}}^{\perp_k}_T(v) = \frac{\prod_{i=1}^r \hat{\mathbb{P}}^{\perp_k}_T(Y_i = v)}{\sum_{v' \in \bigcap_{i=1}^r \text{adom}(Y_i)} \prod_{i=1}^r \hat{\mathbb{P}}^{\perp_k}_T(Y_i = v')}.$$

We use the same product-of-experts combination to define $\mathbb{P}^{\perp_k}_T(v)$ from the true attribute distributions. Since each $\hat{\mathbb{P}}^{\perp_k}_T(Y_i = v)$ is consistent by Proposition 4.2, the continuous mapping theorem [12] gives consistency of $\hat{\mathbb{P}}^{\perp_k}_T(v)$ for $\mathbb{P}^{\perp_k}_T(v)$.

6.2 Generating Repairs

For each sample index $s \in \{1, \ldots, H\}$, Algorithm 2 samples one value v for every distinct $\perp_k$ in each relation $T \in \mathbf{T}$. Every occurrence of $\perp_k$ in T takes the same value v. After all marked nulls in T have been sampled, the resulting complete relation is $T^{(s)}$. The relations obtained with the same sample index form the database repair $\mathbf{T}^{(s)}$. Running Algorithm 2 H times with independent sampling gives the repairs $\mathbf{T}^{(1)}, \ldots, \mathbf{T}^{(H)}$.

Algorithm 2 RepairSampler($\mathbf{T}, s$)

Input: A database $\mathbf{T}$, sample index s, and $\hat{\mathbb{P}}_T^{\perp_k}$ for every $T \in \mathbf{T}$ and distinct $\perp_k \in \text{Null}(T)$
Output: one sampled repair $\mathbf{T}^{(s)}$

1: $\mathbf{T}^{(s)} \leftarrow \mathbf{T}$
2: **for** each relation $T \in \mathbf{T}$ and each distinct $\perp_k \in \text{Null}(T)$ **do**
3: Draw $v \sim \hat{\mathbb{P}}_T^{\perp_k}$
4: Substitute v for every occurrence of $\perp_k$ in $\mathbf{T}^{(s)}$
5: **return** $\mathbf{T}^{(s)}$

Estimator. For an input query q, let $\mathcal{A} \in \Sigma_{(q,\mathbf{T})}$ be a set in the answer space for q. We evaluate q on each complete database repair. Hence, the estimator applies to both safe and unsafe queries. The marked-null distributions estimated from the observed data induce the estimator's measure $\mathsf{P}_{\hat{q}(\mathbf{T})}$ defined in Section 3. We estimate $\mathsf{P}_{\hat{q}(\mathbf{T})}(\mathcal{A})$ without enumerating $\Omega_{q(\mathbf{T})}$. Given H database repairs $\mathbf{T}^{(1)}, \ldots, \mathbf{T}^{(H)}$, we estimate this probability as follows.

$$\mathsf{P}_{\hat{q}_H(\mathbf{T})}(\mathcal{A}) = \frac{1}{H} \sum_{s=1}^{H} \mathbf{1}[q(\mathbf{T}^{(s)}) \in \mathcal{A}]. \tag{5}$$

Because the repairs are sampled independently, Equation 5 averages H independent indicator values whose mean is $\mathsf{P}_{\hat{q}(\mathbf{T})}(\mathcal{A})$. For example, consider $q = \sigma_{income>100K}(Jobs)$ and let $\mathcal{A}$ be the set of results containing at least two employees. Equation 5 is the fraction of the H repairs in which q returns at least two employees.

Theorem 6.1. *Consider a query q and $\mathcal{A} \in \Sigma_{(q,\mathbf{T})}$. As $|\mathbf{T}| \to \infty$ and $H \to \infty$, $\mathsf{P}_{\hat{q}_H(\mathbf{T})}(\mathcal{A})$ is consistent for $\mathsf{P}_{q(\mathbf{T})}(\mathcal{A})$. The asymptotic confidence interval obtained from the H repairs is $[\mathsf{P}_{\hat{q}_H(\mathbf{T})}(\mathcal{A}) \pm z(\alpha)\hat{\sigma}]$ with probability $1-\alpha$, where $\hat{\sigma}^2 = \frac{\mathsf{P}_{\hat{q}_H(\mathbf{T})}(\mathcal{A})\left(1-\mathsf{P}_{\hat{q}_H(\mathbf{T})}(\mathcal{A})\right)}{H}$.*

Algorithm 2 requires the distribution of each marked null to execute Line 3. The factor queries compute the marked-null distributions before sampling. However, estimating the distribution of marked nulls may create an overhead. We propose to sample from the factorization without actually computing it.

6.3 Factorization Sampling

If we sample Y directly from its marginal distribution, i.e., all tuples where Y is observed, the selected values will be skewed toward the group of its separating set where Y is observed more often. Hence, the sample is biased. To avoid this bias, we process the factorization of Y in **reverse order**. By definition, the last factor of $\mathcal{F}(Y)$ has no conditioning attributes, i.e., it is a marginal distribution. Any incomplete attributes in the last factor have MCAR data. We first use this factor to obtain the values required by the preceding factor.

We illustrate with an example. Consider a relation $T(Y, X)$ where Y has MNAR data and X is its separating set. Assume that X has MCAR data. The factorization of Y is $\mathcal{F}(Y) = [(Y, X), (X, \emptyset)]$.

Sampling the marginal factor. The last factor of $\mathcal{F}(Y)$ is $(X, \emptyset)$, which represents the marginal distribution of X. By the MCAR definition and factorization, $(X, Y) \perp\!\!\!\perp R_X$. Thus, the tuples where X is observed form a random sample of $\mathbb{P}_T(X)$. We retain these tuples in a relation $T^* := \sigma_{X \neq \perp}(T)$. For each distinct $\perp_k$ in X, we select one tuple uniformly from T^* and assign its value of X to every occurrence of $\perp_k$. Now the tuples in T have a complete X. The attribute Y is still incomplete.

Sampling the conditional factor. The second factor in the reverse order of $\mathcal{F}(Y)$ is (Y, X), which represents the conditional distribution of Y given X. By the independence $Y \perp\!\!\!\perp (R_Y, R_X) \mid X$, the observed values of Y form a random sample of the conditional distribution $\mathbb{P}_T(Y \mid X)$. For each value $x \in \text{adom}(X)$, we obtain a relation T_x^* from T^* that contains the observed values of Y:

$$T_x^* := \sigma_{Y \neq \perp \wedge X = x}(T^*).$$

The relation T_x^* contains all tuples in which $X = x$ and Y is observed. For each tuple $t \in T$ where $X = x$ and Y is missing, we select one tuple uniformly from T_x^* and assign its value of Y to the missing value in t. This yields a repaired relation $T^{(s)}$. The resulting repaired tuples in $T^{(s)}$ are not biased because we first sampled X randomly from $\hat{\mathbb{P}}_T(X = x)$, then sampled Y from the conditional distribution $\hat{\mathbb{P}}_T(Y \mid X = x)$.

Let $\mathcal{F}(Y) = [(\mathbf{Z}_1, \mathbf{X}_1), \ldots, (\mathbf{Z}_n, \mathbf{X}_n)]$ be the ordered factorization of Y. Algorithm 3 initializes $t' = t$ and processes the factors from factor n to factor 1. For factor $(\mathbf{Z}_i, \mathbf{X}_i)$, it retains the tuples that match t' on $\mathbf{X}_i$ and on the attributes of $\mathbf{Z}_i$ that already have values in t'. It selects one of these tuples uniformly. It assigns the selected tuple's values to the attributes of $\mathbf{Z}_i$ that are missing in t'. The values sampled in t' remain fixed while the preceding factors are processed. We assume that every relation T_i^* constructed by FactorSampler is nonempty.

Algorithm 3 FactorSampler($T, t, Y, \mathcal{F}(Y)$)

Input: A relation T, a tuple t with a missing value in Y, and its ordered factorization $\mathcal{F}(Y)$
Output: The tuple t' containing the values sampled from $\mathcal{F}(Y)$

1: $t' \leftarrow t$
2: **for** $i = n, n-1, \ldots, 1$ **do**
3: **for** each $A \in \mathbf{Z}_i \setminus \{Y\}$ **do**
4: **if** $t'[A] = \perp_{k'}$ and $\perp_{k'}$ occurs more than once **then**
5: Select one occurrence of $\perp_{k'}$ uniformly and let u be its tuple
6: $u' \leftarrow$ FactorSampler$(T, u, A, \mathcal{F}(A))$
7: $t'[A] \leftarrow u'[A]$
8: $\mathbf{Z}_{i,m} \leftarrow \{A \in \mathbf{Z}_i \mid t'[A] = \perp\}$
9: **if** $\mathbf{Z}_{i,m} \neq \emptyset$ **then**
10: $\mathbf{Z}_{i,o} \leftarrow \mathbf{Z}_i \setminus \mathbf{Z}_{i,m}$
11: $T_i^* \leftarrow \sigma_{\mathbf{Z}_{i,m} \neq \perp \wedge \mathbf{X}_i = t'[\mathbf{X}_i] \wedge \mathbf{Z}_{i,o} = t'[\mathbf{Z}_{i,o}]}(T)$
12: Select one tuple r uniformly from T_i^*
13: $t'[\mathbf{Z}_{i,m}] \leftarrow r[\mathbf{Z}_{i,m}]$
14: **return** t'

Our SQL implementation selects one tuple uniformly from T_i^* by ordering the tuples randomly and returning the first tuple. FactorSampler performs its random selections independently for each repair. When $\perp_k$ occurs more than once, we select one occurrence uniformly and pass its tuple to FactorSampler.

Table 2: An incomplete relation $T(Y, X)$

	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
Y	1	$\perp_2$	$\perp_3$	$\perp_5$	1	2	3	4	5	5
X	$\perp_1$	x_2	$\perp_4$	x_1	x_1	x_2	x_1	x_2	x_1	x_2

Example 6.2. Consider relation $T(Y, X)$ in Table 2, where $\text{adom}(Y) = \{1, 2, 3, 4, 5\}$ and $\text{adom}(X) = \{x_1, x_2\}$. Let the factorization be $\mathcal{F}(Y) = [(Y, X), (X, \emptyset)]$. For t_1, the missing cell is in X. Since X is MCAR, $\mathcal{F}(X) = [(X, \emptyset)]$. We select one tuple uniformly from $\sigma_{X\neq\perp}(T)$ and assign its value of X to $\perp_1$. For t_2, the value $X = x_2$ is observed, so we select one tuple uniformly from $\sigma_{Y\neq\perp \wedge X=x_2}(T)$ and assign its value of Y to $\perp_2$. For t_3, we first sample one value of X uniformly from $\sigma_{X\neq\perp}(T)$ and assign the sampled value to $\perp_4$. If this value is x_1, we then sample Y from $\sigma_{Y\neq\perp \wedge X=x_1}(T)$ and assign it to $\perp_3$. Otherwise, we select one tuple uniformly from $\sigma_{Y\neq\perp \wedge X=x_2}(T)$ and assign its value of Y to $\perp_3$. The sampled value of X determines which group is used to sample Y. For tuple t_4, we sample for $\perp_5$ from $\sigma_{Y\neq\perp \wedge X=x_1}(T)$.

Theorem 6.3. *Consider a marked null $\perp_k$ whose occurrences are all in attribute Y. Assume that $\perp_k$ does not occur in the separating set of Y. Then $t'[Y]$ returned by FactorSampler follows the distribution $\hat{\mathbb{P}}_T^{\perp_k}(Y)$.*

When all occurrences of $\perp_k$ are in one attribute, FactorSampler replaces the draw in Line 3 of RepairSampler. When $\perp_k$ occurs in several attributes, RepairSampler draws its value from the combined distribution defined in Section 6.1.

6.4 Implementation

We implement our method using *repair* and *query encodings*. In *repair encoding*, given an input query, we generate queries that implement factorization sampling and store the observed values and the H sampled values for each relation in an inline CTE using the structure of the relations in Monte Carlo databases (MCDB) [33]. Our method then evaluates the input query once over the encoded CTEs and returns the final result. In *query encoding*, factorization sampling generates the sampled values for each repair. For each repair, we rewrite q using its sampled values following [18]. We compute the final result and its probabilities from the union of the H query results. Neither encoding materializes the H repairs as separate databases. Our empirical evaluation indicates that repair encoding outperforms query encoding. Therefore, we report CAMC using repair encoding.

6.4.1 Repair Encoding. Following the representation of MCDB [33], our repair encoding does not store the H repairs as separate relation instances. For each input tuple t, one row represents the values of t in all H repairs. An observed cell in t stores one value because its value is the same across all repairs. A missing cell in t stores a *bundle* of the sampled values $(v_1, \ldots, v_H)$, where v_s is its value in repair $\mathbf{T}^{(s)}$. All occurrences of the same marked null store the same bundle. Next, we explain how the MCDB representation stores tuple presence and evaluates query operators over the H repairs.

Tuple Presence. For each input tuple t, the row representing t also stores one indicator for each repair. Before evaluating q, all indicators are 1 because every input tuple is present at every repair index. If a query operator removes the tuple represented by the row from the result of repair $\mathbf{T}^{(s)}$, it changes the indicator at position s to 0. For example, let $H = 3$ and suppose that the sampled values of a missing cell $t[A]$ are $(1, 2, 1)$. Before evaluating the query, the indicators of t are $(1, 1, 1)$. For $q = \sigma_{A=1}(T)$, the selection changes them to $(1, 0, 1)$ because $t[A] = 1$ at indices 1 and 3.

Evaluating the Query. MCDB evaluates a query once over all database instances represented by its tuple bundles [33]. Our repair encoding evaluates one modified query over the encoded database to compute the results of all H repairs. An observed cell uses its single stored value at every repair index. A missing cell uses its sampled value v_s at index s. The modified query applies each relational operator at every index s. Therefore, the rows at index s form the query result $q(\mathbf{T}^{(s)})$.

Join Operator. For a database with multiple relations, repair index s denotes one sampled database [33]. At index s, both relations use the marked-null values sampled for that index. If the input has relations T and S, repair index s therefore contains both $T^{(s)}$ and $S^{(s)}$. The join compares tuples from $T^{(s)}$ only with tuples from $S^{(s)}$.

A join combines the tuples of two relations that agree on the join attributes. If a join attribute is missing in a tuple, the tuple holds H sampled values for it and may agree with different tuples in different repairs. We use the same split operation as MCDB before applying the join [33]. It applies to one tuple at a time and partitions the tuple's repairs by the sampled value of the join attribute. For each distinct sampled value, it outputs one copy of the tuple. The copy holds this value in all repairs and is present exactly in the repairs that sampled it. After the split, every tuple produced by the split has one value for the join attributes, and the two relations join on these values. A joined tuple appears in the repairs when both of its input tuples are present.

Projection and Duplicate Removal. Projection removes the attributes that are not included in the output at each repair index. Before combining equal output tuples, MCDB splits any remaining output attribute whose sampled values differ across repairs [33]. After the split, equal rows have fixed output values. MCDB combines these rows and sets the indicator at index s to 1 when at least one of them is present at index s.

6.4.2 Computing Answer Probabilities. For an input query q, we estimate $\mathsf{P}_{\hat{q}}(t)$ for each output tuple t by the fraction of the H repairs whose query result contains t. Let $\bar{\mathbf{A}}$ be the output attributes of q. To compute these marginals without materializing the repairs, the query keeps the repair index with each output tuple. If output tuple t appears in $q(\mathbf{T}^{(s)})$, the query returns the row (t, s), containing the values of t and the repair index $idx = s$. If the same output tuple appears in several query results, the query returns one row for each corresponding repair index. For example, if t appears in $q(\mathbf{T}^{(1)})$, $q(\mathbf{T}^{(3)})$, and $q(\mathbf{T}^{(4)})$, the query returns $(t, 1)$, $(t, 3)$, and

$(t, 4)$. The final query groups these rows by the output attributes $\bar{\mathbf{A}}$. All rows with the same output tuple values are placed in the same group. The inclusion of *idx* prevents the union from eliminating rows for the same tuple that come from different repairs. If the same output tuple is produced more than once within one repair, the query retains one row for that tuple and repair index. This is correct because the tuple marginal depends on whether the tuple appears in a repair, not on how many times it is produced. For an output tuple t, count$(*)$ is the number of repairs in which t appears. Therefore, count$(*)/H$ estimates $\mathsf{P}_{\hat{q}}(t)$. The estimation query is

$$\hat{q}_H := \gamma_{\bar{\mathbf{A}};\, \mathsf{P}_{\hat{q}}(t) \leftarrow \text{count}(*)/H}\Big(\bigcup_{s=1}^{H} \pi_{\bar{\mathbf{A}},\, idx \leftarrow s}\big(q(\mathbf{T}^{(s)})\big)\Big). \quad (6)$$

The union in Equation 6 denotes the indexed rows produced by one modified query over the repair encoding. It does not require storing the H repaired databases.

Reporting Answers. We report the marginal probability of each tuple in the final answer. For marked nulls, tuple marginals may not fully describe the answer probability space because several tuples may depend on the same marked null [18]. Our method can return the probabilities of the desired subsets of the answer space $\mathcal{A} \in \Sigma_{(q,\mathbf{T})}$ including tuple marginals.

6.5 Time Complexity and Limitations

Time Complexity. For fixed q and $\mathbf{T}$, sampling the H repairs requires H executions of RepairSampler, and the encoded query processes H repair indices. Query encoding evaluates one rewritten copy of q per repair, whereas repair encoding evaluates one modified query and shares the computation of the observed values across repairs.

Proposition 6.4. *Given database* $\mathbf{T}$*, query* q*, and fixed* H*, CAMC computes* $\hat{q}_H$ *in polynomial time in* $|\mathbf{T}|$.

Limitations. The accuracy of CAMC depends on the number of repairs H. The confidence interval in Theorem 6.1 shrinks at the rate $1/\sqrt{H}$. For fixed relative error and confidence, estimating $\mathsf{P}_{q(\mathbf{T})}(\mathcal{A})$ by Monte Carlo sampling requires a number of repairs proportional to $1/\mathsf{P}_{q(\mathbf{T})}(\mathcal{A})$ [66]. A result with a small probability may appear in a few repairs. Hence, many repairs are needed to observe it. CAMC evaluates the input query over the H repairs, so its running time grows with the required number of repairs.

7 Direct Estimation for Aggregation Queries

To evaluate queries with aggregation functions, database systems drop tuples containing NULL values, which yields *biased* answers under MAR and MNAR data. Using the observed values without accounting for the missingness mechanisms yields an estimate biased towards the groups that are more often observed. For example, in Table 1b, ignoring tuples with missing values yields an average income of \$226.67K, significantly below the true average of \$413.33K. One baseline approach is to use distribution estimation frameworks, such as [45], to reconstruct and materialize the joint distribution and evaluate aggregation queries over it. However, as we show in the experiments, this approach does not scale to large datasets and large active domains. As discussed in Section 6, CAMC evaluates the input query over H sampled repairs, and H must grow in proportion to $1/\mathsf{P}_{q(\mathbf{T})}(\mathcal{A})$. Hence, CAMC is impractical when $\mathsf{P}_{q(\mathbf{T})}(\mathcal{A})$ is small.

We propose *Causality-Aware Direct Estimation (CADE)*, which estimates query answers directly from the factor-query results. For aggregation queries expressible through expectations, e.g., sum and count, CADE applies the law of total expectation over the separating-set values [4]. We support the min and max aggregation functions in the next section. The results in this section hold for both non-repeating and marked nulls. Consistency under Definition 3.1 requires an estimate of the query-answer probability space. For an aggregation query, CADE computes an aggregate estimate and its confidence interval from the factor queries. The query estimate converges in probability to the convergent answer. For a query q, let $\mathbf{A}^q$ denote the attributes referenced by q, and let $\mathbf{A}^q_m = \mathbf{A}^q \cap \mathbf{A}_m$. We first present CADE for an average query over one relation, followed by queries over multiple relations.

7.1 Querying a Single Relation

Consider $q = avg(\pi_{\mathbf{Y}}(\sigma_\theta(T)))$, where $\mathbf{Y} \subseteq \mathbf{A}^q_m$. By Definition 3.3, a convergent estimator concentrates its probability mass on a single possible answer, which we call the convergent answer. We first identify this answer for q and then construct an estimator that converges to it.

Proposition 7.1. *Consider* $q = avg(\pi_{\mathbf{Y}}(\sigma_\theta(T)))$. *Then* $\mathbb{E}[\mathbf{Y} \mid \theta]$ *is the convergent answer of* q.

We apply Algorithm 1 to $\mathcal{F}(\mathbf{A}^q_m)$ with the following modification. Let $(\mathbf{Z}_j, \mathbf{X}_j)$ be the factor in $\mathcal{F}(\mathbf{A}^q_m)$ that contains $\mathbf{Y}$, and let $\mathbf{X} = \mathbf{X}_j$. We use the law of total expectation $\mathbb{E}[\mathbf{Y} \mid \theta] = \sum_{\mathbf{X}} \mathbb{E}[\mathbf{Y} \mid \mathbf{X}, \theta]\mathbb{P}_T(\mathbf{X} \mid \theta)$ to estimate q. We use the sample mean $\hat{\mu}_{\mathbf{Y}|\mathbf{X},\theta}$ to estimate $\mathbb{E}[\mathbf{Y} \mid \mathbf{X}, \theta]$. The conditional independence $\mathbf{Y} \perp\!\!\!\perp (R_{\mathbf{Y}}, R_{\mathbf{X}}) \mid \mathbf{X}$ justifies computing $\hat{\mu}_{\mathbf{Y}|\mathbf{X},\theta}$ from the observed data. We modify the factor query C_j as follows: $C_j := \gamma_{\mathbf{X};\, \hat{\mu}_{\mathbf{Y}|\mathbf{X},\theta} \leftarrow \text{avg}(\mathbf{Y})}\big(\sigma_{\mathbf{Y}\neq\perp \wedge \mathbf{X}\neq\perp \wedge \theta}(T)\big)$.

The remaining factor queries are modified similarly to include θ evaluation and estimate $\hat{\mathbb{P}}_T(\mathbf{X} \mid \theta)$. The modified query $\ddot{q}$ of Algorithm 1 returns $\hat{\mu}_{\mathbf{Y}|\mathbf{X},\theta}$ and $\hat{\mathbb{P}}_T(\mathbf{X} \mid \theta)$. The final query estimator is $\hat{q} := \gamma_{\emptyset;\, a \leftarrow \text{sum}(\hat{\mu}_{\mathbf{Y}|\mathbf{X},\theta} \cdot \hat{\mathbb{P}}_T(\mathbf{X}|\theta))}(\ddot{q})$.

Example 7.2. Consider $q = avg(\pi_{income}(Jobs))$ on the relation in Figure 1a with the m-graph in Figure 1d. The attribute *income* has MNAR data, and its separating set is the MCAR attribute *job*. The m-graph in Figure 1d gives $income \perp\!\!\!\perp R_{income}, R_{job} \mid job$, so $\mathbf{X} = \{job\}$. The estimator returns $\hat{q} = \sum_v \hat{\mu}_{income|\, job=v} \hat{\mathbb{P}}_T(job = v) \approx 149.7K$ over $v \in \{Executive, Assistant, Consultant\}$, with empirical proportions $\frac{3}{11}$, $\frac{3}{11}$, and $\frac{5}{11}$.

Marked Nulls. The query estimator $\hat{q}$ remains unchanged under marked nulls. A tuple containing a marked null in $\mathbf{Y}$ is excluded from $\hat{\mu}_{\mathbf{Y}|x,\theta}$. The remaining observed values of $\mathbf{Y}$ within each group form a random sample because $\mathbf{Y} \perp\!\!\!\perp (R_{\mathbf{Y}}, R_{\mathbf{X}}) \mid \mathbf{X}$. When tuples share a marked null in $\mathbf{X}$, their group memberships are dependent and introduce covariance. To obtain convergent answers and confidence intervals under marked nulls, we require the following assumption.

Assumption 1. *For each marked-null symbol* $\perp_k \in \text{Null}(T)$, *let* $T_k = \{t \in T : \perp_k \in t\}$. *As* $|T| \to \infty$,

$$\max_{\perp_k \in \text{Null}(T)} \frac{|T_k|}{|T|} \longrightarrow 0.$$

Assumption 1 always holds for non-repeating nulls because every null symbol occurs in one tuple. Let $z(\alpha)$ denote the $(1-\alpha/2)$ quantile of the standard normal distribution.

Theorem 7.3. *Consider $q = avg(\pi_{\mathbf{Y}}(\sigma_\theta(T)))$. Let $\mu_{\mathbf{Y}|x,\theta} = \mathbb{E}[\mathbf{Y} \mid \mathbf{X} = x, \theta]$ and $w_{x|\theta} = \mathbb{P}_T(\mathbf{X} = x \mid \theta)$, with estimates $\hat{\mu}_{\mathbf{Y}|x,\theta}$ and $\hat{w}_{x|\theta} = \hat{\mathbb{P}}_T(\mathbf{X} = x \mid \theta)$. If Assumption 1 holds, then $\hat{q} \xrightarrow{p} a$, where $a = \sum_x \mu_{\mathbf{Y}|x,\theta} w_{x|\theta}$ is the convergent answer of q. The confidence interval is $[\hat{q} \pm z(\alpha)\hat{\sigma}]$ with probability $1-\alpha$, where*

$$\hat{\sigma}^2 \approx \sum_{x,x'} \Big[\hat{w}_{x|\theta}\hat{w}_{x'|\theta}\mathrm{Cov}(\hat{\mu}_{\mathbf{Y}|x,\theta}, \hat{\mu}_{\mathbf{Y}|x',\theta}) + \hat{\mu}_{\mathbf{Y}|x,\theta}\hat{\mu}_{\mathbf{Y}|x',\theta}\mathrm{Cov}(\hat{w}_{x|\theta}, \hat{w}_{x'|\theta}) + 2\hat{w}_{x|\theta}\hat{\mu}_{\mathbf{Y}|x',\theta}\mathrm{Cov}(\hat{\mu}_{\mathbf{Y}|x,\theta}, \hat{w}_{x'|\theta}) \Big].$$

Computing Confidence Intervals. We compute the variance $\hat{\sigma}^2$ using the same scan that computes the modified factor query and the remaining factor queries. For the confidence interval, we add two aggregates to the modified factor query. The first aggregate returns $n_{x,\theta}$, the number of tuples with $\mathbf{X} = x$ for which $\mathbf{Y}$ is observed and θ holds. The second returns $s^2_{\mathbf{Y}|x,\theta}$, the sample variance of $\mathbf{Y}$ over these tuples. Under MAR, the separating sets of $\mathbf{Y}$ and the incomplete attributes in θ are complete. Therefore, we compute the variance of the query estimator from the observed tuples within each group using the stratified estimator [74]: $\hat{\sigma}^2 = \sum_x \hat{\mathbb{P}}_T(\mathbf{X} = x \mid \theta)^2 s^2_{\mathbf{Y}|x,\theta}/n_{x,\theta}$. Under MCAR, $\mathbf{X} = \emptyset$, so the variance reduces to the sample variance $s^2_{\mathbf{Y}|\theta}/n_{\mathbf{Y},\theta}$, where $n_{\mathbf{Y},\theta} = |\sigma_{\mathbf{Y}\neq\perp \wedge \theta}(T)|$ is the number of tuples in which $\mathbf{Y}$ is observed and θ holds.

7.1.1 Group-By Queries. Let $\mathbf{A}_g$ be the attributes that appear in the group-by clause in query q. For each $g \in \mathrm{adom}(\mathbf{A}_g)$, we add $\mathbf{A}_g = g$ to the selection condition and group the modified factor queries by $\mathbf{A}_g$. When $\mathbf{A}_g$ is complete, this is equivalent to applying the estimator to each group independently. When $\mathbf{A}_g$ is incomplete, the factor queries estimate the probability and the aggregation of each group. Theorem 7.3 applies to each group.

7.2 Querying Multiple Relations

7.2.1 Join. Consider $q = avg(\pi_{\mathbf{Y}}(T \bowtie_\Phi S))$, where T and S are relations in database $\mathbf{T}$ and $\mathbf{Y} \subseteq \mathbf{A}_T$. For simplicity, consider $\mathbf{Y} = \{Y\}$, where Y is incomplete. Let $\mathbf{A}^T_\Phi$ and $\mathbf{A}^S_\Phi$ be the join attributes of T and S that occur in Φ, respectively. Let $\mathbf{A}^{q,T}_m$ be the incomplete attributes in $\mathbf{Y} \cup \mathbf{A}^T_\Phi$, with factorization $\mathcal{F}(\mathbf{A}^{q,T}_m)$. Let $\mathbf{A}^{q,S}_m$ be the incomplete attributes in $\mathbf{A}^S_\Phi$, with factorization $\mathcal{F}(\mathbf{A}^{q,S}_m)$. Let $\mathbf{X}^T$ and $\mathbf{X}^S$ be the unions of the separating sets in $\mathcal{F}(\mathbf{A}^{q,T}_m)$ and $\mathcal{F}(\mathbf{A}^{q,S}_m)$, respectively. For tuples $t \in T$ and $s \in S$, let $J_{t,s}$ be the indicator that $t.\mathbf{A}^T_\Phi$ and $s.\mathbf{A}^S_\Phi$ satisfy Φ. The conditional expectation is

$$\mathbb{E}[Y \mid \Phi] = \frac{\sum_y \sum_{t\in T}\sum_{s\in S} y\, \mathsf{P}_{\mathbf{T}}(t.Y = y, J_{t,s} = 1)}{\sum_{t\in T}\sum_{s\in S} \mathsf{P}_{\mathbf{T}}(J_{t,s} = 1)}. \tag{7}$$

When Y occurs in Φ, the joint probability $\mathsf{P}_{\mathbf{T}}(t.Y = y, J_{t,s} = 1)$ evaluates Φ at the same value y. When Y is an attribute of S, the numerator uses $s.Y = y$ in place of $t.Y = y$.

Proposition 7.4. *Consider $q = avg(\pi_Y(T \bowtie_\Phi S))$. Assume that*

$$\lim_{\substack{|T|\to\infty \\ |S|\to\infty}} \frac{1}{|T||S|} \sum_{t\in T}\sum_{s\in S} \mathsf{P}_{\mathbf{T}}(J_{t,s} = 1) > 0.$$

Then $\mathbb{E}[Y \mid \Phi]$ in Equation 7 is the convergent answer of q.

We estimate $\mathbb{E}[Y \mid \Phi]$ without constructing $T \times S$. By the law of total expectation, we compute $\mathbb{E}[Y \mid \Phi]$ over the values of $\mathbf{X}^T$ and $\mathbf{A}^T_\Phi$ using the factor queries for T and S. Let $v^T \in \mathrm{adom}(\mathbf{A}^T_\Phi)$ and $v^S \in \mathrm{adom}(\mathbf{A}^S_\Phi)$. Let $\Phi(v^T, v^S)$ be a Boolean function that is true if the join values v^T and v^S satisfy Φ and false otherwise. Let $x_T \in \mathrm{adom}(\mathbf{X}^T)$ and $x_S \in \mathrm{adom}(\mathbf{X}^S)$. Algorithm 1 generates the factor queries for T and S from $\mathcal{F}(\mathbf{A}^{q,T}_m)$ and $\mathcal{F}(\mathbf{A}^{q,S}_m)$. For each x_T and v^T, let $\hat{\mu}_{Y|x_T,v^T}$ be the sample mean of the observed Y values in tuples with $\mathbf{X}^T = x_T$ and $\mathbf{A}^T_\Phi = v^T$. The sample mean $\hat{\mu}_{Y|x_T,v^T}$ estimates $\mathbb{E}[Y \mid \mathbf{X}^T = x_T, \mathbf{A}^T_\Phi = v^T]$. For relation T, we modify the factor query containing Y to return $\hat{\mu}_{Y|x_T,v^T}$:

$$C_j := \gamma_{\mathbf{X}^T,\mathbf{A}^T_\Phi;\, \hat{\mu}_{Y|x_T,v^T} \leftarrow \mathrm{avg}(Y)}\big(\sigma_{Y\neq\perp \wedge \mathbf{X}^T\neq\perp \wedge \mathbf{A}^T_\Phi\neq\perp}(T)\big).$$

The remaining factor queries are unchanged. For each v^T, let $P^S_\Phi(v^T)$ denote the probability that the join attributes of a tuple drawn from $\mathbb{P}_S$ satisfy Φ with v^T. The factor queries for S estimate $P^S_\Phi(v^T)$ as follows:

$$\hat{P}^S_\Phi(v^T) = \sum_{x_S} \hat{\mathbb{P}}_S(\mathbf{X}^S{=}x_S) \sum_{v^S:\,\Phi(v^T,v^S)} \hat{\mathbb{P}}_S(\mathbf{A}^S_\Phi{=}v^S \mid \mathbf{X}^S{=}x_S). \tag{8}$$

The query estimator $\hat{q}$ returns the following estimate of Equation 7:

$$\hat{q} = \frac{\sum_{x_T}\sum_{v^T} \hat{\mathbb{P}}_T(\mathbf{X}^T{=}x_T)\hat{\mu}_{Y|x_T,v^T}\hat{\mathbb{P}}_T(\mathbf{A}^T_\Phi{=}v^T \mid \mathbf{X}^T{=}x_T)\hat{P}^S_\Phi(v^T)}{\sum_{x_T}\sum_{v^T} \hat{\mathbb{P}}_T(\mathbf{X}^T{=}x_T)\hat{\mathbb{P}}_T(\mathbf{A}^T_\Phi{=}v^T \mid \mathbf{X}^T{=}x_T)\hat{P}^S_\Phi(v^T)}. \tag{9}$$

Hence, computing $\hat{q}$ does not enumerate the tuple pairs in $T \times S$. If Y is conditionally independent of the join attributes given $\mathbf{X}^T$, $\hat{\mu}_{Y|x_T,v^T}$ reduces to $\hat{\mu}_{Y|x_T}$. If Y occurs in Φ, v^T fixes the value of Y and $\hat{\mu}_{Y|x_T,v^T}$ equals that value.

Marked Nulls. We apply Assumption 1 separately to T and S. Let $\mathrm{Null}_\Phi(T) = \{\perp_k{:}\perp_k \in t.\mathbf{A}^T_\Phi \text{ for some } t \in T\}$. We define $\mathrm{Null}_\Phi(S)$ similarly for S. If $\mathrm{Null}_\Phi(T) \cap \mathrm{Null}_\Phi(S) = \emptyset$, we handle marked nulls as in Section 7.1, separately for T and S.

For each $\perp_k \in \mathrm{Null}_\Phi(T) \cap \mathrm{Null}_\Phi(S)$, all occurrences of $\perp_k$ in $\mathbf{A}^T_\Phi$ and $\mathbf{A}^S_\Phi$ have the same value. Hence, the conditions $=$, $\leq$, and $\geq$ between two occurrences of $\perp_k$ are true, and the conditions $\neq$, $<$, and $>$ are false. Let $\hat{\mathbf{N}}$ and $\hat{\mathbf{D}}$ denote the numerator and denominator of Equation 9, respectively.

Theorem 7.5. *Consider $q = avg(\pi_Y(T \bowtie_\Phi S))$. If Assumption 1 holds for T and S, then $\hat{q} \xrightarrow{p} \mathbb{E}[Y \mid \Phi]$, where $\mathbb{E}[Y \mid \Phi]$ in Equation 7 is the convergent answer of q. The confidence interval is $[\hat{q} \pm z(\alpha)\hat{\sigma}]$ with probability $1-\alpha$, where*

$$\hat{\sigma}^2 \approx \frac{1}{\hat{\mathbf{D}}^2}\left(\mathrm{Var}(\hat{\mathbf{N}}) + \hat{q}^2\,\mathrm{Var}(\hat{\mathbf{D}}) - 2\hat{q}\,\mathrm{Cov}(\hat{\mathbf{N}}, \hat{\mathbf{D}})\right).$$

Computing Confidence Intervals. We compute $\hat{\sigma}^2$ from the same factor-query outputs used to compute $\hat{q}$. We extend the factor queries to return the group sizes and the within-group variance $s^2_{Y|x_T,v^T}$. The group sizes, within-group variances, and join-value

probabilities are used to compute $\mathrm{Var}(\hat{\mathbf{N}})$, $\mathrm{Var}(\hat{\mathbf{D}})$, and $\mathrm{Cov}(\hat{\mathbf{N}}, \hat{\mathbf{D}})$. The covariance is nonzero because $\hat{\mathbf{N}}$ and $\hat{\mathbf{D}}$ share the match probability $\hat{P}^S_\Phi(v^T)$ and the estimates computed from T. The variance and covariance calculations include covariances across join values v^T and v'^T.

Self-Join. For $q = avg(\pi_Y(T \bowtie_\Phi T))$, we use Equation 9 with $\hat{\mathbb{P}}_S$ replaced by $\hat{\mathbb{P}}_T$ and compute the factor queries once. For two distinct tuples $t, t' \in T$, Equation 9 applies with $\hat{\mathbb{P}}_S$ replaced by $\hat{\mathbb{P}}_T$. The equation does not apply to a pair (t, t) because both join-attribute values come from the same tuple and are dependent [26]. There are $|T|$ such pairs among the $|T|^2$ pairs in $T \times T$. Their fraction is therefore $\frac{1}{|T|}$, which converges to zero as $|T| \to \infty$. Hence, these pairs do not change the convergent answer.

Attributes from Both Relations. When $\mathbf{Y} \subseteq \mathbf{A}_T \cup \mathbf{A}_S$, the average of each attribute in $\mathbf{Y}$ is estimated separately using Equation 9. The consistency and the confidence interval follow as in Theorem 7.5.

7.2.2 Union and Set Difference. Consider relations T and S with the same attributes $\mathbf{Y}$, and, for simplicity, let $\mathbf{Y} = \{Y\}$. For each y, let $\mathsf{P}_T(y)$ and $\mathsf{P}_S(y)$ be the probabilities estimated from the observed data that y appears in T and S, respectively. If these occurrence events are independent and the output is not empty, the estimators for $q = avg(\pi_Y(T \cup S))$ and $q' = avg(\pi_Y(T \setminus S))$ are $\hat{q} = \frac{\sum_y y\,[1-(1-\mathsf{P}_T(y))(1-\mathsf{P}_S(y))]}{\sum_y [1-(1-\mathsf{P}_T(y))(1-\mathsf{P}_S(y))]}$ and $\hat{q}' = \frac{\sum_y y\,\mathsf{P}_T(y)(1-\mathsf{P}_S(y))}{\sum_y \mathsf{P}_T(y)(1-\mathsf{P}_S(y))}$. Consistency and the confidence intervals follow from those of the input probabilities. The occurrence events are independent when T and S are distinct relations and $\mathrm{Null}(T) \cap \mathrm{Null}(S) = \emptyset$, and the estimators do not apply otherwise.

8 Direct Estimation for Non-aggregation Queries

The answer to a non-aggregation query is a distribution over its possible results and is not expressible through expectations. In this section, we use an estimator for Boolean queries as the base estimator for non-aggregation queries. We use the log transformation to write the query probability as an expectation, which CADE estimates. Then the exponential maps the estimate back to a probability.

Boolean Queries. Consider a Boolean query $q = (\sigma_\theta(T) \neq \emptyset)$. The answer to q is a distribution over {true, false}. Thus, we estimate $\mathsf{P}_{q(T)}(\text{false})$ and obtain $\mathsf{P}_{q(T)}(\text{true})$ as its complement. For each tuple $t \in T$, let $\mathsf{P}_q(t)$ denote the probability that t satisfies θ.

Proposition 8.1. *Consider $q = (\sigma_\theta(T) \neq \emptyset)$ under non-repeating nulls. Then*

$$\mathsf{P}_{q(T)}(\text{false}) = \prod_{t \in T} \big(1 - \mathsf{P}_q(t)\big). \tag{10}$$

For aggregation queries, the law of total expectation expresses the convergent answer as a sum over the values of a separating set. Equation 10 is a product over tuples, so we cannot estimate it by the same aggregation rule.

We apply the log transformation to write the product in Equation 10 as a sum. Taking the logarithm gives $\sum_{t \in T} \log(1 - \mathsf{P}_q(t))$. Let $\mathbf{X}$ be the union of the separating sets in $\mathcal{F}(\mathbf{A}^q_m)$. For each tuple t with observed $\mathbf{X} = x$, let $\mathsf{P}_{\hat{q}}(t)$ be the estimate of $\mathsf{P}_q(t)$ computed from the factor queries. By Proposition 4.2, $\mathsf{P}_{\hat{q}}(t)$ is consistent for $\mathsf{P}_q(t)$. We define an attribute L with value $L(t) = \log(1 - \mathsf{P}_{\hat{q}}(t))$ for each tuple t with observed $\mathbf{X}$. The attribute L is missing for each tuple with missing $\mathbf{X}$. Let $\mu_L = \frac{1}{|T|} \sum_{t \in T} \log(1 - \mathsf{P}_q(t))$. Hence, $|T| \cdot \mu_L$ is the logarithm of Equation 10.

Let $q_L = avg(\pi_L(T))$. We apply the aggregation estimator in Section 7.1 to q_L and obtain the estimate $\hat{\mu}_L$ of μ_L. We map the result to probability via the exponential. Thus, $\mathrm{EXP}(|T| \cdot \hat{\mu}_L)$ estimates $\mathsf{P}_{q(T)}(\text{false})$, and $\hat{q} = 1 - \mathrm{EXP}(|T| \cdot \hat{\mu}_L)$ estimates $\mathsf{P}_{q(T)}(\text{true})$. If $\mathsf{P}_{\hat{q}}(t) = 1$ for some tuple, $\hat{q}$ returns 1 without evaluating the logarithm.

For each $x \in \mathrm{adom}(\mathbf{X})$, let $\hat{\mu}_{L|x}$ be the sample mean of $L(t)$ over the tuples with observed $\mathbf{X} = x$. By Theorem 7.3, the aggregation estimator gives $\hat{\mu}_L = \sum_x \hat{\mu}_{L|x} \hat{\mathbb{P}}_T(\mathbf{X} = x)$. Hence, we do not compute $\mathsf{P}_{\hat{q}}(t)$ for tuples with missing $\mathbf{X}$ because their probabilities are included through $\hat{\mathbb{P}}_T(\mathbf{X} = x)$ in $\hat{\mu}_L$.

Let $A_1, \ldots, A_r$ be the incomplete attributes in $\mathbf{A}^q \cup \mathbf{F}_{\mathbf{A}^q_m}$. Let $\Gamma_1, \ldots, \Gamma_{2^r}$ be all conjunctions of the form $\Gamma_i = \bigwedge_{j=1}^r \gamma^i_j$, where γ^i_j is either $A_j = \perp$ or $A_j \neq \perp$. Each predicate Γ_i selects the tuples that share the same set of attributes with missing values.

Marked Nulls. When tuples share a marked null in $\mathbf{A}^q_m$ or its separating sets, all occurrences of the shared symbol have the same value. Hence, the events that these tuples satisfy θ are dependent, and Equation 10 does not hold. Under Assumption 1, the covariance introduced by tuples sharing a marked null vanishes as $|T| \to \infty$. CADE-Bool remains unchanged under marked nulls.

Theorem 8.2. *Consider $q = (\sigma_\theta(T) \neq \emptyset)$. Then $\hat{q}$ is consistent for $\mathsf{P}_{q(T)}(\text{true})$. If Assumption 1 holds, the confidence interval is $[\hat{q} \pm z(\alpha)\hat{\sigma}]$ with probability $1 - \alpha$, where $\hat{\sigma}^2 \approx (1 - \hat{q})^2 |T|^2 \,\mathrm{Var}(\hat{\mu}_L)$.*

Computing Confidence Intervals. We compute the variance $\hat{\sigma}^2$ of Theorem 8.2 in the same scan that produces $\hat{q}$. We obtain $\mathrm{Var}(\hat{\mu}_L)$ as in Theorem 7.3 with the attribute L in place of $\mathbf{Y}$, including the reductions under MAR and MCAR.

Algorithm 4 CADE-Bool$(q, \mathcal{S})$

Input: Boolean query $q = (\sigma_\theta(T) \neq \emptyset)$, database schema $\mathcal{S}$
Output: Query estimator $\hat{q}$ for $\mathsf{P}_{q(T)}(\text{true})$

1: For each Γ_i and each $x \in \mathrm{adom}(\mathbf{X})$, compute $\mathsf{P}_{\hat{q}}(t)$ for the tuples in $\sigma_{\Gamma_i \wedge \mathbf{X}=x}(T)$ using Algorithm 1
2: **if** $\mathsf{P}_{\hat{q}}(t) = 1$ for some tuple t **then**
3: **return** 1
4: Compute $L(t) = \log\big(1 - \mathsf{P}_{\hat{q}}(t)\big)$
5: Let $q_L := avg(\pi_L(T))$
6: $\hat{q}_L \leftarrow$ the estimate of q_L by the aggregation estimator
7: **return** $\hat{q} := \gamma_{\emptyset;\, Pr \leftarrow 1-\mathrm{EXP}(|T| \cdot \hat{\mu}_L)}(\hat{q}_L)$

8.1 Querying a Single Relation

Reporting Answers. For both non-repeating and marked nulls, we report the marginal probability of each output tuple. Representing dependencies among output tuples requires additional output information and is outside the scope of this work [18].

8.1.1 Selection. Consider $q = \sigma_\theta(T)$. We view the selection as a set of Boolean queries over the values of $\mathbf{A}_T$. For each $y \in \mathrm{adom}(\mathbf{A}_T)$, the output tuple y appears in $q(T)$ if the Boolean query $q_y = (\sigma_{\theta \wedge (\mathbf{A}_T = y)}(T) \neq \emptyset)$ is true. For each y, let $L_y(t) = \log(1 - \mathsf{P}_{\hat{q}_y}(t))$

for each tuple t with observed $\mathbf{X}$, where $\mathsf{P}_{\hat{q}_y}(t)$ estimates $\mathsf{P}_{q_y}(t)$. Let $q_{L_y} = avg(\pi_{L_y}(T))$. Following Algorithm 4, we apply the aggregation estimator to q_{L_y} for all $y \in \text{adom}(\mathbf{A}_T)$ in one pass by grouping the query by $\mathbf{A}_T$. Let $\hat{q}_{L_y}$ denote the resulting query, which returns $\hat{\mu}_{L_y}$ for each y. The final estimation query is $\hat{q} := \gamma_{\mathbf{A}_T;\, a \leftarrow 1-\text{EXP}(|T|\cdot\hat{\mu}_{L_y})}(\hat{q}_{L_y})$.

Proposition 8.3. *Consider $q = \sigma_\theta(T)$. For each fixed $y \in \text{adom}(\mathbf{A}_T)$, $\mathsf{P}_{\hat{q}}(y)$ is consistent and, if Assumption 1 holds, has a confidence interval obtained from Theorem 8.2.*

8.1.2 Projection. Consider $q = \pi_{\mathbf{Y}}(T)$. We view the projection as a set of Boolean queries. For each value $y \in \text{adom}(\mathbf{Y})$, the output tuple y appears in $q(T)$ if the query $q_y = (\sigma_{\mathbf{Y}=y}(T) \neq \emptyset)$ is true.

For each y, let $L_y(t) = \log(1 - \mathsf{P}_{\hat{q}_y}(t))$ for each tuple t with observed $\mathbf{X}$, where $\mathsf{P}_{\hat{q}_y}(t)$ estimates $\mathsf{P}_{q_y}(t)$. Let $q_{L_y} = avg(\pi_{L_y}(T))$. Following Algorithm 4, we apply the aggregation estimator to q_{L_y} for all $y \in \text{adom}(\mathbf{Y})$ in one pass by grouping the query by $\mathbf{Y}$. Let $\hat{q}_{L_y}$ denote the resulting query, which returns $\hat{\mu}_{L_y}$ for each y. The final estimation query is $\hat{q} := \gamma_{\mathbf{Y};\, a \leftarrow 1-\text{EXP}(|T|\cdot\hat{\mu}_{L_y})}(\hat{q}_{L_y})$.

Proposition 8.4. *Consider $q = \pi_{\mathbf{Y}}(T)$. For each fixed $y \in \text{adom}(\mathbf{Y})$, $\mathsf{P}_{\hat{q}}(y)$ is consistent and, if Assumption 1 holds, has a confidence interval obtained from Theorem 8.2.*

8.1.3 Min and Max Aggregation Functions. Consider query $q = max(\pi_Y(\sigma_\theta(T)))$. The answer of q is at most y if and only if no tuple satisfies $\theta \wedge Y > y$. Hence, for the Boolean query $q_y = (\sigma_{\theta \wedge Y > y}(T) \neq \emptyset)$, the probability that the answer of q is at most y equals $\mathsf{P}_{q_y(T)}(\text{false})$. Evaluating q_y for each $y \in \text{adom}(Y)$ estimates the answer of q. The minimum is symmetric.

8.2 Querying Multiple Relations

Boolean Join. Consider the Boolean join query $q = (T \bowtie_\Phi S \neq \emptyset)$. Let $\mathsf{P}_q(t \bowtie s)$ denote the probability that the tuple pair (t, s) satisfies Φ. We apply the Boolean estimator to the join, with $\mathsf{P}_q(t \bowtie s)$ in place of $\mathsf{P}_q(t)$. Let $\mathbf{X}^T$ and $\mathbf{X}^S$ be the unions of the separating sets of the incomplete join attributes of T and S. For each tuple pair (t, s) with observed $\mathbf{X}^T$ and $\mathbf{X}^S$, the factor queries for T and S compute $\mathsf{P}_{\hat{q}}(t \bowtie s)$ [4]. By Proposition 4.2, $\mathsf{P}_{\hat{q}}(t \bowtie s)$ is consistent for $\mathsf{P}_q(t \bowtie s)$. We define an attribute L with value $L(t, s) = \log(1 - \mathsf{P}_{\hat{q}}(t \bowtie s))$ for each tuple pair with observed $\mathbf{X}^T$ and $\mathbf{X}^S$. Let $q_L = avg(\pi_L(T \bowtie_\Phi S))$. Following Algorithm 4, we apply the aggregation estimator to q_L. Let $\hat{q}_L$ denote the resulting query, which returns $\hat{\mu}_L$. By Theorem 7.5 with L in place of Y, $\hat{\mu}_L$ is consistent, and tuple pairs with missing $\mathbf{X}^T$ or $\mathbf{X}^S$ are included through the estimated group probabilities. The query estimator is $\hat{q} := \gamma_{\emptyset;\, Pr \leftarrow 1-\text{EXP}(|T \bowtie_\Phi S|\cdot\hat{\mu}_L)}(\hat{q}_L)$.

Under non-repeating nulls, two joined tuples are dependent only when they share a tuple from T or S [26]. Hence, Equation 10 need not hold over the join. However, the fraction of dependent pairs among all pairs of joined tuples is at most $\frac{1}{|T|} + \frac{1}{|S|}$, which converges to 0 as $|T| \to \infty$ and $|S| \to \infty$ [4]. Hence, the covariance they introduce vanishes. For a self-join, the fraction of dependent pairs is at most $\frac{2}{|T|}$, and the same argument applies with $S = T$.

Marked Nulls. The Boolean-join estimator remains unchanged under marked nulls. Marked nulls in the join attributes, including symbols in $\text{Null}_\Phi(T) \cap \text{Null}_\Phi(S)$, are handled as in Section 7.2.

Proposition 8.5. *Consider $q = (T \bowtie_\Phi S \neq \emptyset)$. Then $\hat{q}$ is consistent. If Assumption 1 holds for T and S, the confidence interval is $[\hat{q} \pm z(\alpha)\hat{\sigma}]$ with probability $1 - \alpha$, where $\hat{\sigma}^2 \approx (1 - \hat{q})^2 |T \bowtie_\Phi S|^2 \,\text{Var}(\hat{\mu}_L)$.*

8.2.1 Join. Consider $q = T \bowtie_\Phi S$. We view the join as a set of Boolean queries over the values of $\mathbf{A}_T \cup \mathbf{A}_S$. For each $y \in \text{adom}(\mathbf{A}_T \cup \mathbf{A}_S)$, the value y appears in $q(T, S)$ if the Boolean query $q_y = (\sigma_{(\mathbf{A}_T \cup \mathbf{A}_S)=y}(T \bowtie_\Phi S) \neq \emptyset)$ is true. For each y, let $L_y(t, s) = \log(1 - \mathsf{P}_{\hat{q}_y}(t \bowtie s))$ for each tuple pair with observed $\mathbf{X}^T$ and $\mathbf{X}^S$. Let $q_{L_y} = avg(\pi_{L_y}(T \bowtie_\Phi S))$. We apply the Boolean-join estimator to each q_y and compute all values in one pass by grouping the aggregation query by $\mathbf{A}_T \cup \mathbf{A}_S$. Let $\hat{q}_{L_y}$ denote the resulting query, which returns $\hat{\mu}_{L_y}$ for each y. The final estimation query is $\hat{q} := \gamma_{\mathbf{A}_T \cup \mathbf{A}_S;\, a \leftarrow 1-\text{EXP}(|T \bowtie_\Phi S|\cdot\hat{\mu}_{L_y})}(\hat{q}_{L_y})$.

Proposition 8.6. *Consider $q = T \bowtie_\Phi S$. For each fixed $y \in \text{adom}(\mathbf{A}_T \cup \mathbf{A}_S)$, $\mathsf{P}_{\hat{q}}(y)$ is consistent and, if Assumption 1 holds for T and S, has a confidence interval obtained from Proposition 8.5.*

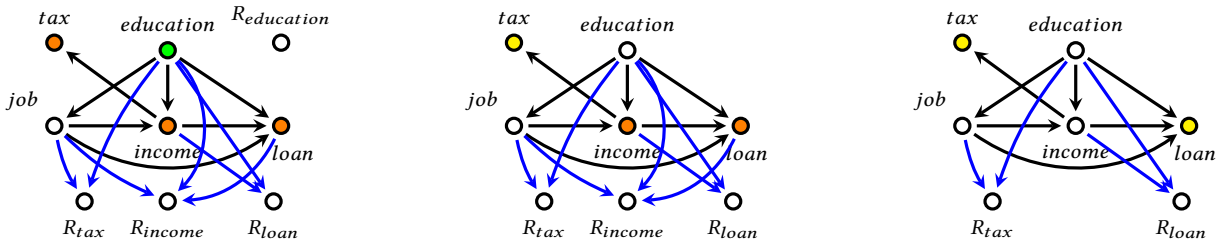
8.2.2 Union and Set Difference. Consider relations T and S with the same attributes $\mathbf{Y}$. For an output tuple y, let $\mathsf{P}_T(y)$ denote the probability that y appears in T. We define $\mathsf{P}_S(y)$ similarly for S. The preceding estimators return $\mathsf{P}_T(y)$ and $\mathsf{P}_S(y)$ with their output tuples. If the occurrence events for y in T and S are independent, the estimators are $\mathsf{P}_{\hat{q}}(y) = 1 - (1 - \mathsf{P}_T(y))(1 - \mathsf{P}_S(y))$ for $q = T \cup S$, and $\mathsf{P}_{\hat{q}'}(y) = \mathsf{P}_T(y)(1 - \mathsf{P}_S(y))$ for $q' = T \setminus S$. Consistency and the confidence intervals follow from those of the input probabilities.

The occurrence events for y in T and S are independent when T and S are distinct relations and $\text{Null}(T) \cap \text{Null}(S) = \emptyset$. When $\text{Null}(T) \cap \text{Null}(S) \neq \emptyset$, all occurrences of $\perp_k$ have the same value, and the occurrence events are dependent. Unlike the join estimator, which averages over the tuple pairs, the union and set difference estimators combine two probabilities for a fixed y, and the dependence introduced by a shared symbol does not vanish as $|T| \to \infty$ and $|S| \to \infty$. Hence, the estimators do not apply.

8.2.3 Combining Operators. We evaluate compositions of selections, projections, and joins through the Boolean queries of their output values. For example, for $q = \pi_{\mathbf{Y}}(\sigma_\theta(T \bowtie_\Phi S))$, we define $q_y = (\sigma_{\theta \wedge (\mathbf{Y}=y)}(T \bowtie_\Phi S) \neq \emptyset)$ and apply the Boolean-join estimator to each q_y. Union and set difference combine the probabilities returned by their input estimators. Hence, every operator returns its output values with their marginal probabilities. Reporting these marginals does not assume independence between the output values. Unlike extensional evaluation over probabilistic databases, which requires independence conditions at each operator of a query plan [19, 66], the composition requires no assumption beyond those stated for the operators.

8.3 Query Complexity of CADE

For every query in this section and the preceding section, the number of factor queries is determined by the factorization, and the number of joins among them is given by Proposition 4.2. For a join query, the factor queries are generated separately for T and S, and each relation is scanned once. The estimators add one aggregation query: the modified factor query for aggregation queries, and q_L for non-aggregation queries. Union and set difference add no factor



(a) Non-factorizable MNAR **(b) Resolve *education*** **(c) Resolve *income***

Figure 2: MNAR-Repair example on non-factorizable MNAR data, where ● MCAR, ● MAR, ● MNAR, ○ Complete.

queries because their input estimators return the output values with their probabilities. The aggregates required for the confidence intervals are computed in the same factor queries, so the confidence intervals require no additional query.

9 Repairing Non-Factorizable Data

Our query estimation algorithms do not support querying attributes with non-factorizable MNAR data. Complete imputation eliminates missingness but is resource-intensive and prone to systematic bias under MNAR [31, 73, 75]. We instead repair selected incomplete attributes to make non-factorizable MNAR data factorizable. We call this approach *MNAR-Repair*. For MAR attributes with large separating sets, we also introduce *MAR-Repair*. A *partial repair* of a relation T replaces at least one null symbol in $\text{Null}(T)$ with a value in the domain of its attribute. All occurrences of the null symbol in T take this value. MNAR-Repair applies to both marked and non-repeating nulls. The MAR-Repair algorithm and its guarantees consider non-repeating nulls.

Definition 9.1. Given a relation T with non-factorizable MNAR data, a partial repair T' of T is an **MNAR-Repair** of T if no missingness mechanism in the m-graph of T' has any incomplete parents.

Let T be a relation with non-factorizable MNAR data. Each $A \in \mathbf{A}_m$ has repair cost $c(A) > 0$. The *Minimal MNAR-Repair* problem finds a subset $\mathbf{C} \subseteq \mathbf{A}_m$ of minimum total cost $\sum_{A \in \mathbf{C}} c(A)$ such that no missingness mechanism has an incomplete parent after repairing $\mathbf{C}$. For Minimal MNAR-Repair, repairing an attribute replaces every null symbol in the attribute and makes the attribute complete.

Theorem 9.2. *The Minimal MNAR-Repair problem is NP-hard.*

Approximating MNAR-Repair. We formulate Minimal MNAR-Repair as a weighted Set Cover problem and apply its greedy approximation algorithm [14]. An *MNAR edge* has the form $A \to R_B$, where A and B are incomplete attributes. Repairing A makes A complete, so $A \to R_B$ no longer represents MNAR missingness. Repairing B makes B complete, so R_B is no longer the missingness mechanism of an incomplete attribute. The *gain* of repairing Y is the number of remaining MNAR edges that repairing Y eliminates. The algorithm maintains a repair set $\mathbf{C} \subseteq \mathbf{A}_m$, initially empty. At each iteration, it computes the gain-to-cost ratio $\eta(Y) = \text{gain}(Y)/c(Y)$ for every $Y \in \mathbf{A}_m \setminus \mathbf{C}$. The algorithm adds the attribute Y with the largest $\eta(Y)$ to $\mathbf{C}$. It then repairs the attributes in $\mathbf{C}$.

Using adjacency lists and a max-heap, the algorithm runs in $O((r+|\mathbf{A}_m|)\log|\mathbf{A}_m|)$ time, where r is the initial number of MNAR edges. It produces a repair within a factor $H_d = \sum_{i=1}^{d} 1/i \le \ln d + 1$ of the minimum-cost MNAR-Repair, where d is the largest initial gain (Theorem E.4 in Appendix E.1). Algorithm 5 gives the pseudocode. The extension to multiple relations is available in [4]. MNAR-Repair also applies to factorizable MNAR data when the separating sets are large (Appendix E.4).

Algorithm 5 MNAR-Repair

Input: m-graph G_T and repair costs $c : \mathbf{A}_m \to \mathbb{R}_{>0}$
Output: repair set $\mathbf{C} \subseteq \mathbf{A}_m$

```
C ← ∅
U ← set of all MNAR edges in G_T
while U ≠ ∅ do
    for each Y ∈ A_m \ C do
        gain(Y) ← |{ A → R_B ∈ U | A = Y or B = Y }|
        η(Y) ← gain(Y)/c(Y)
    Choose Y* = arg max_Y η(Y)
    C ← C ∪ {Y*}
    U ← U \ { A → R_B ∈ U | A = Y* or B = Y* }
return C
```

Example 9.3. In Figure 2a, the root attribute *education* is MCAR. Repairing *education* makes *education* complete and makes *tax* MAR (Figure 2b). Both *loan* and *income* have one outgoing edge, so their gain is equal. Hence, repairing either *loan* or *income* eliminates the two remaining MNAR dependencies. Figure 2c shows the result of repairing *income*. The resulting repair eliminates all MNAR dependencies without imputing every incomplete attribute.

Imputation. The attributes in $\mathbf{C}$ can be imputed by an expert or by a trained imputation model. For model-based imputation, we use Markov Blanket Imputation (MBI), which imputes the missing values of each $Y \in \mathbf{C}$ from the conditional distribution of Y given its parents, children, and spouses in the m-graph [3, 41]. Conditioned on its Markov blanket, Y is independent of the remaining attributes.
MAR-Repair. For an MAR attribute Y with a large separating set, we introduce MAR-Repair. For non-repeating nulls, MAR-Repair uniformly selects missing values within each separating-set group so that Y has the same probability of being missing in every group. For marked nulls, MAR-Repair selects null symbols and imputes all occurrences of every selected symbol so that Y has the same probability of being missing in every group. The repaired attribute becomes MCAR, and its factor query therefore has no separating set.

Theorem E.1 establishes this for non-repeating nulls. Appendix E.2 gives the algorithm and its minimum-cost guarantee.

Example 9.4. Table 3 illustrates MAR-Repair on a relation in which *income* is MAR with the complete separating set $\{job\}$. This relation is a separate instance of *jobs* from the ones in Table 1b and Figure 1a. The column ω reports, for each *job* group, the fraction of its tuples in which *income* is missing. In Table 3a, this fraction is 2/3 for Executives and 1/3 for the other groups, so *income* is MAR. Table 3b shows one MAR-Repair of *income*, which imputes one Executive tuple and equalizes the fraction to 1/3 in every group. The repaired attribute is MCAR because the missing values to impute are selected uniformly within each group, not because a single repaired table has equal fractions.

Table 3: MAR-Repair Example

(a) MAR relation *jobs*

ID	*job*	*income*	ω
1	Assistant	30K	
2	Assistant	⊥	1/3
3	Assistant	70K	
4	Executive	⊥	
5	Executive	⊥	2/3
6	Executive	900K	
7	Consultant	100K	
8	Consultant	⊥	1/3
9	Consultant	130K	

(b) MAR-Repair of *jobs*

ID	*job*	*income*	ω
1	Assistant	30K	
2	Assistant	⊥	1/3
3	Assistant	70K	
4	Executive	⊥	
5	Executive	800K	1/3
6	Executive	900K	
7	Consultant	100K	
8	Consultant	⊥	1/3
9	Consultant	130K	

Sampled MAR-Repair. When the smallest fraction of missing values over the groups is near zero, MAR-Repair imputes most of the missing values of Y. Following the sample-and-clean paradigm [72], we instead repair a subset of the relation sampled uniformly without replacement. We evaluate the query over the repaired sample and compute its confidence interval. If the width of this interval exceeds a threshold specified by the user, we add previously unsampled tuples to an unrepaired version of the current sample, recompute the smallest fraction of missing values over the groups, repair the expanded sample, and evaluate the query again. We call this procedure *cumulative sampling*. The interval used to stop must account for both the uniform sampling without replacement and the random selection of the imputed cells. An interval that accounts for only one of these sources does not establish the stated confidence level for the answer over the entire relation. Appendix E.3 reports the behavior of cumulative sampling on a real-world dataset.

10 Experiments

Methods for Factorizable MNAR Data. We evaluate CADE over aggregation and non-aggregation queries on MCAR, MAR, and factorizable MNAR data. CADE uses the direct estimators of Sections 7 and 8. We compare CADE with the following methods. *Causality-Aware Monte Carlo (CAMC)* uses the Monte Carlo evaluation of Section 6. It processes each factorization in reverse order and draws $H = 783$ sampled repairs directly from observed tuples. CAMC evaluates the input query over these repairs using relation encoding. The reported CAMC time includes factorization sampling and relation encoding. We divide each of these two times equally among the queries evaluated over the same sampled repairs. *Causality-Aware Extensional Evaluation (CAEX)* uses the extensional evaluation of Section 5 on data with non-repeating nulls. *Query Encoding (QE)* uses the query encoding of [18]. The method in [18] receives the distributions as input. QE first computes these distributions from the factor queries. In all experiments, QE samples $H = 783$ repairs from these distributions. For each sampled repair, QE substitutes the sampled values into a separate copy of the input query. It combines the H query results using `UNION ALL`. For non-aggregation queries, we also compare with *Certain*. It returns the output tuples that occur in every repair, which is called naive evaluation [1, 39]. Certain returns empty answers for aggregation queries. Therefore, we do not report it in the aggregation results. For aggregation queries, we also compare CADE with an interval-based method that returns an interval of possible answers [76]. We also compare CADE with a distribution-centric extension of [45]. It materializes the joint distributions of the input relations before evaluating the query [4].

Methods for Non-Factorizable MNAR Data. We use MNAR-Repair on non-factorizable MNAR data. We compare GAIN [75], MICE [70], and MissForest [65]. We also compare MNAR-Repair followed by each method. We evaluate MNAR-Repair followed by MBI [41]. The standalone methods impute every missing cell and do not model the causes of missingness [6, 48, 78]. The methods preceded by MNAR-Repair impute only the cells that MNAR-Repair selects. Each standalone method produces one complete relation, and we evaluate the query directly on that relation, which returns a single answer without a confidence interval. MNAR-Repair returns partially repaired relations. We evaluate queries over these relations using CADE.

System Specification. We run our queries over PostgreSQL 15.8. All experiments run on a Linux server with an Intel Xeon E5-2670 v3 @2.30GHz processor and 512GB RAM.

Datasets. We use 13 real-world datasets listed in Table 4. We inject missingness in three complete real-world datasets. Ten datasets have naturally occurring missingness. For injected data, we use missingness rates of 5%, 10%, and 20% under each mechanism with non-repeating and marked nulls. For MCAR data, we construct an m-graph in which the missingness in each incomplete attribute is independent of the rest of the attributes. For MAR data, we construct an m-graph in which the missingness in each incomplete attribute depends on the values of the complete attributes. For factorizable MNAR data, we fix an order over the selected incomplete attributes. The missingness in each attribute can depend on the values of other incomplete attributes. For example, we inject missingness in attribute A based on a complete attribute B, which gives us MAR on A. When we inject B with MCAR, it turns A into MNAR. For datasets with naturally occurring missingness, we find the missingness mechanism via the MVPC algorithm [67].

Queries. We evaluate 30 queries for each injected dataset. Half are aggregation queries and half are non-aggregation queries. They include five randomly generated SQL queries for each missingness type and are listed in [4]. The queries range from single-relation queries to joins over the number of relations reported in Table 4.

Table 5: Mean evaluation metric on injected factorizable MNAR data with marked nulls.

Dataset	Query type and metric	CAMC			CADE			QE			Certain			Interval-Based			Distribution-Centric		
		5%	10%	20%	5%	10%	20%	5%	10%	20%	5%	10%	20%	5%	10%	20%	5%	10%	20%
Bank Marketing	Non-aggregation (TVD)	0.0362	0.0415	0.0631	0.0727	0.0927	0.1387	0.0362	0.0415	0.0631	0.0514	0.0577	0.0894	N/A	N/A	N/A	N/A	N/A	N/A
	Non-aggregation (Δw)	0.0179	0.0185	0.0209	0.1501	0.1551	0.1548	0.0179	0.0185	0.0209	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
	Aggregation (Accuracy %)	97.0627	96.2921	94.8560	97.1518	96.1288	95.9229	97.0627	96.2921	94.8560	N/A	N/A	N/A	–	–	–	–	–	–
	Aggregation (Δw)	0.0039	0.0057	0.0048	0.2699	0.2703	0.2840	0.0039	0.0057	0.0048	N/A	N/A	N/A	–	–	–	N/A	N/A	N/A
NYC Taxi Trips	Non-aggregation (TVD)	0.0106	–	–	0.0586	0.0878	0.1473	–	–	–	0.0750	0.1099	0.1716	N/A	N/A	N/A	N/A	N/A	N/A
	Non-aggregation (Δw)	0.0126	–	–	0.0004	0.0006	0.0011	–	–	–	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
	Aggregation (Accuracy %)	99.9898	–	–	99.9881	99.8333	99.1086	–	–	–	N/A	N/A	N/A	–	–	–	–	–	–
	Aggregation (Δw)	0.0000	–	–	0.0067	0.0066	0.0064	–	–	–	N/A	N/A	N/A	–	–	–	N/A	N/A	N/A
Bitcoin Heist	Non-aggregation (TVD)	0.1949	0.3912	–	0.0437	0.0637	0.1022	–	–	–	0.3181	0.3299	0.3562	N/A	N/A	N/A	N/A	N/A	N/A
	Non-aggregation (Δw)	0.0150	0.0224	–	0.0000	0.0000	0.0000	–	–	–	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
	Aggregation (Accuracy %)	–	–	–	97.7600	96.9204	92.1491	–	–	–	N/A	N/A	N/A	–	–	–	–	–	–
	Aggregation (Δw)	–	–	–	0.1041	0.1030	0.1023	–	–	–	N/A	N/A	N/A	–	–	–	N/A	N/A	N/A

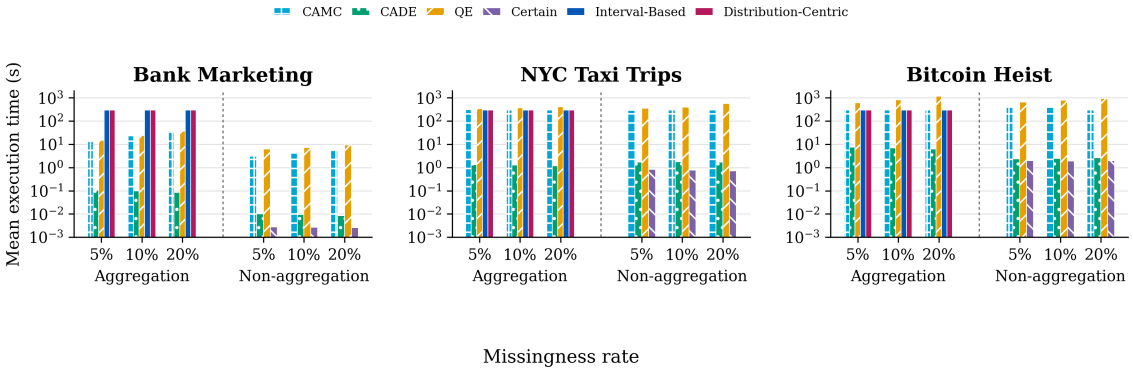


Figure 3: Mean execution time on injected factorizable MNAR data with marked nulls. The vertical axis is logarithmic.

Table 4: Dataset properties

Dataset Name	Missingness (%)	# Tuples	# Relations
Injected MCAR, MAR, and MNAR Data			
Bank Marketing [47]	injected 5%, 10%, and 20%	45K	4
NYC Taxi Trips [60]		729K	4
Bitcoin Heist [69]		2.9M	4
Real-world MCAR Data			
Building Permits [51]	56.65%	780K	2
Street Construction Permits [53]	26%	2.42M	2
Real-world MAR Data			
Employees Info [52]	22.23%	32.1K	2
SF Salaries [35]	15.31%	149K	2
Heart Health [58]	10%	445K	2
Real-world Factorizable MNAR Data			
Student Admission [2]	6.5%	157	3
Aircraft Performance [50]	13.28%	861	4
Medical Condition [43]	23.76%	10K	3
Real-world Non-factorizable MNAR Data			
Communities & Crime [68]	13.8%	2.2K	2
NHANES [49]	31.7%	128K	2

Evaluation Metrics. We report wall-clock runtimes. We limit the execution time of every aggregation and non-aggregation query to five minutes. A timed-out query contributes five minutes to the mean runtime. We use "–" for a timeout or unavailable ground truth in tables and N/A when a metric does not apply to a method. For an aggregation estimate $\hat{a}$ and a nonzero ground-truth value a, we define accuracy as $(1-|\hat{a}-a|/|a|)\times 100\%$ and report its average over the queries. For a nonzero point estimate, we define the normalized confidence-interval width Δw as the confidence-interval width divided by the absolute value of the point estimate. When the point estimate is zero, we report N/A. We use Total Variation Distance (TVD) between the estimated distribution and the ground-truth distribution: $d_{TV}(\tilde{P}, P) = \frac{1}{2}\sum_{y\in\Omega}|\tilde{P}(y) - P(y)|$. TVD ranges from 0 to 1, where 0 indicates identical distributions and 1 indicates that the two distributions have no output tuple in common. We report TVD for non-aggregation queries, whose answers are distributions over output tuples. Aggregation queries return numerical estimates, so we report accuracy rather than TVD. Since Certain returns a set of output tuples without probabilities, we report its TVD as the fraction of the complete-data answer that it does not return.

We average accuracy, TVD, and Δw over the queries completed by each method. We report the CAMC, CAEX, and QE metrics only for their completed queries. This explains why their metrics may appear better than CADE's when they time out on some queries. The marked-null results use five factorizable MNAR queries of each type. Every method is evaluated on the same applicable queries. For non-repeating nulls, the CADE, CAEX, and QE results use 13 non-aggregation queries and 15 aggregation queries. These queries include MCAR, MAR, and factorizable MNAR data. The non-repeating-null CAMC results use the five factorizable MNAR queries of each type. The non-repeating-null Certain results use ten factorizable MNAR non-aggregation queries. Certain and the

Table 6: Mean evaluation metric on injected data with non-repeating nulls.

Dataset	Query type and metric	CAMC			CAEX			CADE			QE			Certain			Interval-Based			Distribution-Centric		
		5%	10%	20%	5%	10%	20%	5%	10%	20%	5%	10%	20%	5%	10%	20%	5%	10%	20%	5%	10%	20%
Bank Marketing	Non-aggregation (TVD)	0.0366	0.0414	0.0621	0.0045	0.0082	0.0147	0.0379	0.0555	0.0918	0.0207	0.0324	0.0526	0.0605	0.0875	0.1342	N/A	N/A	N/A	N/A	N/A	N/A
	Non-aggregation (Δw)	0.0167	0.0168	0.0164	0.0043	0.0077	0.0160	0.0578	0.0598	0.0597	0.0129	0.0151	0.0180	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
	Aggregation (Accuracy %)	97.1754	96.7483	94.4435	97.8987	97.2473	96.8332	94.6639	93.8429	93.0043	98.0468	96.8794	96.5645	N/A	N/A	N/A	100%	100%	100%	91.1850	90.2300	88.7400
	Aggregation (Δw)	0.0030	0.0043	0.0033	0.2841	0.2776	0.2680	0.2733	0.2801	0.3047	0.0026	0.0033	0.0041	N/A	N/A	N/A	0.5910	0.8825	1.2415	N/A	N/A	N/A
NYC Taxi Trips	Non-aggregation (TVD)	0.0233	0.0366	0.0607	0.0000	0.0000	0.0001	0.0343	0.0571	0.1034	0.0000	0.0000	–	0.0463	0.0764	0.1251	N/A	N/A	N/A	N/A	N/A	N/A
	Non-aggregation (Δw)	0.0077	0.0108	0.0134	0.0041	0.0086	0.0155	0.0002	0.0002	0.0004	0.0049	0.0049	–	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
	Aggregation (Accuracy %)	99.9928	99.9171	99.9887	99.9748	99.9437	99.9494	99.9094	99.7544	99.4262	–	–	–	N/A	N/A	N/A	100%	100%	100%	99.8000	99.7700	99.8150
	Aggregation (Δw)	0.0000	0.0000	0.0000	0.0105	0.0086	0.0084	0.0089	0.0089	0.0095	–	–	–	N/A	N/A	N/A	0.8775	0.9770	1.1370	N/A	N/A	N/A
Bitcoin Heist	Non-aggregation (TVD)	0.1728	0.1825	0.2764	0.0057	0.0102	0.0184	0.0277	0.0463	0.0829	–	–	–	0.1416	0.1902	0.2788	N/A	N/A	N/A	N/A	N/A	N/A
	Non-aggregation (Δw)	0.0064	0.0069	0.0049	0.0025	0.0039	0.0060	0.0001	0.0001	0.0001	–	–	–	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
	Aggregation (Accuracy %)	–	–	–	–	–	–	97.4633	96.9068	95.1164	–	–	–	N/A	N/A	N/A	–	–	–	89.9000	89.2000	87.8500
	Aggregation (Δw)	–	–	–	–	–	–	0.0947	0.0977	0.1008	–	–	–	N/A	N/A	N/A	–	–	–	N/A	N/A	N/A

standalone imputation methods do not return a confidence interval, so Δw does not apply to these methods. We report accuracy and TVD for datasets with injected missingness and ground truth.

10.1 Factorizable MNAR Data Analysis

Marked-Null Experiments. Figure 3 and Table 5 report the results on the five factorizable MNAR queries of each type. CADE completes every query, and Certain completes every non-aggregation query. Interval-Based and Distribution-Centric exceed the five-minute limit on every aggregation query. Neither method applies to non-aggregation queries. CADE has a lower mean execution time than CAMC, QE, Interval-Based, and Distribution-Centric on every dataset and missingness rate. CAMC completes all five queries of each type on Bank Marketing at every missingness rate. Its mean execution time ranges from 13.13 to 37.58 seconds for aggregation queries and from 3.09 to 6.10 seconds for non-aggregation queries. QE also completes every Bank Marketing query. CAMC and QE use the same sampled repairs and return the same evaluation metrics for these queries. On NYC Taxi Trips, CAMC completes one query of each type at 5% missingness and none at 10% or 20% missingness. On Bitcoin Heist, CAMC completes no aggregation query. It completes two Bitcoin Heist non-aggregation queries at 5% missingness, one at 10%, and none at 20%. QE does not complete any NYC Taxi Trips or Bitcoin Heist query. The CAMC and QE metrics on these two datasets are therefore based only on their completed queries. CADE returns an aggregation accuracy above 92% on every dataset and missingness rate. Its TVD is lower than Certain on NYC Taxi Trips and Bitcoin Heist at every missingness rate.

Non-Repeating-Null Experiments. Figure 4 and Table 6 report the results on data with non-repeating nulls. CADE and CAEX complete every non-aggregation query. CADE also completes every aggregation query. Certain completes every non-aggregation query. CADE has a lower mean execution time than CAEX and QE on every dataset and missingness rate. CADE is the only method that completes both query types on every dataset within the timeout. CADE is faster because it skips computing the distributions of tuples with missing separating sets. CAEX has a lower TVD than CADE on every dataset and missingness rate. CADE reduces TVD relative to Certain by 17.6%–80.3%. The largest reductions are on Bitcoin Heist.

The aggregation accuracies of CADE, CAEX, and QE are above 93% for the queries they complete. The interval-based method returns an interval that contains the aggregation result, which gives it 100% accuracy. However, its Δw ranges from 0.5910 to 1.2415 on the completed queries. The distribution-centric method estimates and materializes the join distribution before evaluating the query [45]. This computation increases its execution time. QE evaluates H copies of each query and combines their results using `UNION ALL`. The interval-based and distribution-centric methods exceed the timeout on all factorizable MNAR aggregation queries and some MAR aggregation queries. Their evaluation metrics are therefore computed from their completed MCAR and MAR queries. The non-repeating-null CAMC entries contain only the five factorizable MNAR queries of each type. The other estimator entries also contain the MCAR and MAR queries described in the experiment settings.

Results on Real-World Missingness. For aggregation queries, CADE has a mean execution time of 0.112 seconds, whereas QE requires 79.479 seconds. Their mean Δw values are 0.124 and 0.001, respectively. For non-aggregation queries, CADE has a mean execution time of 0.014 seconds, whereas QE requires 79.663 seconds. Their mean Δw values are 0.057 and 0.128, respectively. QE takes longer because it computes the factor distributions and evaluates $H = 783$ copies of each query. Interval-Based has a mean execution time of 241.658 seconds. Its mean Δw over the completed queries is 1.001. Distribution-Centric has a mean execution time of 240.550 seconds.

10.2 Non-factorizable MNAR Data Analysis

Table 7 reports the results for non-factorizable MNAR data on the injected and real-world datasets. The runtime columns of Table 7 report imputation time. MNAR-Repair reduces the runtime of every imputation method on every dataset. The reductions range from 24% to 76% on the injected datasets. MNAR-Repair with MBI has the lowest runtime on the injected datasets, whereas MNAR-Repair with GAIN has the lowest runtime on the real-world datasets. Averaged over the nine comparisons on the injected datasets, the mean TVD of the MNAR-Repair variants is 0.173, compared with 0.206 for the standalone methods. The standalone methods have lower TVD on Bank Marketing and Bitcoin Heist. MNAR-Repair imputes only the cells it selects, and CADE estimates the remaining missing values from the observed active domain. A value that does not

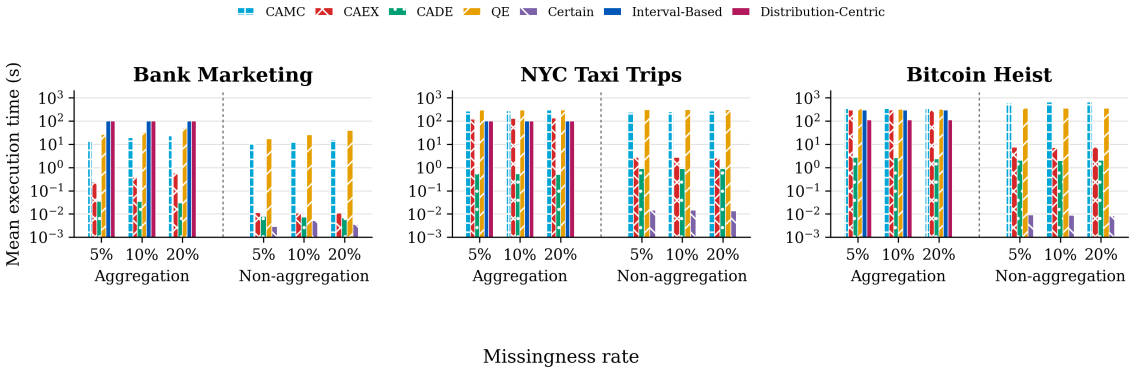


Figure 4: Mean execution time on injected data with non-repeating nulls. The vertical axis is logarithmic.

occur in an observed cell cannot appear in the answer, whereas an imputation method may generate it. The Δw of the MNAR-Repair variants is at most 0.1 on every injected dataset. Hence, MNAR-Repair preserves the quality of the imputation methods, returns confidence intervals, and reduces their runtime.

Minimal Repair Effort. At 20% missingness, MNAR-Repair modifies 32.6% of the missing cells in Bank Marketing and 50.0% in NYC Taxi Trips and Bitcoin Heist.

11 Related Work

Possible answers with probabilistic guarantees. Some methods answer queries on incomplete data by computing possible answers with probabilistic guarantees [5, 17, 18]. These proposals do not account for the causes of missingness and support only a limited set of query types, such as Boolean queries.

Causality in databases. There is extensive research on the modeling of causality in databases [44, 61, 63, 79]. Researchers use causal graphs to detect selection bias in data collection [79] or causal models to detect bias in OLAP queries over binary treatment and outcome attributes by performing independence tests [63]. However, to our knowledge, there has been no work on using the causes of missingness to answer queries over incomplete databases.

Query evaluation in probabilistic databases. A probabilistic database (PDB) represents a probability distribution over possible database instances [66]. In the tuple-independent model, each input tuple has an independent existence event, and safe queries admit efficient *extensional* evaluation, where probabilities are propagated through relational operators [19]. Unsafe queries require *intensional* evaluation: computing the probability of a Boolean formula over tuple events, which is #P-hard in general [19]. Hybrid approaches [34] decompose the query plan so that safe subplans are evaluated extensionally and only the unsafe residual requires symbolic computation. Dissociation-based bounds [22] approximate query probabilities via standard SQL.

Sampling-based probabilistic query evaluation. MCDB [33] introduced the possible-world sampling paradigm with three execution optimizations: lazy instantiation, tuple bundles, and a split operator for joins over uncertain attributes. PIP [36] ports tuple bundles into PostgreSQL with symbolic representations of continuous distributions, and SimSQL [9] scales the paradigm to distributed execution with recursive stochastic tables. Subsequent work extends it to stochastic package optimization [8] and relational probabilistic programming [7]. FastPDB [30] takes a different approach, sampling base-tuple combinations directly from the query lineage, which requires provenance tracking through every relational operator. These systems begin with input uncertainty distributions rather than estimating them from a missingness mechanism. Our framework estimates the required distributions from observed data using the m-graph and factorization (Sections 4 and 5.1). CAMC then uses the MCDB relation encoding to evaluate the repairs sampled from these estimates.

In-database imputation. ImputeDB [11] integrates imputation into the query plan, deciding at each operator whether to impute or drop incomplete tuples. More recently, [56] scales MICE-based imputation within a relational engine by exploiting computation sharing across imputation rounds. These approaches perform imputation within or before query evaluation; MICE-based methods

Table 7: The performance of imputation methods vs. MNAR-Repair on injected and real-world non-factorizable MNAR data

Dataset	Miss%	Standalone GAIN		MNAR-Repair +GAIN			Standalone MICE		MNAR-Repair +MICE			Standalone MissForest		MNAR-Repair +MissForest			MNAR-Repair +MBI		
		Time	TVD	Time	TVD	Δw	Time	TVD	Time	TVD	Δw	Time	TVD	Time	TVD	Δw	Time	TVD	Δw
Injected non-factorizable MNAR																			
Bank Marketing	20%	8.34	0.096	**6.36**	0.099	0.041	4.92	0.099	**1.18**	0.106	0.020	13.05	0.085	**3.63**	0.090	0.045	**0.44**	0.100	0.043
NYC Taxi Trips	20%	91.48	0.632	**61.15**	0.243	0.100	101.57	0.183	**48.57**	0.199	0.092	62.75	0.325	**30.30**	0.205	0.095	**12.64**	0.204	0.092
Bitcoin Heist	20%	337.62	0.145	**226.89**	0.214	0.013	418.10	0.143	**204.73**	0.176	0.002	200.29	0.145	**103.84**	0.226	0.016	**22.94**	0.223	0.000
Real-world non-factorizable MNAR																			
Communities & Crime	13.79%	13.25	–	**11.41**	–	–	113.32	–	**30.16**	–	–	67.70	–	**57.33**	–	–	**39.66**	–	–
NHANES	31.67%	27.84	–	**25.54**	–	–	58.26	–	**51.03**	–	–	53.18	–	**51.17**	–	–	**54.23**	–	–

may produce several completed datasets rather than one. Our extensional and direct estimators instead represent the uncertainty over repairs and estimate query answers directly from the observed data using the m-graph, without materializing a completed relation. CAMC samples repairs when a query is outside the scope of those estimators.

References

[1] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases*. Addison Wesley.

[2] Zeeshan Ahmad. 2025. Student Admission Records. https://www.kaggle.com/datasets/zeeshier/student-admission-records

[3] Constantin F. Aliferis, Alexander Statnikov, and Ioannis Tsamardinos. 2010. Local Causal and Markov Blanket Induction for Causal Discovery and Feature Selection for Classification. Part I: Algorithms and Empirical Evaluation. *Journal of Machine Learning Research* 11 (2010), 171–234.

[4] Lubna Alzamil, Arash Termehchy, and Karthika Mohan. 2026. *Causality-Aware Query Answering on Incomplete Data*. https://research.engr.oregonstate.edu/idea/querying-incomplete-data

[5] Marcelo Arenas, Pablo Barceló, and Mikaël Monet. 2020. Counting problems over incomplete databases. In *Proceedings of the 39th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. 165–177.

[6] Lauren J Beesley, Irina Bondarenko, Michael R Elliot, Allison W Kurian, Steven J Katz, and Jeremy MG Taylor. 2021. Multiple imputation with missing data indicators. *Statistical methods in medical research* 30, 12 (2021), 2685–2700.

[7] Nofar Ben Amara, Samira Hadouaj, and Niccolò Meneghetti. 2024. StarfishDB: A Query Execution Engine for Relational Probabilistic Programming. In *Proceedings of the ACM on Management of Data*, Vol. 2.

[8] Matteo Brucato, Nishant Yadav, Azza Abouzied, Peter J. Haas, and Alexandra Meliou. 2020. Stochastic Package Queries in Probabilistic Databases. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 269–283. https://doi.org/10.1145/3318464.3389765

[9] Zhuhua Cai, Zografoula Vagena, Luis Perez, Subramanian Arumugam, Peter J. Haas, and Christopher Jermaine. 2013. SimSQL: A System for Scalable Stochastic Analytics. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 1133–1136.

[10] Andrea Calì, Domenico Lembo, and Riccardo Rosati. 2003. Query rewriting and answering under constraints in data integration systems. In *Proceedings of the 18th International Joint Conference on Artificial Intelligence (IJCAI)*. 16–21.

[11] José Cambronero, John K. Feser, Micah J. Smith, and Samuel Madden. 2017. Query Optimization for Dynamic Imputation. In *Proceedings of the VLDB Endowment*, Vol. 10. 1310–1321.

[12] George Casella and Roger L. Berger. 2002. *Statistical Inference* (2nd ed.). Duxbury/Thomson Learning, Pacific Grove, CA. See especially Chapter 8 (Properties of Estimators) for combining independent estimators.

[13] Xu Chu, Ihab F Ilyas, Sanjay Krishnan, and Jiannan Wang. 2016. Data Cleaning: Overview and Emerging Challenges. In *Proceedings of the 2016 International Conference on Management of Data*. 2201–2206.

[14] Vasek Chvatal. 1979. A greedy heuristic for the set-covering problem. *Mathematics of operations research* 4, 3 (1979), 233–235.

[15] Cristina Civili and Leonid Libkin. 2018. Approximating Certainty in Querying Data and Metadata. In *Sixteenth International Conference on Principles of Knowledge Representation and Reasoning*.

[16] Marco Console, Paolo Guagliardo, Leonid Libkin, and Etienne Toussaint. 2020. Coping with incomplete data: Recent advances. In *Proceedings of the 39th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. 33–47.

[17] Marco Console, Matthias Hofer, and Leonid Libkin. 2020. Queries with Arithmetic on Incomplete Databases. In *Proceedings of the 39th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. Association for Computing Machinery, 179–189.

[18] Marco Console, Leonid Libkin, and Liat Peterfreund. 2023. Querying incomplete numerical data: Between certain and possible answers. In *Proceedings of the 42nd ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS '23)*. ACM, 349–358. https://doi.org/10.1145/3584372.3588660

[19] Nilesh Dalvi and Dan Suciu. 2007. Efficient query evaluation on probabilistic databases. *The VLDB Journal* 16, 4 (2007), 523–544.

[20] C. J. Date. 2005. *Database in Depth - Relational Theory for Practitioners*. O'Reilly.

[21] Dong Deng, Raul Castro Fernandez, Ziawasch Abedjan, Sibo Wang, Michael Stonebraker, Ahmed K Elmagarmid, Ihab F Ilyas, Samuel Madden, Mourad Ouzzani, and Nan Tang. 2017. The Data Civilizer System.. In *Cidr*.

[22] Wolfgang Gatterbauer and Dan Suciu. 2013. Dissociation and Propagation for Approximate Lifted Inference with Standard Relational Database Management Systems. arXiv:1310.6257 [cs.DB]

[23] Amélie Gheerbrant, Leonid Libkin, and Cristina Sirangelo. 2013. When is naive evaluation possible?. In *Proceedings of the 32nd ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems* (New York, New York, USA) *(PODS '13)*. Association for Computing Machinery, New York, NY, USA, 75–86. https://doi.org/10.1145/2463664.2463674

[24] Georg Gottlob, Nicola Leone, and Francesco Scarcello. 2001. The complexity of query answering in logic programming with incomplete data. *Journal of Artificial Intelligence Research* 12 (2001), 403–432. https://doi.org/10.1613/jair.780

[25] Martin Grohe and Peter Lindner. 2019. Probabilistic Databases with an Infinite Open-World Assumption. In *Proceedings of the 38th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems* (Amsterdam, Netherlands) *(PODS '19)*. Association for Computing Machinery, New York, NY, USA, 17–31. https://doi.org/10.1145/3294052.3319681

[26] Peter J. Haas, Jeffrey F. Naughton, S. Seshadri, and Arun N. Swami. 1996. Selectivity and Cost Estimation for Joins Based on Random Sampling. *J. Comput. System Sci.* 52, 3 (1996), 550–569.

[27] Daniel F Heitjan and Srabashi Basu. 1996. Distinguishing "Missing At Random" and "Missing Completely At Random". *The American Statistician* 50, 3 (1996), 207–213.

[28] Joseph M. Hellerstein. 2008. Quantitative Data Cleaning for Large Databases. *IEEE Data Engineering Bulletin* 23, 4 (2008), 55–61.

[29] Geoffrey E. Hinton. 2002. Training Products of Experts by Minimizing Contrastive Divergence. *Neural Computation* 14, 8 (2002), 1771–1800.

[30] Aaron Huber, Oliver Kennedy, Atri Rudra, Zhuoyue Zhao, Su Feng, and Boris Glavic. 2025. FastPDB: Towards Bag-Probabilistic Queries at Interactive Speeds. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–25.

[31] Rachael A Hughes, Jon Heron, Jonathan AC Sterne, and Kate Tilling. 2019. Accounting for missing data in statistical analyses: multiple imputation is not always the answer. *International journal of epidemiology* 48, 4 (2019), 1294–1304.

[32] Tomasz Imieliński and Witold Lipski. 1984. Incomplete information in relational databases. *Journal of the ACM (JACM)* 31, 4 (1984), 761–791. https://doi.org/10.1145/1634.1886

[33] Ravi Jampani, Fei Xu, Mingxi Wu, Luis Leopoldo Perez, Christopher Jermaine, and Peter J Haas. 2008. Mcdb: a monte carlo approach to managing uncertain data. In *Proceedings of the 2008 ACM SIGMOD international conference on Management of data*. 687–700.

[34] Abhay Jha, Dan Olteanu, and Dan Suciu. 2010. Bridging the gap between intensional and extensional query evaluation in probabilistic databases. In *Proceedings of the 13th International Conference on Extending Database Technology*. 323–334.

[35] Kaggle. 2024. SF Salaries. https://www.kaggle.com/datasets/kaggle/sf-salaries Accessed: 2024-09-18.

[36] Oliver Kennedy and Christoph Koch. 2010. PIP: A Database System for Great and Small Expectations. In *Proceedings of the 26th IEEE International Conference on Data Engineering (ICDE)*. 157–168.

[37] Viktor Leis, Kan Kundhikanjana, Alfons Kemper, and Thomas Neumann. 2015. Efficient Processing of Window Functions in Analytical SQL Queries. *Proceedings of the VLDB Endowment* 8, 10 (2015), 1058–1069.

[38] Leonid Libkin. 2016. SQL's Three-Valued Logic and Certain Answers. *ACM Trans. Database Syst.* 41, 1, Article 1 (March 2016), 28 pages. https://doi.org/10.1145/2877206

[39] Leonid Libkin. 2018. Certain answers meet zero-one laws. In *Proceedings of the 37th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. 195–207.

[40] R.J.A. Little and D.B. Rubin. 2002. *Statistical analysis with missing data*. Wiley. http://books.google.com/books?id=aYPwAAAAMAAJ

[41] Yang Liu and Anthony Constantinou. 2023. Improving the Imputation of Missing Data with Markov Blanket Discovery. In *International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=GrpU6dxFmMN

[42] M. Loève. 1960. *Probability Theory*. Van Nostrand. https://books.google.com/books?id=nI-5tgEACAAJ

[43] Ciobanu Marius. 2025. Medical Condition Prediction Dataset. https://www.kaggle.com/datasets/marius2303/medical-condition-prediction-dataset

[44] Alexandra Meliou, Wolfgang Gatterbauer, S Moore, and Dan Suciu. 2011. The complexity of causality and responsibility for query answers and non-answers. *Proceedings of the VLDB Endowment* 4, 1 (2011), 34–45.

[45] Karthika Mohan, Judea Pearl, and Jin Tian. 2013. Graphical models for inference with missing data. In *Advances in Neural Information Processing Systems (NIPS '13)*, Vol. 26.

[46] Karthika Mohan, Judea Pearl, and Jin Tian. 2013. Missing data as a causal inference problem. In *Proceedings of the neural information processing systems conference (nips)*.

[47] S. Moro, P. Rita, and P. Cortez. 2014. Bank Marketing. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C5K306.

[48] Jeffrey Näf, Erwan Scornet, and Julie Josse. 2024. What Is a Good Imputation Under MAR Missingness? *arXiv preprint arXiv:2403.19196* (2024).

[49] National Center for Health Statistics. n.d. National Health and Nutrition Examination Survey. https://www.cdc.gov/nchs/nhanes/. Accessed July 14, 2026.

[50] Heitor Nunes. 2022. Aircraft Performance. https://www.kaggle.com/datasets/heitornunes/aircraft-performance-dataset-aircraft-bluebook

[51] City of Chicago. 2024. Building Permits. https://data.cityofchicago.org/Buildings/Building-Permits/ydr8-5enu Accessed: 2024-09-18.

[52] City of Chicago. 2024. Current Employee Names, Salaries, and Position Title. https://data.cityofchicago.org/Administration-Finance/Current-Employee-Names-Salaries-and-Position-Title/xzkq-xp2w/about_data Accessed: 2024-09-26.

[53] City of New York. 2024. Street Construction Permits (2022-Present). https://data.cityofnewyork.us/Transportation/Street-Construction-Permits-2022-Present-/tqtj-sjs8 Accessed: 2024-09-18.

[54] Judea Pearl. 1988. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, San Francisco, CA.

[55] Massimo Perini and Milos Nikolic. 2024. In-Database Data Imputation. *Proc. ACM Manag. Data* 2, 1, Article 70 (March 2024), 27 pages. https://doi.org/10.1145/3639326

[56] Massimo Perini and Milos Nikolic. 2024. In-Database Data Imputation. *Proceedings of the ACM on Management of Data* 2, 1 (March 2024), 1–27. https://doi.org/10.1145/3639326

[57] PostgreSQL Global Development Group. 2026. PostgreSQL Documentation: Window Functions. https://www.postgresql.org/docs/current/functions-window.html.

[58] Kamil Pytlak. 2022. Indicators of Heart Disease. https://www.kaggle.com/datasets/kamilpytlak/personal-key-indicators-of-heart-disease Accessed: 2024-09-18.

[59] Erhard Rahm and Hong Hai Do. 2000. Data Cleaning: Problems and Current Approaches. *IEEE Data Engineering Bulletin* 23, 4 (2000), 3–13.

[60] Paris Rohan. 2020. NYC Taxi Trip Duration. https://www.kaggle.com/datasets/parisrohan/nyc-taxi-trip-duration Accessed: 2024-09-30.

[61] Sudeepa Roy and Dan Suciu. 2014. A formal approach to finding explanations for database queries. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data - SIGMOD '14*. ACM, 1579–1590. https://doi.org/10.1145/2588555.2588578

[62] Donald B. Rubin. 1976. Inference and missing data. *Biometrika* 63, 3 (1976), 581–592. https://doi.org/10.1093/biomet/63.3.581

[63] Babak Salimi, Johannes Gehrke, and Dan Suciu. 2018. Bias in OLAP Queries: Detection, Explanation, and Removal. arXiv:1803.04562 [cs.DB] https://arxiv.org/abs/1803.04562

[64] Abraham Silberschatz, Henry F. Korth, and S. Sudarshan. 2020. *Database System Concepts* (7 ed.). McGraw-Hill.

[65] Daniel J Stekhoven and Peter Bühlmann. 2012. MissForest—non-parametric missing value imputation for mixed-type data. *Bioinformatics* 28, 1 (2012), 112–118.

[66] Dan Suciu, Dan Olteanu, Christopher Re, and Christoph Koch. 2011. *Probabilistic Databases*. Morgan & Claypool.

[67] Ruibo Tu, Cheng Zhang, Paul Ackermann, Karthika Mohan, Hedvig Kjellström, and Kun Zhang. 2019. Causal discovery in the presence of missing data. In *The 22nd International Conference on Artificial Intelligence and Statistics*. Pmlr, 1762–1770.

[68] UCI Machine Learning Repository. 2009. Communities and Crime. https://archive.ics.uci.edu/dataset/183/communities+and+crime. Accessed July 14, 2026.

[69] UCI Machine Learning Repository. 2020. Bitcoin Heist Ransomware Address. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C5BG8V.

[70] Stef van Buuren and Karin Groothuis-Oudshoorn. 2011. mice: Multivariate Imputation by Chained Equations in R. *Journal of Statistical Software* 45, 3 (2011), 1–67. https://doi.org/10.18637/jss.v045.i03

[71] Benito van der Zander, Maciej Liśkiewicz, and Johannes Textor. 2014. Constructing Separators and Adjustment Sets in Ancestral Graphs. In *Proc. UAI*.

[72] Jiannan Wang, Sanjay Krishnan, Michael J Franklin, Ken Goldberg, Tim Kraska, and Tova Milo. 2014. A sample-and-clean framework for fast and accurate query processing on dirty data. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data (SIGMOD '14)*. 469–480. https://doi.org/10.1145/2588555.2610505

[73] Ian R White and John B Carlin. 2010. Bias and efficiency of multiple imputation compared with complete-case analysis for missing covariate values. *Statistics in medicine* 29, 28 (2010), 2920–2931.

[74] Taro Yamane. 1973. Statistics: An introductory analysis. (1973).

[75] Jinsung Yoon, James Jordon, and Mihaela van der Schaar. 2018. GAIN: Missing data imputation using generative adversarial nets. In *International Conference on Machine Learning*. PMLR, 5689–5698.

[76] An-Zhen Zhang, Jian-Zhong Li, and Hong Gao. 2019. Interval estimation for aggregate queries on incomplete data. *Journal of Computer Science and Technology* 34, 6 (2019), 1203–1216. https://doi.org/10.1007/s11390-019-1970-4

[77] Guoqiang Zhang and Zhiming Zhang. 2019. Querying and Reasoning with Incomplete Information in Data Management. *Journal of Data Science and Analytics* 5, 2 (2019), 119–135. https://doi.org/10.1007/s41060-019-00132-1

[78] Youran Zhou, Sunil Aryal, and M Reda Bouadjenek. 2024. A Comprehensive Review of Handling Missing Data: Exploring Special Missing Mechanisms. *arXiv preprint arXiv:2404.04905* (2024).

[79] Jiongli Zhu, Sainyam Galhotra, Nazanin Sabri, and Babak Salimi. 2022. Consistent range approximation for fair predictive modeling. *arXiv preprint arXiv:2212.10839* (2022).

A Proof for Section 4

Throughout the appendix, whenever a proof applies the law of large numbers to tuples in the growing-database sequence of Section 3, it assumes that those tuples are independent draws from the fixed relation distribution.

A.1 Factorization Query Consistency

Proof of Proposition 4.2. Consider a factor $(\mathbf{Z}, \mathbf{X})$ and values z and x returned by its factor query. The query's relative frequency converges in probability to

$$\mathbb{P}_T(\mathbf{Z} = z \mid \mathbf{X} = x, R_{\mathbf{Z}} = 0, R_{\mathbf{X}} = 0)$$

as the number of tuples in this observed group grows without bound. The factorization condition $\mathbf{Z} \perp\!\!\!\perp (R_{\mathbf{Z}}, R_{\mathbf{X}}) \mid \mathbf{X}$ makes this conditional probability equal to $\mathbb{P}_T(\mathbf{Z} = z \mid \mathbf{X} = x)$. Thus, every returned factor probability is consistent.

The same argument applies to a factor with an empty separating set. When the last marginal of an independent sum is returned by the preceding factor query, its count must range over all tuples in which that last factor's attributes are observed, rather than only the tuples in which the preceding target attributes are also observed. With this counting rule, the last factor's missingness condition makes the returned marginal consistent. Without this rule, selection of the preceding target attributes may depend on the last separating-set value, and reusing that selected group's frequency need not estimate the marginal.

For a fixed factorization and fixed active domains, Algorithm 1 applies only finitely many joins, products, and sums to these factor estimates. The two independence conditions in the definition of a factorization give the factorization identity

$$\mathbb{P}_T(\mathbf{Y}) = \sum_{\mathbf{V}} \prod_i \mathbb{P}_T(\mathbf{Z}_i \mid \mathbf{X}_i).$$

Splitting this expression into independent sums and changing the elimination order preserves the same finite sum of products. The continuous mapping theorem [12] therefore gives $\ddot{q} \xrightarrow{p} \mathbb{P}_T(\mathbf{Y})$ for every fixed value of $\mathbf{Y}$. We next count the join operations of $\ddot{q}$. Consider an independent sum s_i with $\kappa(i)$ factors. If $\kappa(i) = 1$, Algorithm 1 sets J_i to the factor query of the single factor and uses no join. If $\kappa(i) \geq 2$, the algorithm generates the factor queries $C^i_1, \ldots, C^i_{\kappa(i)-1}$. It generates no query for the last factor, because $C^i_{\kappa(i)-1}$ also returns the marginal of $\mathbf{X}^i_{\kappa(i)-1}$. The algorithm initializes J_i with C^i_1 and joins it with each of $C^i_2, \ldots, C^i_{\kappa(i)-1}$, which uses $\kappa(i) - 2$ joins. Both cases use $\max\{\kappa(i) - 2, 0\}$ joins. The algorithm then joins the queries C_{s_i} of the independent sums, which uses one join fewer than the number of independent sums in $\mathcal{F}(\mathbf{Y})$. Its remaining steps apply a projection or an aggregation to a single query and use no join. Summing these counts over the independent sums gives the number of join operations in Proposition 4.2. □

B Proofs for Section 5

Proof of Theorem 5.1. Consider a predicate Γ_i and a tuple t satisfying Γ_i. If $\mathbf{A}_m^{q,t} = \emptyset$ and t satisfies θ, then $\hat{q}_i$ retains the input tuple marginal $\mathsf{P}(t)$. Otherwise, $\hat{\mathsf{P}}_t(\theta(\mathbf{A}^q))$ is a finite sum of the probabilities returned by C_{Γ_i}. Each probability returned by C_{Γ_i} is computed using sums, products, and normalization of the factor-query estimates. For every probability returned by C_{Γ_i}, the probability used as the denominator of the normalization is greater than zero. The factor-query estimates are consistent by Proposition 4.2. The continuous mapping theorem [12] therefore gives $\hat{\mathsf{P}}_t(\theta(\mathbf{A}^q)) \xrightarrow{p} \mathsf{P}_t(\theta(\mathbf{A}^q))$. The input relation provides the tuple marginal $\mathsf{P}(t)$. Applying the continuous mapping theorem to $\mathsf{P}(t)\hat{\mathsf{P}}_t(\theta(\mathbf{A}^q))$ gives $\mathsf{P}_{\hat{q}}(t) \xrightarrow{p} \mathsf{P}_{q(T)}(t)$. The multivariate central limit theorem gives joint asymptotic normality of the factor-query estimates. The Delta method preserves joint asymptotic normality through the sums, products, and normalization used by the query estimator. The corresponding variance and covariance estimates are consistent by the continuous mapping theorem. The Delta method applied to $\mathsf{P}(t)\hat{\mathsf{P}}_t(\theta(\mathbf{A}^q))$ gives the variance and confidence interval in Theorem 5.1. □

Proof of Theorem 5.2. For tuples $t \in T$ and $s \in S$, Equation 2 gives

$$\mathsf{P}_{q(T,S)}(t \bowtie s) = \mathsf{P}(t)\mathsf{P}(s)\mathsf{P}_\Phi(t,s).$$

The attached marginals $\mathsf{P}(t)$ and $\mathsf{P}(s)$ and the match-probability estimate $\hat{P}_\Phi(t,s)$ consistently estimate the factors in Equation 2. Since multiplication is continuous, the continuous mapping theorem gives $\mathsf{P}_{\hat{q}}(t \bowtie s) \xrightarrow{p} \mathsf{P}_{q(T,S)}(t \bowtie s)$. The Delta method applied to $\mathsf{P}(t)\mathsf{P}(s)\hat{P}_\Phi(t,s)$ gives the variance in Theorem 5.2 when $\mathsf{P}(t)$, $\mathsf{P}(s)$, and $\hat{P}_\Phi(t,s)$ are independently estimated. When $\mathsf{P}(t)$, $\mathsf{P}(s)$, and $\hat{P}_\Phi(t,s)$ are not independently estimated, the variance also contains

$$\begin{aligned}&2\mathsf{P}(t)\mathsf{P}(s)\hat{P}_\Phi(t,s)^2\operatorname{Cov}(\mathsf{P}(t),\mathsf{P}(s))\\&+2\mathsf{P}(t)\mathsf{P}(s)^2\hat{P}_\Phi(t,s)\operatorname{Cov}(\mathsf{P}(t),\hat{P}_\Phi(t,s))\\&+2\mathsf{P}(t)^2\mathsf{P}(s)\hat{P}_\Phi(t,s)\operatorname{Cov}(\mathsf{P}(s),\hat{P}_\Phi(t,s)).\end{aligned}$$

Consistent estimates of the displayed variance and covariance terms give the confidence interval in Theorem 5.2. □

Proof of Theorem 5.3. Consider an output tuple y. For each $t \in T_y$, we define $p_t = \mathsf{P}(t)\mathsf{P}_t(y)$ and $\hat{p}_t = Pr_\pi(t,y)$. The independence condition of the Projection computation rule gives

$$\mathsf{P}_{q(T)}(y) = 1 - \prod_{t\in T_y}(1-p_t) \qquad \text{and} \qquad \mathsf{P}_{\hat{q}}(y) = 1 - \prod_{t\in T_y}(1-\hat{p}_t).$$

For fixed T_y, consistency of every $\hat{p}_t$ and the continuous mapping theorem establish the consistency of $\mathsf{P}_{\hat{q}}(y)$. The derivative of $\mathsf{P}_{q(T)}(y)$ with respect to p_t is

$$\prod_{t'\in T_y\setminus\{t\}}(1-p_{t'}).$$

When the estimates are not independently estimated, the variance in Theorem 5.3 also includes

$$\sum_{t\in T_y}\sum_{t'\in T_y\setminus\{t\}}\left(\prod_{t''\in T_y\setminus\{t\}}(1-\hat{p}_{t''})\right)\times\left(\prod_{t''\in T_y\setminus\{t'\}}(1-\hat{p}_{t''})\right)\operatorname{Cov}(\hat{p}_t,\hat{p}_{t'}).$$

When the estimates are independent, these covariance terms are zero. The Delta method gives the confidence interval in Theorem 5.3. □

B.1 Other Query Operators

B.1.1 Union and Set Difference. **Computation Rule.** Consider relations T and S with the same attributes $\mathbf{Y}$. For a tuple y, let $\mathsf{P}_T(y) = 0$ when y does not occur in T. Define $\mathsf{P}_S(y)$ similarly. If whether y belongs to T is independent of whether y belongs to S, then

$$\mathsf{P}_{T\cup S}(y) = 1 - (1-\mathsf{P}_T(y))(1-\mathsf{P}_S(y))$$

and

$$\mathsf{P}_{T\setminus S}(y) = \mathsf{P}_T(y)(1-\mathsf{P}_S(y))$$

for union and set difference, respectively [19, 66].

Query Rewriting. The input relations provide each tuple and its marginal. For union, define

$$\begin{aligned}C_\cup &:= \pi_{\mathbf{Y},\,term\leftarrow 1,\,Pr\leftarrow \mathsf{P}_T(y)}(T) \cup \pi_{\mathbf{Y},\,term\leftarrow 2,\,Pr\leftarrow \mathsf{P}_S(y)}(S),\\ \hat{q} &:= \gamma_{\mathbf{Y};\,\mathsf{P}_{\hat{q}}(y)\leftarrow 1-\prod(1-Pr)}(C_\cup).\end{aligned}$$

The *term* column prevents equal rows from the two relations from being merged before the product is evaluated. For set difference, define

$$\begin{aligned}C_\setminus &:= \pi_{\mathbf{Y},\,term\leftarrow 1,\,c\leftarrow 1,\,Pr\leftarrow \mathsf{P}_T(y)}(T) \cup \pi_{\mathbf{Y},\,term\leftarrow 2,\,c\leftarrow 0,\,Pr\leftarrow \mathsf{P}_S(y)}(S),\\ C'_\setminus &:= \gamma_{\mathbf{Y};\,c_1\leftarrow \operatorname{sum}(c),\,\mathsf{P}_{\hat{q}'}(y)\leftarrow \operatorname{sum}(c\cdot Pr)\prod(1-(1-c)Pr)}(C_\setminus),\\ \hat{q}' &:= \sigma_{c_1>0\,\wedge\,\mathsf{P}_{\hat{q}'}(y)>0}(C'_\setminus).\end{aligned}$$

The condition $c_1 > 0$ removes tuples absent from T.

Theorem B.1. *Consider $q = T \cup S$ and $q' = T \setminus S$. If the input tuple marginals are consistently estimated and the independence condition of the computation rule holds, every output tuple marginal returned by $\hat{q}$ and $\hat{q}'$ is consistent. For a fixed tuple y, let $\hat{p}_T = \mathsf{P}_T(y)$ and $\hat{p}_S = \mathsf{P}_S(y)$ denote the estimated input tuple marginals. Assume that these estimates are jointly asymptotically normal. If their variances and covariance are consistently estimated, the Delta-method variance estimators are*

$$\begin{aligned}\hat{\sigma}_y^2 &= (1-\hat{p}_S)^2\operatorname{Var}(\hat{p}_T) + (1-\hat{p}_T)^2\operatorname{Var}(\hat{p}_S)\\&\quad+2(1-\hat{p}_S)(1-\hat{p}_T)\operatorname{Cov}(\hat{p}_T,\hat{p}_S),\\ \hat{\sigma}_y'^2 &= (1-\hat{p}_S)^2\operatorname{Var}(\hat{p}_T) + \hat{p}_T^2\operatorname{Var}(\hat{p}_S)\\&\quad-2\hat{p}_T(1-\hat{p}_S)\operatorname{Cov}(\hat{p}_T,\hat{p}_S).\end{aligned}$$

When the estimates are independent, the covariance terms are zero. The intervals $[\mathsf{P}_{\hat{q}}(y) \pm z(\alpha)\hat{\sigma}_y]$ and $[\mathsf{P}_{\hat{q}'}(y) \pm z(\alpha)\hat{\sigma}'_y]$ are asymptotic confidence intervals with probability $1-\alpha$ when their estimated variances are positive.

Proof of Theorem B.1. Consider an output tuple y. Let p_T and p_S be its exact input tuple marginals. Let $\hat{p}_T$ and $\hat{p}_S$ be their estimates. The computation rules use

$$1-(1-\hat{p}_T)(1-\hat{p}_S) \quad \text{and} \quad \hat{p}_T(1-\hat{p}_S)$$

for union and set difference, respectively. Under the independence condition, substituting the exact input tuple marginals gives the exact output tuple marginals. Consistency of the input estimates and the continuous mapping theorem prove consistency of both estimators.

For the confidence intervals, the input estimates must be jointly asymptotically normal. The Delta method applied to each computation rule gives the variance in Theorem B.1. A consistent estimate of the full variance and the Delta method give the stated confidence intervals. □

B.1.2 Aggregation.

Computation Rule. Consider $q = avg(\pi_Y(T))$, where Y is numerical. For each $t \in T$, let $p_t = \mathrm{P}_T(t)$ be its tuple marginal. Extensional evaluation computes the expected SUM and the expected COUNT [19, 66]. When $\sum_{t\in T} p_t > 0$, we define AVG as their ratio:

$$\mu = \frac{\sum_{t\in T} p_t t.Y}{\sum_{t\in T} p_t}.$$

Query Rewriting. The input relation provides each tuple t and its estimated marginal $\hat{p}_t$. The rewriting constructs C_{avg} with one row $(t, \hat{p}_t, \hat{s}_t)$ for every tuple with $\hat{p}_t > 0$, where $\hat{s}_t = \hat{p}_t t.Y$. The aggregation estimator is

$$\hat{q} := \gamma_{\emptyset;\, \hat{\mu}\leftarrow \mathrm{sum}(\hat{s}_t)/\mathrm{sum}(\hat{p}_t)}(C_{avg}).$$

Let $\hat{S} = \sum_{t\in T} \hat{s}_t$ and $\hat{C} = \sum_{t\in T} \hat{p}_t$. Let S and C denote their exact counterparts. The estimator returns an AVG value only when $\hat{C} > 0$.

Theorem B.2. *Consider $q = avg(\pi_Y(T))$. If $\hat{S} \xrightarrow{p} S$, $\hat{C} \xrightarrow{p} C$, and $C > 0$, then $\hat{\mu} = \hat{S}/\hat{C}$ is consistent for S/C. Assume that $\hat{S}$ and $\hat{C}$ are jointly asymptotically normal. If their variances and covariance are consistently estimated, the Delta-method variance estimator is*

$$\hat{\sigma}^2_{avg} = \frac{\mathrm{Var}(\hat{S}) - 2\hat{\mu}\,\mathrm{Cov}(\hat{S},\hat{C}) + \hat{\mu}^2\,\mathrm{Var}(\hat{C})}{\hat{C}^2}.$$

If $\hat{\sigma}^2_{avg} > 0$, then $[\hat{\mu} \pm z(\alpha)\hat{\sigma}_{avg}]$ is an asymptotic confidence interval with probability $1 - \alpha$.

Proof of Theorem B.2. Because $C > 0$, the ratio is continuous at (S, C). The convergence of $\hat{S}$ and $\hat{C}$ and the continuous mapping theorem give $\hat{\mu} = \hat{S}/\hat{C} \xrightarrow{p} S/C$.

Joint asymptotic normality and the Delta method give the variance of the ratio. Substituting $\hat{S}$, $\hat{C}$, and their consistently estimated variances and covariance gives the variance in Theorem B.2. When this estimated variance is positive, the corresponding normal interval has asymptotic probability $1 - \alpha$. □

B.1.3 Möbius Rule. **Computation Rule.** Consider a union of conjunctive queries (UCQ) q with output attributes $\mathbf{Y}$. For each output tuple y, write the condition that y appears in q as $q_1(y) \wedge \cdots \wedge q_k(y)$, where each $q_i(y)$ is a union of conjunctive queries. Every UCQ admits such a form [66]. For each nonempty subset $s \subseteq \{1, \ldots, k\}$, let $q_s(y) = \bigvee_{i\in s} q_i(y)$. Some queries q_s may be logically equivalent. We combine all logically equivalent queries q_s into one union query u. For each resulting query u, define its coefficient as

$$c(u) = \sum_{\substack{\emptyset \neq s \subseteq \{1,\ldots,k\} \\ q_s \equiv u}} (-1)^{|s|+1}.$$

This combination of equivalent terms in inclusion–exclusion is called Möbius inversion [66]. It gives

$$\mathrm{P}_q(y) = \sum_{u:\, c(u)\neq 0} c(u)\, \mathrm{P}_u(y). \tag{11}$$

A query with coefficient 0 does not need to be evaluated.

Query Rewriting. Let $\hat{u}$ estimate the tuple marginals of each resulting union query u, and assign a different integer j_u to each u. When y does not appear in $\hat{u}$, we use $\mathrm{P}_{\hat{u}}(y) = 0$. The query estimator is

$$\hat{q} := \gamma_{\mathbf{Y};\, \mathrm{P}_{\hat{q}}(y)\leftarrow \mathrm{sum}(Pr_\mu)}\left(\bigcup_{u:\, c(u)\neq 0} \pi_{\mathbf{Y},\, term\leftarrow j_u,\, Pr_\mu \leftarrow c(u)\mathrm{P}_{\hat{u}}(y)}(\hat{u}) \right).$$

Here, *term* identifies the union query u that produced each row, so distinct contributions are not merged before the sum.

C Proofs for Section 6

C.1 Marked-Null Distribution Consistency

C.1.1 Per-Symbol Distributions for Marked Nulls. Section 6.1 associates each marked null with one value and estimates its distribution. Distinct marked nulls may have different distributions. Factorizability gives the same conditional distribution of an attribute within a fixed value of its separating set.

Consider an attribute $Y \in \mathbf{A}_m$ with separating set $\mathbf{X}$, and let $\perp_k$ be a marked null occurring in Y. The derivation in this subsection uses the occurrences of $\perp_k$ in Y and assumes that $\perp_k$ does not occur in $\mathbf{X}$. If one marked null occurs in several attributes, Section 6.1 combines its estimated attribute distributions using a product of experts. One value sampled from this distribution is assigned to every occurrence of the marked null. Let $T_{Y,k} = \{t \in T : t[Y] = \perp_k\}$ be the tuples in which Y contains $\perp_k$. For each $t \in T_{Y,k}$ and $x \in \mathrm{adom}(\mathbf{X})$, let $\hat{P}_t(\mathbf{X} = x)$ be the estimated probability that the value of $t[\mathbf{X}]$ is x. If $t[\mathbf{X}]$ is observed, then $\hat{P}_t(\mathbf{X} = x) = 1$ for $x = t[\mathbf{X}]$ and is 0 otherwise. If an attribute of $t[\mathbf{X}]$ is missing, Algorithm 1 computes $\hat{P}_t(\mathbf{X} = x)$ from the factorization of $\mathbf{X}$. This definition applies when $\mathbf{X}$ contains one or several attributes.

The estimated distribution of the separating-set value among the occurrences of $\perp_k$ is

$$\hat{\mathbb{P}}^{\perp_k}_T(\mathbf{X} = x) := \frac{1}{|T_{Y,k}|} \sum_{t\in T_{Y,k}} \hat{P}_t(\mathbf{X} = x). \tag{12}$$

When all occurrences have observed separating-set values, Equation (12) is their empirical fraction. When some separating-set values are missing, each such occurrence contributes its estimated distribution. In either case, $\sum_x \hat{\mathbb{P}}^{\perp_k}_T(\mathbf{X} = x) = 1$.

By factorizability, $Y \perp\!\!\!\perp (R_Y, R_{\mathbf{X}}) \mid \mathbf{X}$. Therefore, the conditional distribution of Y given $\mathbf{X} = x$ is shared by observed cells and cells containing a marked null. The distribution of $\perp_k$ is

$$\hat{\mathbb{P}}^{\perp_k}_T(Y = v) = \sum_{x\in \mathrm{adom}(\mathbf{X})} \hat{\mathbb{P}}_T(Y = v \mid \mathbf{X} = x)\, \hat{\mathbb{P}}^{\perp_k}_T(\mathbf{X} = x), \tag{13}$$

which is Equation 4 in the main text.

Lemma C.1. *Consider a relation T with factorizable MNAR data, an attribute $Y \in \mathbf{A}_m$, and a marked null $\perp_k$ appearing in Y. Let $\mathbf{X}$ be the separating set of Y. Assume that $\perp_k$ does not occur in $\mathbf{X}$. Then $\hat{\mathbb{P}}^{\perp_k}_T(Y)$ is a consistent estimator of $\mathbb{P}^{\perp_k}_T(Y)$.*

Proof. By Proposition 4.2, the factor queries consistently estimate $\mathbb{P}_T(Y = v \mid \mathbf{X} = x)$ and every probability used to compute $\hat{P}_t(\mathbf{X} = x)$ in Equation (12). The augmentation definition keeps the schema, m-graph, and active domains unchanged along the sequence. Therefore, the factor queries return a fixed number of probabilities, and these estimates converge jointly. Replacing these estimates with their probabilities under $\mathbb{P}_T$ in Equation (12) gives $\mathbb{P}_T^{\perp_k}(\mathbf{X} = x)$. The difference between the two averages is bounded by the largest difference between an estimated expression and its probability under $\mathbb{P}_T$. Therefore, $\hat{\mathbb{P}}_T^{\perp_k}(\mathbf{X} = x)$ is consistent for $\mathbb{P}_T^{\perp_k}(\mathbf{X} = x)$ whether the number of occurrences of $\perp_k$ remains fixed or tends to infinity. Equation (13) is a finite sum of products of these consistent estimators. The continuous mapping theorem therefore gives $\hat{\mathbb{P}}_T^{\perp_k}(Y) \xrightarrow{p} \mathbb{P}_T^{\perp_k}(Y)$. □

C.1.2 Marked Nulls Occurring in Several Attributes. Suppose that the same $\perp_k$ occurs in attributes $Y_1, \ldots, Y_r$. For each Y_i, Lemma C.1 applies to the occurrences of $\perp_k$ in Y_i. Therefore, $\hat{\mathbb{P}}_T^{\perp_k}(Y_i = v) \xrightarrow{p} \mathbb{P}_T^{\perp_k}(Y_i = v)$ for every $1 \le i \le r$. Since r is fixed, the continuous mapping theorem gives

$$\prod_{i=1}^{r} \hat{\mathbb{P}}_T^{\perp_k}(Y_i = v) \xrightarrow{p} \prod_{i=1}^{r} \mathbb{P}_T^{\perp_k}(Y_i = v).$$

Summing these products over v' gives

$$\sum_{v' \in \bigcap_{i=1}^{r} \mathrm{adom}(Y_i)} \prod_{i=1}^{r} \hat{\mathbb{P}}_T^{\perp_k}(Y_i = v') \xrightarrow{p} \sum_{v' \in \bigcap_{i=1}^{r} \mathrm{adom}(Y_i)} \prod_{i=1}^{r} \mathbb{P}_T^{\perp_k}(Y_i = v').$$

Assume that the sum on the right is not zero. Applying the continuous mapping theorem to the product-of-experts expression in Section 6.1 gives $\hat{\mathbb{P}}_T^{\perp_k}(v) \xrightarrow{p} \mathbb{P}_T^{\perp_k}(v)$.

C.1.3 Reduction to Non-Repeating Nulls. If $\perp_k$ occurs in exactly one cell of tuple t, then $T_{Y,k} = \{t\}$ and Equation (12) gives $\hat{\mathbb{P}}_T^{\perp_k}(\mathbf{X} = x) = \hat{P}_t(\mathbf{X} = x)$. Thus, an observed separating set contributes one value, while a missing separating set contributes the distribution computed from its factorization. Equation (13) therefore reduces to the tuple-specific distribution used for a non-repeating null.

Proof of Theorem 6.1. Consider a sequence of databases $\{\mathbf{T}_n\}$ as in Section 3. Consider $\mathcal{A} \in \Sigma_{(q,\mathbf{T}_n)}$. Suppose that $H \to \infty$ as $n \to \infty$. For every $\mathbf{T}_n$, the factor queries estimate the distribution of each marked null from the observed data. By Lemma C.1, the estimated distribution is consistent when the marked null occurs in one attribute. Appendix C.1.2 establishes the same result when the marked null occurs in several attributes. After estimating these distributions, Algorithm 2 independently samples the H database repairs $\mathbf{T}_n^{(1)}, \ldots, \mathbf{T}_n^{(H)}$. For each sample index s, the value $\mathbf{1}[q(\mathbf{T}_n^{(s)}) \in \mathcal{A}]$ is 1 exactly when the query result belongs to $\mathcal{A}$. Equation 5 is the average of these H values. By the law of large numbers, $\mathsf{P}_{\hat{q}_H(\mathbf{T}_n)}(\mathcal{A})$ converges as $H \to \infty$ to the probability of $\mathcal{A}$ computed from the estimated marked-null distributions.

Each sum over v includes all possible values of $\perp_k$. When the number of marked nulls is fixed, the consistency of their estimated distributions gives

$$\sum_{T \in \mathbf{T}_n} \sum_{\perp_k \in \mathrm{Null}(T)} \sum_{v} \left| \hat{\mathbb{P}}_T^{\perp_k}(v) - \mathbb{P}_T^{\perp_k}(v) \right| \xrightarrow{p} 0.$$

For every $\mathcal{A} \in \Sigma_{(q,\mathbf{T}_n)}$, the difference between the probabilities computed from the estimated and true marked-null distributions satisfies

$$\left| \mathsf{P}_{\hat{q}(\mathbf{T}_n)}(\mathcal{A}) - \mathsf{P}_{q(\mathbf{T}_n)}(\mathcal{A}) \right| \le \frac{1}{2} \sum_{T \in \mathbf{T}_n} \sum_{\perp_k \in \mathrm{Null}(T)} \sum_{v} \left| \hat{\mathbb{P}}_T^{\perp_k}(v) - \mathbb{P}_T^{\perp_k}(v) \right|.$$

When the number of marked nulls increases with n, we require the same convergence. It follows that the probability of $\mathcal{A}$ computed from the estimated marked-null distributions converges to $\mathsf{P}_{q(\mathbf{T}_n)}(\mathcal{A})$ as $n \to \infty$. Together with the convergence as $H \to \infty$, this proves that $\mathsf{P}_{\hat{q}_H(\mathbf{T}_n)}(\mathcal{A})$ is consistent for $\mathsf{P}_{q(\mathbf{T}_n)}(\mathcal{A})$.

When the probability of $\mathcal{A}$ computed from the estimated marked-null distributions is strictly between 0 and 1, the central limit theorem gives the asymptotic variance in Theorem 6.1. Replacing the probability of $\mathcal{A}$ with Equation 5 gives $\hat{\sigma}^2$ in Theorem 6.1. The confidence interval contains $\mathsf{P}_{q(\mathbf{T}_n)}(\mathcal{A})$ with probability $1 - \alpha$ when

$$\sqrt{H} \sum_{T \in \mathbf{T}_n} \sum_{\perp_k \in \mathrm{Null}(T)} \sum_{v} \left| \hat{\mathbb{P}}_T^{\perp_k}(v) - \mathbb{P}_T^{\perp_k}(v) \right| \xrightarrow{p} 0.$$

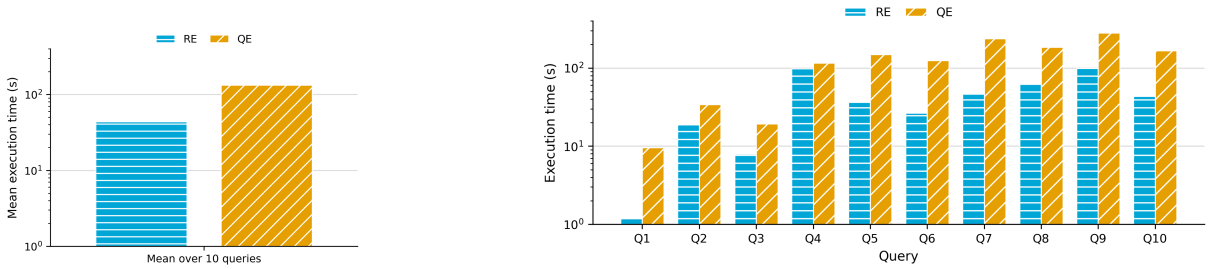
If the probability of $\mathcal{A}$ computed from the estimated marked-null distributions is 0 or 1, all H query results give the same value in Equation 5, and the confidence interval has width zero. □

Proof of Proposition 6.4. For fixed q, Algorithm 1 produces a fixed number of factor queries, and each factor query is evaluated in polynomial time in $|\mathbf{T}|$. RepairSampler processes each distinct marked null once in every repair. The number of distinct marked nulls is at most the number of missing cells in $\mathbf{T}$. When factorization sampling is used, FactorSampler processes the fixed factorization of the attribute that contains the marked null. Its recursive calls process the fixed factorizations of the attributes required by these factors. Thus, FactorSampler performs a fixed number of selections over $\mathbf{T}$ for each marked null, and these selections take polynomial time in $|\mathbf{T}|$. Since H is fixed, sampling and storing the H repairs takes polynomial time and space in $|\mathbf{T}|$. The encoded query evaluates the fixed query q over these repairs and then groups its results. These operations are polynomial in $|\mathbf{T}|$ for fixed q and fixed H. Therefore, CAMC computes $\hat{q}_H$ in polynomial time in $|\mathbf{T}|$. □

Proof of Theorem 6.3. The procedure selects one occurrence of $\perp_k$ uniformly and passes its tuple t to FactorSampler. FactorSampler initializes $t' = t$ and processes $\mathcal{F}(Y)$ in reverse order. For factor $(\mathbf{Z}_i, \mathbf{X}_i)$, the relation T_i^* contains all tuples that match the values of $\mathbf{X}_i$ and $\mathbf{Z}_{i,o}$ in t' and have observed values of $\mathbf{Z}_{i,m}$. Selecting one tuple uniformly returns $\mathbf{Z}_{i,m} = \mathbf{z}_{i,m}$ with probability

$$\hat{\mathbb{P}}_T(\mathbf{Z}_{i,m} = \mathbf{z}_{i,m} \mid \mathbf{X}_i = t'[\mathbf{X}_i], \mathbf{Z}_{i,o} = t'[\mathbf{Z}_{i,o}]).$$

Processing the factors from factor n to factor 1 therefore follows the factorization in reverse order. The values sampled in t' remain fixed while the preceding factors are processed. If a required separating-set value is a repeated marked null, the recursive call obtains its value from its own factorization before the earlier factor is processed. Thus, for each $x \in \mathrm{adom}(\mathbf{X})$, the probability of obtaining



(a) Mean over ten queries.

(b) Execution time for each query.

Figure 5: Execution time of repair encoding (RE) and query encoding (QE) on NYC Taxi Trips with $H = 738$ sampled repairs. The vertical axes are logarithmic.

$Y = v$ after obtaining $\mathbf{X} = x$ is $\hat{\mathbb{P}}_T(Y = v \mid \mathbf{X} = x)$. Equation (12) gives the probability of obtaining $\mathbf{X} = x$ after selecting one occurrence of $\perp_k$ uniformly. Therefore, the probability of obtaining $Y = v$ is

$$\sum_{x \in \mathrm{adom}(\mathbf{X})} \hat{\mathbb{P}}_T(Y = v \mid \mathbf{X} = x)\, \hat{\mathbb{P}}_T^{\perp_k}(\mathbf{X} = x),$$

which equals $\hat{\mathbb{P}}_T^{\perp_k}(Y = v)$ by Equation (13). □

The value of $\mathbf{X}$ sampled from the marginal factors remains fixed in t' when the conditional factor for Y is processed. Therefore, the value of Y is sampled conditional on the same value of $\mathbf{X}$ that is retained for the repaired tuple. After the values of all marked nulls have been sampled, RepairSampler uses these values to construct the complete relation $T^{(s)}$.

C.2 Repair Encoding versus Query Encoding

C.2.1 Query Encoding. Our second implementation follows the query encoding of [18]. For each marked null $\perp_k$ and database repair $\mathbf{T}^{(s)}$, let $v_{k,s}$ be the value sampled for $\perp_k$ in that repair. Instead of materializing each database repair $\mathbf{T}^{(s)}$, the method rewrites q as $q^{(s)}$ using the sampled values $v_{k,s}$. Evaluating $q^{(s)}$ on the incomplete database $\mathbf{T}$ gives the same result as evaluating q on repair $\mathbf{T}^{(s)}$: $q^{(s)}(\mathbf{T}) = q(\mathbf{T}^{(s)})$. The implementation attaches the repair index s to every row returned by $q^{(s)}$. It then unions the results of the H rewritten queries. The union is used to compute the estimates in Equations 5 and 6. The encoded query contains one rewritten copy $q^{(s)}$ for each repair. Therefore, its size grows with H.

Non-aggregation Queries. The estimator is computed as in repair encoding, but $q^{(s)}(\mathbf{T})$ replaces $q(\mathbf{T}^{(s)})$:

$$\hat{q}_H := \gamma_{\bar{\mathbf{A}};\, \mathrm{P}_{\hat{q}}(t) \leftarrow \mathrm{count}(*)/H} \left(\bigcup_{s=1}^{H} \pi_{\bar{\mathbf{A}},\, idx \leftarrow s} \left(q^{(s)}(\mathbf{T}) \right) \right).$$

Comparison with [18]. Both methods sample complete repairs independently, evaluate q on every repair, and estimate a probability by averaging indicators. The method of [18] estimates the probability that the query result contains fewer than, exactly, or more than k tuples whose values belong to intervals specified by the user. This probability is the case of Equation 5 in which $\mathcal{A}$ contains the query results satisfying that condition. Their method assumes that a probability distribution and an efficient sampler are available for every marked null. Our method estimates the marked-null distributions from observed data using the m-graph and factor queries. For an additive error ε, the method samples $\lceil \varepsilon^{-2} \rceil$ repairs and guarantees an absolute error of at most ε with probability at least 0.75. Theorem 6.1 instead gives an asymptotic confidence interval with confidence $1 - \alpha$ from H repairs. For both methods, the variance of the sample average depends on the probability being estimated. However, the additive-error guarantee of [18] uses a fixed absolute error ε that does not change with the estimated probability. Our asymptotic confidence interval uses the estimated variance, so its width is largest at an estimated probability of 0.5 and decreases as the estimate moves away from 0.5.

For a fixed set of H sampled repairs, our query encoding and the query encoding of [18] evaluate the same query results. For each database repair $\mathbf{T}^{(s)}$, both methods construct a rewritten query whose evaluation on the incomplete database equals $q(\mathbf{T}^{(s)})$. Both methods then combine the results of the H rewritten queries. For each repair, the construction of [18] counts the output tuples whose values lie in intervals specified by the user and checks whether this count is less than, equal to, or greater than k. It averages the resulting indicators over the sampled repairs. This condition defines one possible set $\mathcal{A}$ in Equation 5, whereas our method estimates the probabilities of the desired subsets of the answer space, including tuple marginals. Consequently, both query encodings contain one rewritten copy of q for each repair and have the same worst-case dependence on H. The construction of [18] is polynomial in the input size and ε^{-1}, where it uses $\lceil \varepsilon^{-2} \rceil$ repairs. Our repair encoding follows [33] and does not create H rewritten copies of q. It stores an observed cell once and stores the H sampled values of a missing cell in the same input row. It then evaluates one modified query while recording the repairs in which each tuple remains present. Thus, query encoding and repair encoding produce the same H query results, but they represent and evaluate these results differently. The worst-case evaluation time of repair encoding still grows with H, even though the modified query contains only one copy of q. The method of [18] receives a sampler for every marked null, whereas our method obtains the sampled values from observed data. When factor queries compute the marked-null distributions, this computation incurs an additional cost for our method. Factorization

sampling avoids this computation by sampling directly from the observed tuples.

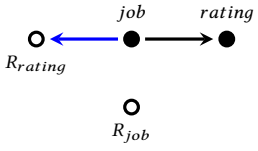
C.2.2 A Comparison Between Repair Encoding and Query Encoding. QE uses query encoding [18] and evaluates H copies of the query. RE uses repair encoding [33] and constructs tuple bundles before evaluating the query. The tuple bundles allow RE to share operations across repairs when sampled values repeat. QE repeats the query evaluation for each sampled repair.

D Direct-Estimation Proofs and Examples

D.1 Join Aggregation Example

Example D.1. Given $q = avg\Big(\pi_{income}\Big(T \bowtie_{T.job=S.job} \sigma_{rating\geq 3}(S)\Big)\Big)$ over *Jobs* (T, Figure 1a) with the m-graph in Figure 1d, and *Ratings* (S, Figure 6b) with the m-graph in Figure 6a. In both relations, *job* is the separating set for the other incomplete query attribute, so $\mathbf{X}^T = \mathbf{X}^S = \{job\}$. As in Section 7.1, the factor queries for S include the condition $rating \geq 3$. Because *job* is also the join attribute, $\hat{\mathbb{P}}_T(\mathbf{A}^T_\Phi = v^T \mid \mathbf{X}^T = x_T)$ is 1 when $v^T = x_T$ and is 0 otherwise. Hence, v^T equals x_T, and we write $\hat{\mu}_{income|x_T}$ for $\hat{\mu}_{income|x_T,v^T}$. The observed *income* values give $\hat{\mu}_{income|\text{Assistant}} = 110\text{K}$, $\hat{\mu}_{income|\text{Executive}} = 250\text{K}$, and $\hat{\mu}_{income|\text{Consultant}} = 340\text{K}/3$. The corresponding values of $\hat{\mathbb{P}}_T(\mathbf{X}^T{=}x_T)$ are 3/11, 3/11, and 5/11. In *Ratings*, the estimated probabilities of $rating \geq 3$ given Assistant, Executive, and Consultant are 0, 1, and 1, respectively. We condition the equal values of $\hat{\mathbb{P}}_S(\mathbf{X}^S{=}x_S)$ on $rating \geq 3$ and obtain $\hat{P}^S_\Phi(\text{Assistant}) = 0$ and $\hat{P}^S_\Phi(\text{Executive}) = \hat{P}^S_\Phi(\text{Consultant}) = 1/2$. We substitute these values into Equation 9 and obtain

$$\hat{q} = \frac{\frac{3}{11}(110)(0) + \frac{3}{11}(250)(\frac{1}{2}) + \frac{5}{11}(\frac{340}{3})(\frac{1}{2})}{\frac{3}{11}(0) + \frac{3}{11}(\frac{1}{2}) + \frac{5}{11}(\frac{1}{2})} = \frac{1975}{12}\text{K} \approx 164.6\text{K}.$$



(a) *Ratings* **m-graph**

job	*rating*
Assistant	2.5
Assistant	$\perp_4$
Executive	4
Executive	3.5
$\perp_5$	$\perp_6$
Consultant	$\perp_7$
Consultant	3

(b) *Ratings* **relation**

Figure 6: *Ratings* **relation and its m-graph**

D.2 Derivation of MNAR Statistics

The expectations $\mathbb{E}[\mathbf{Y}\mathbf{1}_\theta]$ and $\mathbb{E}[\mathbf{Y} \mid \theta]$, and probabilities written with $\mathbb{P}_T$ or $\mathbb{P}_S$, are taken under the relation distributions. The variances and covariances describe the random generation of the growing database sequence from Section 3.

Proof of Proposition 7.1. For each tuple t, we define I_t as 1 when t satisfies θ and 0 otherwise. We let N and D denote the numerator and denominator in

$$\frac{N}{D} = \frac{\sum_{t\in T} t.\mathbf{Y} I_t}{\sum_{t\in T} I_t}.$$

When $D > 0$, N/D is the answer of q. The values are bounded because the active domains are finite. By the law of large numbers, $N/|T| \xrightarrow{p} \mathbb{E}[\mathbf{Y}\mathbf{1}_\theta]$ and $D/|T| \xrightarrow{p} \mathbb{E}[\mathbf{1}_\theta]$. The denominator has a positive limit by the condition that the selected fraction has a positive limit. Consequently, the probability that $D = 0$ converges to zero. Therefore, the continuous mapping theorem [12] gives

$$\frac{N}{D} \xrightarrow{p} \frac{\mathbb{E}[\mathbf{Y}\mathbf{1}_\theta]}{\mathbb{E}[\mathbf{1}_\theta]} = \mathbb{E}[\mathbf{Y} \mid \theta].$$

□

Proof of Proposition 7.4. By the independence assumption for distinct null symbols in Section 4, distinct tuples are independent under non-repeating nulls. We define

$$N = \sum_{(t,s)\in T\times S} t.Y J_{t,s} \quad \text{and} \quad D = \sum_{(t,s)\in T\times S} J_{t,s}.$$

When $D > 0$, the answer of q is N/D. For the convergence statement, assign N/D any fixed value when $D = 0$. The probability that $D = 0$ converges to zero below, so the choice does not change the limit. By linearity of expectation,

$$\mathbb{E}[N] = \sum_{(t,s)} \mathbb{E}[t.Y J_{t,s}] \quad \text{and} \quad \mathbb{E}[D] = \sum_{(t,s)} \mathrm{P}_\mathbf{T}(J_{t,s}{=}1).$$

Linearity requires no independence between the pairs. We define $N' = N/(|T||S|)$ and $D' = D/(|T||S|)$. Two pairs are dependent only when the pairs share a tuple, so each term is dependent on at most $|T| + |S|$ other terms. The values are bounded because the active domain is finite. Hence, $\mathrm{Var}(N') = O(1/|T| + 1/|S|)$ and $\mathrm{Var}(D') = O(1/|T|{+}1/|S|)$, so $N'{-}\mathbb{E}[N'] \xrightarrow{p} 0$ and $D'{-}\mathbb{E}[D'] \xrightarrow{p} 0$. By assumption, the limit of $\mathbb{E}[D']$ is positive. The convergence of D' then implies that the probability that $D = 0$ converges to zero. By the continuous mapping theorem [12], $N/D - \mathbb{E}[N]/\mathbb{E}[D] \xrightarrow{p} 0$; the ratio $\mathbb{E}[N]/\mathbb{E}[D]$ equals the right-hand side of Equation 7. □

Proof of Theorem 7.3. We consider a value x such that $\mathbb{P}_T(\mathbf{X} = x \mid \theta) > 0$. The factorization of $\mathbf{A}^q_m$ evaluates θ through the factor queries of $\mathcal{F}(\mathbf{A}^q_m)$. For the factor containing $\mathbf{Y}$, factorizability gives $\mathbf{Y} \perp\!\!\!\perp (R_\mathbf{Y}, R_\mathbf{X}) \mid \mathbf{X}$. Therefore, the law of large numbers gives

$$\hat{\mu}_{\mathbf{Y}|x,\theta} \xrightarrow{p} \mu_{\mathbf{Y}|x,\theta}.$$

The remaining factor queries are consistent by Proposition 4.2, so

$$\hat{w}_{x|\theta} \xrightarrow{p} w_{x|\theta}.$$

For a fixed number of values x, the continuous mapping theorem [12] gives

$$\sum_x \hat{\mu}_{\mathbf{Y}|x,\theta}\hat{w}_{x|\theta} \xrightarrow{p} \sum_x \mu_{\mathbf{Y}|x,\theta} w_{x|\theta} = \mathbb{E}[\mathbf{Y} \mid \theta].$$

□

Derivation of the variance in Theorem 7.3. Under MNAR, $\hat{w}_{x|\theta}$ is estimated from the factor queries and therefore has a variance. A first-order Taylor expansion of $\hat{q}$ around $\mathbb{E}[\mathbf{Y} \mid \theta]$ uses the following decomposition. For each value x,

$$\begin{aligned}\hat{w}_{x|\theta}\hat{\mu}_{\mathbf{Y}|x,\theta} - w_{x|\theta}\mu_{\mathbf{Y}|x,\theta} = {} & w_{x|\theta}(\hat{\mu}_{\mathbf{Y}|x,\theta} - \mu_{\mathbf{Y}|x,\theta}) \\ & + \mu_{\mathbf{Y}|x,\theta}(\hat{w}_{x|\theta} - w_{x|\theta}) \\ & + (\hat{w}_{x|\theta} - w_{x|\theta})(\hat{\mu}_{\mathbf{Y}|x,\theta} - \mu_{\mathbf{Y}|x,\theta}).\end{aligned}$$

The last term is a product of estimation errors and is omitted by the first-order expansion. Therefore,

$$\hat{q} - \mathbb{E}[\mathbf{Y} \mid \theta] \approx \sum_{x} \left[w_{x|\theta}(\hat{\mu}_{\mathbf{Y}|x,\theta} - \mu_{\mathbf{Y}|x,\theta}) + \mu_{\mathbf{Y}|x,\theta}(\hat{w}_{x|\theta} - w_{x|\theta}) \right].$$

The variance of $\hat{q}$ is the variance of the first-order approximation:

$$\mathrm{Var}(\hat{q}) \approx \textstyle\sum_x \sum_{x'} \mathrm{Cov}\Big(w_{x|\theta}\hat{\mu}_{\mathbf{Y}|x,\theta} + \mu_{\mathbf{Y}|x,\theta}\hat{w}_{x|\theta},\, w_{x'|\theta}\hat{\mu}_{\mathbf{Y}|x',\theta} + \mu_{\mathbf{Y}|x',\theta}\hat{w}_{x'|\theta} \Big). \tag{14}$$

We expand the covariance and obtain

$$\begin{aligned} \mathrm{Var}(\hat{q}) \approx \sum_{x,x'} \Big[& w_{x|\theta} w_{x'|\theta} \,\mathrm{Cov}(\hat{\mu}_{\mathbf{Y}|x,\theta}, \hat{\mu}_{\mathbf{Y}|x',\theta}) \\ & + \mu_{\mathbf{Y}|x,\theta}\mu_{\mathbf{Y}|x',\theta} \,\mathrm{Cov}(\hat{w}_{x|\theta}, \hat{w}_{x'|\theta}) \\ & + w_{x|\theta}\mu_{\mathbf{Y}|x',\theta} \,\mathrm{Cov}(\hat{\mu}_{\mathbf{Y}|x,\theta}, \hat{w}_{x'|\theta}) \\ & + \mu_{\mathbf{Y}|x,\theta} w_{x'|\theta} \,\mathrm{Cov}(\hat{w}_{x|\theta}, \hat{\mu}_{\mathbf{Y}|x',\theta}) \Big]. \end{aligned}$$

The summation is symmetric over x and x', and covariance is symmetric. Thus, the last two terms have the same sum and give the factor 2 in Theorem 7.3. The confidence interval additionally requires the complete vector of group means and group weights to be jointly asymptotically normal and the displayed covariance terms to be consistently estimated. □

Proof of Theorem 7.5. For every x_S and v^S, the factor queries over S consistently estimate the probabilities in Equation 8 whenever the cardinality of every group used by a required factor query converges to infinity. Consequently, Equation 8 consistently estimates the probability that the join attributes of S satisfy Φ with v^T for every v^T. Likewise, the factor queries over T consistently estimate the probabilities in Equation 9. The conditional mean is consistent when restricting to the observed tuples used by the modified factor query does not change the conditional mean and $|\sigma_{\mathbf{Y}\neq\perp \wedge \mathbf{X}^T = x_T \wedge \mathbf{A}_\Phi^T = v^T}(T)| \to \infty$ for every x_T and v^T with positive probability. When the active domains are fixed, the numerator and denominator of Equation 9 are finite sums of products of consistent estimates. The continuous mapping theorem [12] therefore gives convergence of $\hat{\mathbf{N}}$ and $\hat{\mathbf{D}}$. Under the condition of Proposition 7.4, $\hat{\mathbf{D}}$ converges in probability to a value greater than zero. Hence, another application of the continuous mapping theorem gives $\hat{q} = \hat{\mathbf{N}}/\hat{\mathbf{D}} \xrightarrow{p} \mathbb{E}[Y \mid \Phi]$. Proposition 7.4 identifies this limit as the convergent answer of q. If $|\mathrm{adom}(\mathbf{X}^T)| \to \infty$, $|\mathrm{adom}(\mathbf{A}_\Phi^T)| \to \infty$, $|\mathrm{adom}(\mathbf{X}^S)| \to \infty$, or $|\mathrm{adom}(\mathbf{A}_\Phi^S)| \to \infty$, the accumulated estimation error in the numerator and denominator of Equation 9 must converge to zero. □

Variance in Theorem 7.5. We write

$$g(n, d) = n/d, \qquad \hat{q} = g(\hat{\mathbf{N}}, \hat{\mathbf{D}}).$$

The derivatives used in the estimated variance are

$$\frac{\partial g}{\partial n}(\hat{\mathbf{N}}, \hat{\mathbf{D}}) = \frac{1}{\hat{\mathbf{D}}}, \qquad \frac{\partial g}{\partial d}(\hat{\mathbf{N}}, \hat{\mathbf{D}}) = -\frac{\hat{\mathbf{N}}}{\hat{\mathbf{D}}^2} = -\frac{\hat{q}}{\hat{\mathbf{D}}}.$$

The first-order Delta method therefore gives the estimated variance

$$\hat{\sigma}^2 \approx \frac{1}{\hat{\mathbf{D}}^2} \left(\mathrm{Var}(\hat{\mathbf{N}}) + \hat{q}^2 \,\mathrm{Var}(\hat{\mathbf{D}}) - 2\hat{q} \,\mathrm{Cov}(\hat{\mathbf{N}}, \hat{\mathbf{D}}) \right).$$

The covariance is retained because the numerator and denominator use the same factor-query outputs. When the theorem is applied to q_L, these variance and covariance terms include the covariance between $L(t, s)$ and $L(t, s')$ through t and between $L(t, s)$ and $L(t', s)$ through s. Under marked nulls, the terms also include the covariance between joined tuples containing the same marked null. If the factor-query outputs are jointly asymptotically normal and the three variance terms are consistently estimated, the Delta method gives the confidence interval in the theorem. □

D.3 Derivation of MAR Statistics

Proposition D.2. *Consider $q = avg(\pi_{\mathbf{Y}}(\sigma_\theta(T)))$ when every incomplete query attribute has MAR data and a complete separating set $\mathbf{X}$. Assume that, within every selected group $\mathbf{X} = x$, restricting to the observed tuples used by the modified factor query does not change the conditional mean of $\mathbf{Y}$. For every selected group $\mathbf{X} = x$, we let $n_{x|\theta}$ denote the number of observed $\mathbf{Y}$-values in the group that satisfy θ. If $n_{x|\theta} \to \infty$ for every selected group $\mathbf{X} = x$ with $\mathbb{P}_T(\mathbf{X} = x \mid \theta) > 0$, the estimator in Theorem 7.3 converges in probability to the convergent answer of q. Conditional on the factor-query weights and $n_{x|\theta}$, the part of its estimated variance due to the within-group sample means is*

$$\hat{\sigma}^2 = \sum_x \hat{w}_{x|\theta}^2 \frac{s^2_{\mathbf{Y}|x,\theta}}{n_{x|\theta}},$$

where $s^2_{\mathbf{Y}|x,\theta}$ is the sample variance of the observed $\mathbf{Y}$-values in group $\mathbf{X} = x$ satisfying θ.

Proof. Because $\mathbf{X}$ is complete, each tuple belongs to one observed group. The stated restriction ensures that the observed tuples used by the modified factor query have the conditional mean given $\mathbf{X} = x$ and θ. Factorizability supplies this condition when there is no selection and when θ is determined by $\mathbf{X}$. A predicate on any additional attribute requires the stated condition, whether that attribute is complete or incomplete; for an incomplete attribute, it also covers the restriction to its observed values. The law of large numbers gives consistency of each $\hat{\mu}_{\mathbf{Y}|x,\theta}$, and Proposition 4.2 gives consistency of each $\hat{w}_{x|\theta}$. The finite weighted sum therefore converges in probability to the convergent answer of q by the continuous mapping theorem [12].

Under the stated conditioning, the weights are fixed and the sample means from disjoint groups are independent. The variance of the mean in group x is estimated by $s^2_{\mathbf{Y}|x,\theta}/n_{x|\theta}$, which gives the displayed weighted sum. Without this conditioning, the weights are estimated quantities and their variance and covariance must be retained in the general formula of Theorem 7.3. □

Proposition D.3. *Consider the join aggregation query of Theorem 7.5 when every incomplete query attribute has MAR data and a complete separating set in its relation. Assume that, within every T-side separating-set group, the conditional mean of $\mathbf{Y}$ is the same for every join value with positive probability, and that restricting to the observed tuples used by the modified factor query does not change this mean. If the denominator of Equation 9 converges in probability to a value greater than zero and the cardinality of every group used by a required factor query converges to infinity, the join estimator converges in probability to the convergent answer of q. If the factor-query outputs are jointly asymptotically normal and their covariance is*

consistently estimated, its Delta-method variance is the ratio variance in Theorem 7.5.

Proof. Within every observed group of T and S, the join-value probabilities are consistent by the law of large numbers and the factorization conditions. The two mean conditions in the proposition make the modified factor-query means consistent. The numerator and denominator are finite sums of products of these estimates, so each finite sum converges in probability by the continuous mapping theorem. Because the denominator of Equation 9 converges in probability to a value greater than zero, another application of the continuous mapping theorem gives convergence in probability of the join estimator to the convergent answer of q. The numerator and denominator share group weights and join-value probabilities; hence, their covariance need not be zero. We apply the Delta method to their joint covariance and obtain the variance in Theorem 7.5. □

D.4 Negation, Set Difference, and Union

For a negated predicate, we replace θ by $\neg\theta$ in CADE-Bool. The Boolean complement of $q = (\sigma_\theta(T) \neq \emptyset)$ is $q^\neg = (\sigma_\theta(T) = \emptyset)$, with estimator $\hat{q}^\neg = 1 - \hat{q}$.

Corollary 1. *Consider $q = (\sigma_\theta(T) \neq \emptyset)$ with estimator $\hat{q}$ and its Boolean complement $q^\neg = (\sigma_\theta(T) = \emptyset)$ with estimator $\hat{q}^\neg = 1 - \hat{q}$. Then $\hat{q}^\neg$ is consistent for $\mathsf{P}_{q^\neg(T)}(\text{true})$. If Assumption 1 holds, the confidence interval is $[\hat{q}^\neg \pm z(\alpha)\hat{\sigma}]$ with probability $1-\alpha$, where $\hat{\sigma}^2 = \mathrm{Var}(\hat{q}^\neg) = \mathrm{Var}(\hat{q})$.*

For relations T and S with the same attributes $\mathbf{Y}$, we let $\mathsf{P}_T(y)$ and $\mathsf{P}_S(y)$ denote the estimated input marginals of a fixed tuple y.

Corollary 2. *Consider $q' = T \setminus S$ and a fixed $y \in \mathrm{adom}(\mathbf{Y})$. Assume that the occurrence events for y in T and S are independent and that $\mathsf{P}_T(y)$ and $\mathsf{P}_S(y)$ are consistent. Then*

$$\mathsf{P}_{\hat{q}'}(y) = \mathsf{P}_T(y)(1 - \mathsf{P}_S(y)).$$

The estimate $\mathsf{P}_{\hat{q}'}(y)$ is consistent. If the input marginals are jointly asymptotically normal, the confidence interval is $[\mathsf{P}_{\hat{q}'}(y) \pm z(\alpha)\hat{\sigma}'_y]$ with probability $1-\alpha$, where

$$\begin{aligned}\hat{\sigma}'^2_y = {} & (1 - \mathsf{P}_S(y))^2 \,\mathrm{Var}(\mathsf{P}_T(y)) + \mathsf{P}_T(y)^2 \,\mathrm{Var}(\mathsf{P}_S(y)) \\ & - 2\mathsf{P}_T(y)(1 - \mathsf{P}_S(y)) \,\mathrm{Cov}(\mathsf{P}_T(y), \mathsf{P}_S(y)).\end{aligned}$$

Corollary 3. *Consider $q = T \cup S$ and a fixed $y \in \mathrm{adom}(\mathbf{Y})$. Assume that the occurrence events for y in T and S are independent and that $\mathsf{P}_T(y)$ and $\mathsf{P}_S(y)$ are consistent. Then*

$$\mathsf{P}_{\hat{q}}(y) = \mathsf{P}_T(y) + \mathsf{P}_S(y) - \mathsf{P}_T(y)\mathsf{P}_S(y).$$

The estimate $\mathsf{P}_{\hat{q}}(y)$ is consistent. If the input marginals are jointly asymptotically normal, the confidence interval is $[\mathsf{P}_{\hat{q}}(y) \pm z(\alpha)\hat{\sigma}_y]$ with probability $1-\alpha$, where

$$\begin{aligned}\hat{\sigma}^2_y = {} & (1 - \mathsf{P}_S(y))^2 \,\mathrm{Var}(\mathsf{P}_T(y)) + (1 - \mathsf{P}_T(y))^2 \,\mathrm{Var}(\mathsf{P}_S(y)) \\ & + 2(1 - \mathsf{P}_T(y))(1 - \mathsf{P}_S(y)) \,\mathrm{Cov}(\mathsf{P}_T(y), \mathsf{P}_S(y)).\end{aligned}$$

D.5 Non-Aggregation Query Proofs

Proof of Proposition 8.1. For each tuple $t \in T$, we define E_t as the event that t satisfies θ. The query is false exactly when no tuple satisfies θ. Under non-repeating nulls, the tuples are independent by the assumption for distinct null symbols in Section 4. Therefore,

$$\mathsf{P}_{q(T)}(\text{false}) = \mathsf{P}_{\mathbf{T}}\left(\bigcap_{t\in T} E_t^c\right) = \prod_{t\in T} \mathsf{P}_{\mathbf{T}}(E_t^c) = \prod_{t\in T}\big(1 - \mathsf{P}_q(t)\big).$$

□

Proof of Theorem 8.2. By the law of total expectation and the law of large numbers,

$$\mu_L \xrightarrow{p} \sum_x \mathbb{E}[\log(1 - \mathsf{P}_q(t)) \mid \mathbf{X} = x]\mathbb{P}_T(\mathbf{X} = x).$$

The consistency of the factor queries and Theorem 7.3 applied to q_L give the same limit for $\hat{\mu}_L$.

Non-repeating nulls. By Proposition 8.1,

$$\mathsf{P}_{q(T)}(\text{false}) = \mathrm{EXP}(|T|\mu_L).$$

If the limit of μ_L is less than zero, both $|T|\hat{\mu}_L$ and $|T|\mu_L$ converge to $-\infty$ as $|T| \to \infty$. Hence, continuity of the exponential gives $\hat{q} \xrightarrow{p} 1$ and $\mathsf{P}_{q(T)}(\text{true}) \to 1$. If the limit of μ_L is zero, then $\mathsf{P}_q(t) = 0$ with probability one because $\log(1 - \mathsf{P}_q(t)) \leq 0$. Hence, $\hat{q}$ and $\mathsf{P}_{q(T)}(\text{true})$ are both zero. Therefore, $\hat{q}$ is consistent under non-repeating nulls.

Marked nulls. Equation 10 need not hold because tuples t and t' may be dependent when they share a marked null. We consider the fraction of tuples that satisfy θ, $|T|^{-1}\sum_{t\in T} \mathbf{1}_{E_t}$. Its variance is

$$\frac{1}{|T|^2}\sum_{t\in T}\mathrm{Var}(\mathbf{1}_{E_t}) + \frac{1}{|T|^2}\sum_{\substack{t,t'\in T\\ t\neq t'}}\mathrm{Cov}(\mathbf{1}_{E_t}, \mathbf{1}_{E_{t'}}).$$

Each variance and the absolute value of each covariance are at most $1/4$. Two tuples can be dependent through marked nulls only if they share at least one symbol. For each $\perp_k$, at most $|T_k|^2$ ordered tuple pairs share $\perp_k$. If a pair shares several symbols, we may count it more than once, which still gives an upper bound. Therefore, the variance of the fraction is bounded by

$$\frac{1}{4|T|} + \frac{1}{4|T|^2}\sum_{\perp_k \in \mathrm{Null}(T)} |T_k|^2,$$

which converges to zero under Assumption 1. Thus, the fraction of tuples satisfying θ converges in probability to $\mathbb{P}_T(\theta)$. If the limit of μ_L is less than zero, then $\mathbb{P}_T(\theta) > 0$, and

$$\mathsf{P}_{q(T)}(\text{false}) \leq \mathsf{P}_{q(T)}\left(\left|\frac{1}{|T|}\sum_{t\in T}\mathbf{1}_{E_t} - \mathbb{P}_T(\theta)\right| \geq \mathbb{P}_T(\theta)\right) \longrightarrow 0.$$

Since $\hat{\mu}_L$ converges in probability to a value less than zero and $|T| \to \infty$, $\hat{q} = 1 - \mathrm{EXP}(|T|\hat{\mu}_L) \xrightarrow{p} 1$. If the limit of μ_L is zero, then $\mathsf{P}_q(t) = 0$ with probability one, so $\mathsf{P}_{q(T)}(\text{true}) = 0$ and $\hat{q} \xrightarrow{p} 0$. Therefore, $\hat{q}$ is also consistent under marked nulls according to Definition 3.1.

Confidence interval. Under non-repeating nulls, Theorem 7.3 applied to q_L gives the confidence interval for $\hat{\mu}_L$. Under marked nulls, Proposition D.4 applied to q_L with L in place of $\mathbf{Y}$ includes the covariance introduced by repeated marked nulls in $\mathrm{Var}(\hat{\mu}_L)$. We define $g(u) = 1 - \mathrm{EXP}(|T|u)$, so $\hat{q} = g(\hat{\mu}_L)$ and

$$g'(u) = -|T|\mathrm{EXP}(|T|u).$$

The first-order Delta method gives

$$\mathrm{Var}(\hat{q}) \approx |T|^2 \mathrm{EXP}(2|T|\mu_L)\,\mathrm{Var}(\hat{\mu}_L).$$

We replace $\mathrm{EXP}(|T|\mu_L)$ by $1-\hat{q}$ and obtain

$$\hat{\sigma}^2 \approx (1-\hat{q})^2 |T|^2\,\mathrm{Var}(\hat{\mu}_L).$$

Hence, the asymptotic confidence interval under non-repeating and marked nulls is $[\hat{q} \pm z(\alpha)\hat{\sigma}]$ with probability $1-\alpha$. □

Proof of Corollary 1. For $q = (\sigma_{\neg\theta}(T) \neq \emptyset)$, we apply the Boolean estimator of Theorem 8.2 with $\neg\theta$ in place of θ. Therefore, the same consistency and variance derivation applies. For the Boolean complement $q^{\neg} = (\sigma_{\theta}(T) = \emptyset)$, $\hat{q}^{\neg} = 1-\hat{q}$ is a continuous function of a consistent estimator, so it is consistent by the continuous mapping theorem [12]. Its derivative has magnitude 1, so $\mathrm{Var}(\hat{q}^{\neg}) = \mathrm{Var}(\hat{q})$. The confidence interval follows from Theorem 8.2. □

Proof of Corollary 2. Under the independence condition in Corollary 2, the output marginal $\mathsf{P}_{\hat{q}'}(y) = \mathsf{P}_T(y)(1-\mathsf{P}_S(y))$ is a continuous function of two consistent estimators. Consistency follows from the continuous mapping theorem [12]. The stated variance and confidence interval follow by applying the Delta method. □

Proof of Proposition 8.3. For each $y \in \mathrm{adom}(\mathbf{A}_T)$, we define

$$q_y = (\sigma_{\theta \wedge (\mathbf{A}_T = y)}(T) \neq \emptyset).$$

The query q_y is true exactly when y appears in the selection result. We apply Theorem 8.2 to q_y with L_y in place of L and obtain the stated consistency and confidence interval. □

Proof of Proposition 8.4. For each $y \in \mathrm{adom}(\mathbf{Y})$, the query restricted to output y reduces to

$$q_y = (\sigma_{\mathbf{Y}=y}(T) \neq \emptyset).$$

Its estimated tuple marginal is $\mathsf{P}_{\hat{q}_y}(t)$, and $L_y(t) = \log(1-\mathsf{P}_{\hat{q}_y}(t))$. We apply Theorem 8.2 to q_y with L_y in place of L and obtain the stated consistency; the variance follows by the same Delta-method argument. This gives pointwise consistency and a marginal confidence interval for each fixed y. □

Proof of Proposition 8.5. The query is false exactly when no tuple pair satisfies Φ. We consider the fraction of tuple pairs that satisfy Φ,

$$\frac{1}{|T||S|}\sum_{t\in T}\sum_{s\in S} J_{t,s}.$$

Two tuple pairs may be dependent when they share a tuple or contain the same marked null. We define $S_k = \{s \in S : \perp_k \in s\}$, analogously to T_k in Assumption 1. Among the tuple pairs, at most $|T||S|^2$ ordered pairs share a tuple from T, and at most $|S||T|^2$ ordered pairs share a tuple from S. If $\perp_k$ occurs in both relations, at most $|T_k||S| + |T||S_k|$ tuple pairs contain it. Because the variance of each indicator and the absolute value of each covariance are at most $1/4$, the variance of this fraction is bounded by

$$\frac{1}{4}\left(\frac{1}{|T||S|} + \frac{1}{|T|} + \frac{1}{|S|} + \frac{2}{|T|^2}\sum_{\perp_k \in \mathrm{Null}(T)} |T_k|^2 + \frac{2}{|S|^2}\sum_{\perp_k \in \mathrm{Null}(S)} |S_k|^2\right).$$

The first three terms converge to zero as $|T| \to \infty$ and $|S| \to \infty$. We apply Assumption 1 to each relation and use the fixed-schema bound in Proposition D.4, so the last two terms converge to zero. Therefore, the fraction of satisfying tuple pairs converges in probability to its expectation. If this expectation is greater than zero, the probability that the query is false converges to zero:

$$\mathsf{P}_{q(T,S)}(\mathrm{false}) \longrightarrow 0.$$

The same convergence gives $|T \bowtie_\Phi S| \xrightarrow{p} \infty$. We define $\mu_L = \mathbb{E}[L(t,s)]$ under the fixed joint distributions of T and S. Theorem 7.5 applied to q_L with $L(t,s)$ in place of Y gives $\hat{\mu}_L \xrightarrow{p} \mu_L$. The positive expectation gives $\mu_L < 0$. The continuous mapping theorem [12] gives

$$\mathrm{EXP}(\hat{\mu}_L) \xrightarrow{p} \mathrm{EXP}(\mu_L) < 1.$$

Therefore, the Boolean-join estimator gives $1-\hat{q} \xrightarrow{p} 0$. Thus, both $\hat{q}$ and $\mathsf{P}_{q(T,S)}(\mathrm{true})$ converge to 1. If the expectation is zero, every tuple-pair marginal is zero, so both the query and its estimator return false. Therefore, $\hat{q}$ is consistent.

For the confidence interval, Theorem 7.5 applied to q_L gives $\mathrm{Var}(\hat{\mu}_L)$. The variance includes the covariance between joined tuples that share t or s. Under marked nulls, it also includes the covariance between joined tuples containing the same marked null, which converges to zero under Assumption 1 by the bound above. We define $g(u) = 1-\mathrm{EXP}(|T \bowtie_\Phi S|u)$. The first-order Delta method gives

$$\mathrm{Var}(\hat{q}) \approx (1-\hat{q})^2 |T \bowtie_\Phi S|^2\,\mathrm{Var}(\hat{\mu}_L).$$

Therefore, the confidence interval is the interval stated in Proposition 8.5. □

Proof of Proposition 8.6. For each $y \in \mathrm{adom}(\mathbf{A}_T \cup \mathbf{A}_S)$, we use the following Boolean query to determine whether y appears in the join:

$$q_y = (\sigma_{(\mathbf{A}_T \cup \mathbf{A}_S) = y}(T \bowtie_\Phi S) \neq \emptyset).$$

The proof of Proposition 8.5 applies with $L_y(t,s)$ in place of $L(t,s)$. Thus, $\mathsf{P}_{\hat{q}}(y)$ is consistent and has a marginal confidence interval for each fixed y. □

Proof of Corollary 3. We view the membership of y in each input result as a Boolean query. We apply Theorem 8.2 to obtain the consistency of $\mathsf{P}_T(y)$ and $\mathsf{P}_S(y)$. The output marginal $\mathsf{P}_T(y) + \mathsf{P}_S(y) - \mathsf{P}_T(y)\mathsf{P}_S(y)$ is a continuous function of these two estimates. Consistency follows from the continuous mapping theorem [12]. The stated variance and confidence interval follow by applying the Delta method. □

D.6 Marked Nulls in Direct Estimation

Aggregation over one relation. Consider $q = avg(\pi_{\mathbf{Y}}(\sigma_\theta(T)))$ with separating set $\mathbf{X}$. The estimate $\hat{\mu}_{\mathbf{Y}|x,\theta}$ uses tuples for which $\mathbf{Y}$ and $\mathbf{X}$ are observed and θ holds. The conditional independence $\mathbf{Y} \perp\!\!\!\perp (R_{\mathbf{Y}}, R_{\mathbf{X}}) \mid \mathbf{X}$ makes the observed values of $\mathbf{Y}$ a random sample within each group. The factor queries consistently estimate $\mathbb{P}_T(\mathbf{X} = x \mid \theta)$ by Proposition 4.2, including when a tuple contains a marked null in $\mathbf{X}$. Therefore, the estimator remains the one in Theorem 7.3. Tuples that share a marked null may be dependent. The bound in Appendix D.6.1 shows that Assumption 1 makes the additional covariance caused by shared marked nulls converge to zero.

Non-aggregation queries. When a repeated marked null affects θ, tuples that share it need not satisfy θ independently. The marked-null case in the proof of Theorem 8.2 does not use the product in Equation 10. Instead, Assumption 1 bounds the covariance introduced by tuples that share a marked null.

Queries over joins. We apply Assumption 1 separately within each relation. The proof of Proposition 8.5 includes the covariance from joined tuples that share an input tuple or a marked null, including a marked null that occurs in both relations.

D.6.1 Vanishing Covariance for Marked Nulls in Direct Estimation.

Proposition D.4 (Vanishing marked-null covariance). *Consider the single-relation average query of Proposition 7.1 with a fixed schema, finite active domains, and a positive limiting selected fraction. Under Assumption 1, the additional covariance caused by tuples sharing marked-null symbols converges to zero. Consequently, $\hat{q}$ has the convergent answer in Theorem 7.3, and its confidence interval is computed from the covariance expression in that theorem.*

Proof. The active domains are finite, and the expected fraction of tuples satisfying θ has a positive limit by assumption. Hence, the contribution of each tuple to the first-order expansion of the average is bounded by a constant divided by $|T|$. Two tuple contributions can be dependent through marked nulls only if the tuples share at least one symbol. For a distinct symbol $\perp_k$, at most $|T_k|^2$ ordered tuple pairs share that symbol. Pairs that share more than one symbol may be counted more than once, which still gives an upper bound. Thus, the absolute value of the additional covariance is bounded by a constant multiple of

$$\frac{1}{|T|^2} \sum_{\perp_k \in \mathrm{Null}(T)} |T_k|^2.$$

Because the schema is fixed, each tuple occurs in at most $|\mathbf{A}_T|$ of the sets T_k. Consequently,

$$\frac{1}{|T|^2} \sum_{\perp_k} |T_k|^2 \le |\mathbf{A}_T| \max_{\perp_k} \frac{|T_k|}{|T|},$$

which converges to zero by Assumption 1. The factor queries remain consistent by Proposition 4.2. Because the expected fraction of tuples satisfying θ converges to a value greater than zero, the continuous mapping theorem gives the convergent answer in Theorem 7.3. The covariance induced by tuples sharing marked-null symbols is included in the covariance terms of that theorem. □

E Repair Proofs and Algorithms

Let Y be an MAR attribute with complete separating set $\mathbf{X}$. For every nonempty group $\sigma_{\mathbf{X}=x}(T)$, let n_x be its number of tuples, let m_x be the number of tuples in which Y is missing, and let $\omega_x = m_x/n_x$. An *MAR-Repair* imputes some missing values of Y without making any observed value missing. Within each group $\mathbf{X} = x$, it chooses the missing cells to impute uniformly, without using values of attributes outside $\mathbf{X}$. It is an *exact MAR-Repair* if the repaired relation has the same missingness ratio ω in every group $\mathbf{X} = x$. The MAR-Repair-to-MCAR result and Algorithm 6 consider non-repeating nulls. For marked nulls, all occurrences of the same symbol must be repaired together, whereas Algorithm 6 selects individual missing cells.

Theorem E.1. *Under non-repeating nulls, consider a relation T whose tuples are independent draws from $\mathbb{P}_T$, with an MAR attribute Y and a complete separating set $\mathbf{X}$ such that $R_Y \perp\!\!\!\perp (\mathbf{A}_T \setminus \mathbf{X}) \mid \mathbf{X}$. Then Y has MCAR data after an exact MAR-Repair.*

Proof. Let T' be the repaired relation. The probabilities below include both the random generation of T and the uniform random choice of the missing cells that remain unresolved. Consider a group $\mathbf{X} = x$ and condition on its size n_x, its number m_x of missing Y-cells, and the relation-attribute values of its tuples. The independence condition in the theorem makes each of the n_x cells equally likely to be one of these m_x missing cells. Thus, a cell in the group is originally missing with conditional probability m_x/n_x. Let $r_x = \omega n_x$ be the number of missing cells left unresolved by an exact MAR-Repair. When $m_x > 0$, the uniform choice leaves an originally missing cell unresolved with conditional probability r_x/m_x. Consequently, every cell in the group remains missing with conditional probability

$$\frac{m_x}{n_x}\frac{r_x}{m_x} = \frac{r_x}{n_x} = \omega.$$

If $m_x = 0$, feasibility requires $r_x = 0$, and the same equality holds. Because the value ω is the same in every group, this probability is unchanged by any value of $\mathbf{A}_T$. Since R_Y is binary, $R_Y \perp\!\!\!\perp \mathbf{A}_T$, which is the definition of MCAR. For one fixed realized subset, equal group ratios alone would not establish independence from attributes outside $\mathbf{X}$; the uniform randomized choice in the definition is essential. □

Theorem E.2. *Consider unit cost for each imputed cell and let $\omega_{min} = \min_x \omega_x$. Among exact MAR-Repairs, let ω^* be the largest value such that $0 \le \omega^* \le \omega_{min}$ and $\omega^* n_x$ is an integer for every group x. Then the minimum repair cost is attained at ω^* and equals $\sum_x (m_x - \omega^* n_x)$. In particular, $\omega^* = \omega_{min}$ when $\omega_{min} n_x$ is an integer for every group.*

Proof. An exact target ratio ω cannot exceed ω_{min}, because the repair does not introduce new missing values in a group whose current ratio is smaller than ω. It must also satisfy $\omega n_x \in \mathbb{Z}$, since the number of missing cells remaining in each group is an integer. Let $g = \gcd_x n_x$. If every ωn_x is an integer, Bézout's identity gives integers b_x such that $\sum_x b_x n_x = g$, and hence $\omega g = \sum_x b_x(\omega n_x)$ is an integer. Conversely, every nonnegative integer multiple of $1/g$ makes every ωn_x an integer. For any such feasible ratio, group x requires $m_x - \omega n_x$ imputations, so

$$\mathrm{cost}(\omega) = \sum_x (m_x - \omega n_x).$$

This cost decreases as ω increases. It is therefore minimized by the largest feasible value ω^*. If $\omega_{min} n_x$ is an integer for every x, then ω_{min} itself is feasible and is the largest feasible value. □

Proof of Theorem 9.2. Consider the decision version of Minimal MNAR-Repair: given T and an integer k, decide whether at most k incomplete attributes can be repaired so that no missingness mechanism has an incomplete parent. We reduce Vertex Cover to this problem.

Let $G = (V, E)$ and k be an instance of Vertex Cover. For every vertex $v \in V$, create an incomplete attribute A_v with unit repair

cost. For every edge $\{u, v\} \in E$, add the two MNAR edges

$$A_u \longrightarrow R_{A_v} \qquad \text{and} \qquad A_v \longrightarrow R_{A_u}$$

to the m-graph of a relation T_G. No other MNAR edges are added. The construction has $|V|$ incomplete attributes and $2|E|$ MNAR edges, so it is polynomial in the size of G.

Repairing all missing values of A_v makes A_v complete and removes R_{A_v} as the mechanism of an incomplete attribute. Thus, the pair of MNAR edges created for $\{u, v\}$ is eliminated when either A_u or A_v is repaired.

Suppose G has a vertex cover $C \subseteq V$ with $|C| \leq k$. Repair the attributes $\{A_v : v \in C\}$. Every edge $\{u, v\} \in E$ has an endpoint in C, so both MNAR edges created for it are eliminated. The resulting repair has cost at most k.

Conversely, suppose a set $\mathbf{C}$ of at most k attributes eliminates every MNAR edge of T_G. For any $\{u, v\} \in E$, if neither A_u nor A_v were in $\mathbf{C}$, both would remain incomplete and the two MNAR edges created for $\{u, v\}$ would remain. Therefore, $\{v \in V : A_v \in \mathbf{C}\}$ contains at least one endpoint of every edge and is a vertex cover of size at most k.

The two instances therefore have the same answer. This proves NP-hardness for unit repair costs, and the problem with arbitrary positive costs is NP-hard because it contains the unit-cost case. □

Theorem E.3. *Consider a relation T with an MCAR attribute $A \in \mathbf{A}_m$ and $\mathbb{P}_T(R_A = 0) > 0$. Deleting the tuples in which A is missing preserves the joint distribution of $\mathbf{A}_T$ among the retained tuples.*

Proof. Let $\mathbf{B} = \mathbf{A}_T$. Because A has MCAR data, its missingness mechanism is independent of all relation attributes: $R_A \perp\!\!\!\perp \mathbf{B}$. For every value b with positive probability,

$$\begin{aligned}\mathbb{P}_T(\mathbf{B} = b \mid R_A = 0) &= \frac{\mathbb{P}_T(R_A = 0 \mid \mathbf{B} = b)\mathbb{P}_T(\mathbf{B} = b)}{\mathbb{P}_T(R_A = 0)} \\ &= \mathbb{P}_T(\mathbf{B} = b).\end{aligned}$$

Selecting $R_A = 0$ therefore leaves this joint distribution unchanged. □

E.1 MNAR-Repair Guarantee

As in the reduction above, repairing Y eliminates every MNAR edge in which Y is the incomplete parent or R_Y is the missingness mechanism.

Theorem E.4. *Let d be the largest initial number of MNAR edges incident to any incomplete attribute. Algorithm 5 returns an MNAR-Repair whose cost is at most $H_d = \sum_{i=1}^{d} 1/i$ times the minimum repair cost. With adjacency lists and a max-heap, it runs in $O((r + |\mathbf{A}_m|) \log |\mathbf{A}_m|)$ time, where r is the number of MNAR edges.*

Proof. For an MNAR edge $A \to R_B$, repairing either A or B removes the edge: the former makes its parent complete, and the latter removes the missingness mechanism of an incomplete attribute. Thus, associate with each incomplete attribute Y the set of currently uncovered MNAR edges in which Y is either the parent or the attribute of the missingness mechanism. Choosing attributes whose sets cover all MNAR edges is exactly Minimal MNAR-Repair. The gain in Algorithm 5 is the number of newly covered edges, so the algorithm is the weighted set-cover greedy rule. The incident-edge definition of gain is required: counting only outgoing edges would omit the edges removed by repairing the attribute whose missingness mechanism is the child.

When the algorithm chooses Y with gain h, charge $c(Y)/h$ to each of the h newly covered edges. These charges sum to the algorithm's cost. Consider an attribute Y_o in a minimum repair and order its incident edges by the time the greedy algorithm covers them. Immediately before the jth such edge is covered, at least $d_o - j + 1$ of the d_o edges incident to Y_o remain uncovered. If Y_o has not already been chosen, it is an available choice with gain at least $d_o - j + 1$ and cost $c(Y_o)$. The greedy choice therefore charges the jth edge at most $c(Y_o)/(d_o - j + 1)$. If Y_o has already been chosen, none of its incident edges remains uncovered. Consequently, the total charge to edges assigned to Y_o is at most $c(Y_o)H_{d_o} \leq c(Y_o)H_d$. Assign each MNAR edge to one attribute of the minimum repair that covers it and sum this bound over those attributes. The greedy cost is at most H_d times the minimum cost.

For the running time, store the incident edges of every attribute and its current gain. Each edge is removed once and changes the gains of at most its two incident attributes. Each heap update costs $O(\log |\mathbf{A}_m|)$, giving the stated bound. □

E.2 Minimal MAR-Repair Algorithm

Algorithm 6 implements the minimum-cost exact MAR-Repair of Theorem E.2. Its uniform random selection of the unresolved cells is the randomized procedure used in Theorem E.1; one fixed output alone does not establish MCAR. Let g be the greatest common divisor of the group sizes n_x. The values ω for which every ωn_x is an integer are precisely the nonnegative integer multiples of $1/g$. Thus, the largest feasible ratio is $\omega^* = \lfloor g\omega_{min} \rfloor / g$. The algorithm builds the group-statistics map in one pass and then visits each group once more. It takes $O(|T|)$ time, in addition to the cost of imputing the selected cells, and $O(|\pi_{\mathbf{X}}(T)|)$ space for the map. The theorem and algorithm determine which missing cells are imputed; any imputation method may supply the values placed in those cells.

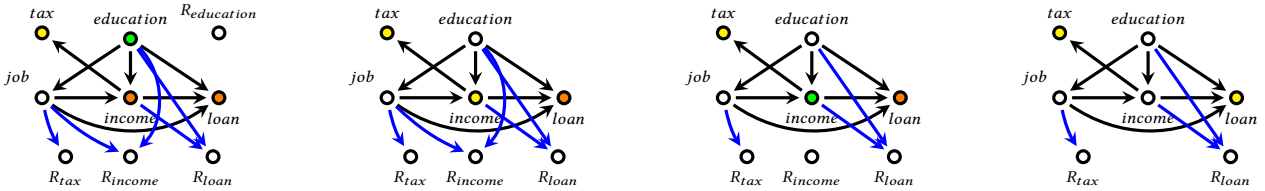
Algorithm 6 Minimal MAR-Repair

Input: relation T, MAR attribute Y, and its complete separating set $\mathbf{X}$
Output: repaired relation T' with missingness ratio ω^* in every group
Function: MinimalMarRepair$(T, Y, \mathbf{X})$

```
Build a map from each x ∈ π_X(T) to (m_x, n_x)
for each x ∈ π_X(T) do
    ω_x ← m_x / n_x
ω_min ← min_x ω_x
g ← gcd{n_x : x ∈ π_X(T)}
ω* ← ⌊g ω_min⌋ / g
T' ← T
for each x ∈ π_X(T) do
    Select m_x − ω* n_x missing Y-cells uniformly in group X = x of T' and impute them
return T'
```

E.3 Sampled MAR-Repair

When ω_{min} is near zero, performing the full minimal MAR-Repair can require substantial imputation. Following the sample-and-clean



(a) Factorizable MNAR m-graph **(b) Resolve missing *education* values** **(c) MAR-Repair on *income*** **(d) Repair *income***

Figure 8: MNAR-Repair example on factorizable MNAR data, where ● MCAR, ● MAR, ● MNAR, ○ Complete

paradigm [72], we avoid repairing the entire relation by performing the repair on a subset sampled uniformly without replacement. Starting from a repaired sample, we evaluate the query's confidence interval. If its width exceeds the threshold specified by the user, we add previously unsampled tuples to an unrepaired version of the current sample, recompute ω_{min}, repair the expanded sample, and evaluate the query again. We call this procedure *cumulative sampling*. The stopping interval must account for both uniform sampling without replacement and the random repair. An interval that accounts for only one source of uncertainty does not establish the stated confidence level for the full-relation answer.

Experimental evaluation. We evaluate the sampled MAR-Repair step in isolation on the SF Salaries dataset [35] (149K tuples, 15.31% MAR missingness). We apply MAR-Repair to the entire dataset, make the repaired attribute MCAR, and evaluate an aggregation query (conditional average), yielding the green flat line in Figure 7. We compare against our cumulative sampling approach, in which we repair a sample, evaluate the query, and compute the interval width. If the width remains high, we expand the unrepaired sample and convert it to MCAR. As the sample grows, the Sampled MAR-Repair estimate (blue) approaches the query result obtained after applying MAR-Repair to the full relation (green). With 1K tuples, the confidence interval is wide; by around 5K tuples, the interval tightens, and the result becomes stable. The 5K-tuple sample contains approximately 3.4% of the relation and produces a result close to the result obtained after repairing the full relation.

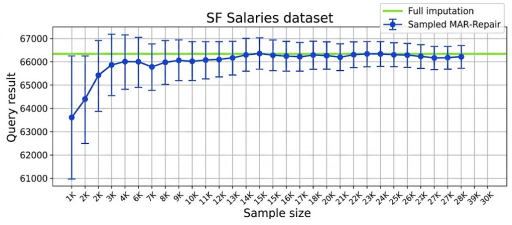


Figure 7: Sampled MAR-Repair convergence on SF Salaries.

E.4 Repairing Factorizable MNAR Data

For factorizable MNAR attributes with large separating sets, breaking MNAR dependencies by repair may cost less than evaluating the group-by operations required by the query estimators. Our MNAR-Repair approach applies to factorizable MNAR data. The following example illustrates the process.

Example E.5. In the m-graph of Figure 8a, the root attribute *education* is MCAR. Resolving its missing values by imputation or by deleting the affected tuples makes *education* complete and yields Figure 8b. The parents of R_{income} are then complete, so *income* becomes MAR. A minimal MAR-Repair makes *income* MCAR, as shown in Figure 8c. Because *income* is still incomplete and is a parent of R_{loan}, *loan* remains MNAR in that panel. Resolving the remaining missing values of *income* makes *income* complete; consequently, every parent of R_{loan} is complete and *loan* becomes MAR in Figure 8d. The final graph has no MNAR attribute. We can apply the MAR query estimators to that graph or apply MAR-Repair again to make its MAR attributes MCAR.

F Distribution-Based Approach

F.1 General Formula

Let Y be a numerical attribute with a finite domain. Its expectation is

$$\mathbb{E}[Y] = \sum_{y} y\, \mathbb{P}_T(Y = y). \tag{15}$$

The sum ranges over the values of Y.

Consider $q = avg(\pi_Y(T))$. Algorithm 1 generates the factorization query $\ddot{q}$ that estimates $\mathbb{P}_T(Y)$. We estimate the convergent answer of q by

$$\hat{q} = \sum_{y} y\, \hat{\mathbb{P}}_T(Y = y). \tag{16}$$

F.2 Estimating Attribute Probabilities

MCAR. If Y is MCAR, then $\mathcal{F}(\{Y\}) = [(\{Y\}, \emptyset)]$. The observed values of Y form a random sample because $\mathbf{A}_T \perp\!\!\!\perp R_Y$. Hence, the factor query computes

$$\hat{\mathbb{P}}_T(Y = y) = \frac{|\sigma_{Y=y}(T)|}{|\sigma_{Y\neq\perp}(T)|}.$$

This estimate is consistent when $\mathbb{P}_T(R_Y = 0) > 0$.

MAR. Suppose Y has a complete separating set $\mathbf{X}$. Then $\mathcal{F}(\{Y\}) = [(\{Y\}, \mathbf{X}), (\mathbf{X}, \emptyset)]$. The factor query computes

$$\hat{\mathbb{P}}_T(Y = y) = \sum_{x} \hat{\mathbb{P}}_T(Y = y \mid \mathbf{X} = x)\, \hat{\mathbb{P}}_T(\mathbf{X} = x). \tag{17}$$

The first factor uses tuples in which Y is observed. The second factor uses all tuples because $\mathbf{X}$ is complete.

Factorizable MNAR. Let $\mathcal{F}(\{Y\}) = [(\mathbf{Z}_1, \mathbf{X}_1), \ldots, (\mathbf{Z}_n, \mathbf{X}_n)]$. The factorization in Section 2 gives [45]

$$\hat{\mathbb{P}}_T(Y = y) = \sum_{\mathbf{V}} \prod_{i=1}^{n} \hat{\mathbb{P}}_T(\mathbf{Z}_i = \mathbf{z}_i \mid \mathbf{X}_i = \mathbf{x}_i). \tag{18}$$

Algorithm 1 computes this estimate. Each factor query uses the observed values of its factor and separating set. The factorization conditions make every factor a consistent estimate [45]. The MCAR and MAR formulas are special cases of Equation 18.

F.3 Average with a Selection

Consider $q = avg(\pi_Y(\sigma_\theta(T)))$. Its convergent answer is $\mathbb{E}[Y \mid \theta]$ by Proposition 7.1. We estimate it by

$$\hat{q} = \sum_y y\, \hat{\mathbb{P}}_T(Y = y \mid \theta) = \frac{\sum_y y\, \hat{\mathbb{P}}_T(Y = y, \theta)}{\hat{\mathbb{P}}_T(\theta)}. \tag{19}$$

Proposition F.1. *Suppose $\ddot{q}$ consistently estimates $\mathbb{P}_T(Y = y, \theta)$ for every y and $\mathbb{P}_T(\theta) > 0$. Then Equation 19 is a consistent estimator of the convergent answer of q.*

Proof. For every y, the estimated joint probability converges to $\mathbb{P}_T(Y = y, \theta)$. The domain of Y is finite. Therefore, the numerator converges to $\sum_y y\, \mathbb{P}_T(Y = y, \theta)$. The denominator converges to $\mathbb{P}_T(\theta)$. Its limit is positive. The ratio therefore converges to $\mathbb{E}[Y \mid \theta]$. □

F.4 Comparison with CADE

Let $(\mathbf{Z}_j, \mathbf{X}_j)$ be the factor that contains Y. The Distribution-Based Approach retains the factor query

$$C_j := \gamma_{\mathbf{X}_j, \mathbf{Z}_j;\, Pr \leftarrow \text{count}(*)/n_j}\big(\sigma_{\mathbf{Z}_j \neq \perp \wedge \mathbf{X}_j \neq \perp \wedge \theta}(T)\big),$$

where n_j is the size of the group $\mathbf{X}_j = \mathbf{x}_j$ in which $\mathbf{Z}_j$ is observed. This query returns one tuple for each observed pair $(\mathbf{x}_j, \mathbf{z}_j)$. Since $Y \in \mathbf{Z}_j$, the query enumerates the observed values of Y within each group. CADE instead uses

$$C_j := \gamma_{\mathbf{X}_j;\, \hat{\mu}_{Y|\mathbf{X}_j,\theta} \leftarrow \text{avg}(Y)}\big(\sigma_{Y \neq \perp \wedge \mathbf{X}_j \neq \perp \wedge \theta}(T)\big).$$

The first query returns at most $|\text{adom}(\mathbf{X}_j)| \cdot |\text{adom}(\mathbf{Z}_j)|$ tuples. The modified query returns at most $|\text{adom}(\mathbf{X}_j)|$ tuples. Consequently, the Distribution-Based Approach can require more time to join the factor-query results.

The same formula applies separately to every group in a group-by query. The probability of the selected tuples in the group must have a positive limit. The estimator computes only the probabilities required by the input query. It does not compute the full distribution $\mathbb{P}_T(\mathbf{A}_T)$.

G Determining Missingness Type

For real-world datasets, we determine the missingness mechanism using the MVPC algorithm [67], which extends the PC algorithm to handle incomplete data. MVPC learns a causal directed acyclic graph (DAG) via conditional independence tests and incorporates the missingness mechanisms into the graph. The faithfulness assumption says that the conditional independences in the data are exactly those represented by separation in the graph. Under this assumption, we classify the type of missingness using the definitions in Section 2: if R_Y has no data-attribute neighbor, the attribute is MCAR; otherwise, if every data-attribute parent of R_Y is complete, it is MAR; otherwise, it is MNAR. Misspecification of the DAG—for example, due to a violation of faithfulness or insufficient data for the independence tests—can lead to incorrect separating sets and biased estimates. In particular, misclassifying the missingness type can invalidate a factorization or its separating sets. Quantifying the sensitivity of query answers to m-graph misspecification is an important direction for future work.

H Evaluation of Factor Queries with CTEs and Window Functions

For a selection condition θ, a factor query computes $\hat{\mathbb{P}}_T(\theta \mid \mathbf{X} = x)$ for each observed value x of the separating set $\mathbf{X}$. The query then assigns this probability to the tuples with $\mathbf{X} = x$. We consider two SQL formulations for this computation. The first formulation places a `GROUP BY` query in an inline CTE and joins its result with the input relation. The second formulation uses `AVG(·) OVER (PARTITION BY X)`. Both formulations compute the same conditional probabilities. We compare their execution times on the NYC Taxi Trips queries.

H.1 Measured Execution Time

Table 8 reports the execution times returned by PostgreSQL `EXPLAIN ANALYZE`. The NYC Taxi Trips relation contains 729,322 tuples and has 5% injected factorizable MNAR missingness with non-repeating nulls. For each method, the two formulations evaluate the same query over the same relation.

Table 8: Execution time of the inline-CTE and window-function formulations on the NYC Taxi Trips queries.

Method	Inline CTE (s)	Window (s)
CAEX	2.08	3.72
CADE	1.40	2.91

H.2 Execution Plans

For the measured queries, PostgreSQL used a grouped aggregate for the inline-CTE formulation. The grouped aggregate first produced one tuple for each value of $\mathbf{X}$. The query then joined this grouped result with the input relation. For the window-function formulation, PostgreSQL sorted the input on $\mathbf{X}$. The window function retained one output tuple for every input tuple while computing the group probability. The inline-CTE formulation was faster for both measured queries. This result applies to the measured PostgreSQL execution plans. Different indexes, data orders, or PostgreSQL settings may produce different execution plans.

H.3 Example

Consider the following relation. The column $\mathbf{1}_\theta$ is one when θ holds and zero otherwise.

tid	X	$\mathbf{1}_\theta$
1	A	1
2	A	0
3	A	1
4	B	0
5	B	1
6	B	0

For each group $\mathbf{X} = x$, the factor query computes

$$\hat{\mathbb{P}}_T(\theta \mid \mathbf{X} = x) = \text{avg}(\mathbf{1}_\theta).$$

The probability is $2/3$ for $\mathbf{X} = A$ and $1/3$ for $\mathbf{X} = B$.

Inline CTE with `GROUP BY`. The inline CTE returns one tuple for each observed value of **X**.

```
WITH group_probability AS (
    SELECT x, AVG(theta::numeric) AS pr
    FROM T
    GROUP BY x
)
SELECT T.tid, T.x, T.theta, C.pr
FROM T
JOIN group_probability AS C
  ON C.x = T.x;
```

The inline CTE returns the following two tuples.

X	*Pr*
A	2/3
B	1/3

The join assigns each probability to the tuples with the corresponding value of **X**. The result therefore contains six tuples.

tid	X	$\mathbf{1}_\theta$	*Pr*
1	A	1	2/3
2	A	0	2/3
3	A	1	2/3
4	B	0	1/3
5	B	1	1/3
6	B	0	1/3

Window function. The window-function formulation computes the same probability for each value of **X**.

```
SELECT tid, x, theta,
       AVG(theta::numeric)
         OVER (PARTITION BY x) AS pr
FROM T;
```

The window function returns the same six-tuple result without a join. It retains every input tuple while computing the probability for its group.

Our implementation uses the inline-CTE formulation for these factor queries. It was faster for the measured queries. An inline CTE also gives each factor query a name in the generated SQL statement. The statement can reference this name wherever the factor query is required. The window-function formulation computes the same conditional probabilities and remains a valid implementation.